



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto “CodeBreaker - Aprende Lenguajes de Programación “

Curso: PATRONES DE SOFTWARE

Docente: ING. PATRICK JOSE CUADROS QUIROGA

Integrantes:

Rosas Chambilla, Jefferson

2021072618

Challo Coaquera Alexsander Wilson

2017057578

Huaman Rivera, Roberto Carlos

2021071077

Tacna – Perú

2026 - II



CodeBreaker - Aprende Lenguajes de Programación

Documento de Especificación de Requerimientos de Software

Versión {1.0}

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo



1.0	Alexsander	Alexsander	Jefferson	10/04/2026	Versión Original
-----	------------	------------	-----------	------------	------------------

ÍNDICE GENERAL

INTRODUCCIÓN	5
I. Generalidades de la Empresa	6
1. Nombre de la Empresa	6
2. Visión	6
3. Misión	6
4. Organigrama	6
5. Cronograma	6
II. Visionamiento de la Empresa	7
1. Descripción del Problema	7
2. Objetivos de Negocios	7
3. Objetivos de Diseño	8
4. Alcance del proyecto	11
5. Funcionalidades Principales	11
6. Viabilidad del Sistema	12
7. Información obtenida del Levantamiento de Información	12
1. Digitalización del Sector de Entretenimiento	21
2. Incremento en la Demanda de Servicios Online	21
3. Mejora en la Experiencia del Usuario	21
4. Potencial de Expansión Comercial	21
5. Integración con Plataformas de Marketing y Redes Sociales	22
6. Fomento del Emprendimiento Local y Colaboración Tecnológica	22
7. Contribución a la Sostenibilidad y Responsabilidad Social	22
III. Análisis de Procesos	22
IV. Especificación de Requerimientos de Software	22
1. Cuadro de requerimientos funcionales Inicial	22
2. Cuadro de requerimientos No funcionales	29
3. Cuadro de Requerimientos funcionales Final	31
4. Reglas de Negocio	37
4.1 Mapa de Trazabilidad de Requisitos	40
5. Otros Requerimientos del Producto	42
V. Fase de Desarrollo	45
1. Perfiles de Usuario	45



2. Modelo Conceptual	46
2.1. Diagrama de Paquetes	46
2.2. Diagrama de Casos de Uso	46
2.3. Diagrama de Casos de Uso - Narrativa	47
CU-01: Registrarse	47
CU-02: Iniciar sesión	47
CU-03: Recuperar contraseña	48
CU-04: Crear consulta de reserva	48
CU-05: Confirmar reserva	49
CU-06: Cancelar reserva	49
CU-07: Publicar opinión	50
CU-08: Gestionar catálogo	50
CU-09: Enviar correos masivos	50
CU-10: Backups automáticos	51
3. Modelo Lógico	51
3.1. Analisis de Objetos	51
3.2. Diagrama de Actividades con objetos	52
3.3. Diagrama de Secuencia	52
3.4. Diagrama de procesos	53
3.5. Diagrama de Clases	75
CONCLUSIONES	76
RECOMENDACIONES	77
REFERENCIAS	77
BIBLIOGRAFÍA	77

INTRODUCCIÓN.....	6
1. Proposito.....	6
2. Alcance.....	7
3. Contexto del producto.....	7
4. Definiciones y acrónimos.....	7
I. Generalidades de la Empresa.....	8
1. Nombre de la Empresa.....	8
2. Visión.....	8
3. Misión.....	8
4. Organigrama.....	8
5. Cronograma.....	9
II. Visionamiento de la Empresa.....	10
1. Descripción del Problema.....	10



2. Objetivos de Negocios.....	10
3. Objetivos de Diseño.....	11
3.1. Usabilidad y Experiencia del Usuario (UX/UI).....	11
3.2. Arquitectura Modular y Escalable.....	11
3.3. Seguridad y Protección de Datos.....	11
3.4. Inteligencia y Automatización.....	12
3.5. Compatibilidad y Portabilidad.....	12
3.6. Rendimiento y Optimización.....	12
3.7. Estética y Consistencia Visual.....	13
3.8. Accesibilidad e inclusión.....	13
3.9. Integración Tecnológica.....	13
3.10. Sostenibilidad y Futuro del Proyecto.....	14
4. Alcance del proyecto.....	14
5. Funcionalidades Principales.....	15
6. Viabilidad del Sistema.....	16
7. Información obtenida del Levantamiento de Información.....	16
7.1 Usuarios Clave Identificados.....	16
7.3 Requerimientos No Funcionales.....	21
7.4 Oportunidades Detectadas.....	22
1. Modernización de la Enseñanza de Programación.....	22
2. Democratización del Acceso a Herramientas de Desarrollo.....	22
3. Feedback Inmediato y Aprendizaje Autogestionado.....	23
4. Cumplimiento de Estándares de Seguridad Educativa.....	23
5. Trazabilidad del Progreso Académico.....	23
6. Fomento de la Innovación Tecnológica en la UPT.....	23
7. Alineación con la Protección de Datos (Ley 29733).....	23
III. Análisis de Procesos.....	23
1. Diagrama del Proceso Actual – Diagrama de Actividades.....	23
2. Diagrama del Proceso Propuesto – (CodeBreaker).....	23
IV. Especificación de Requerimientos de Software.....	25
1. Cuadro de requerimientos funcionales Inicial.....	25
2. Cuadro de requerimientos No funcionales.....	28
3. Cuadro de Requerimientos funcionales Final.....	30
4. Reglas de Negocio.....	30
4.1 Mapa de Trazabilidad de Requisitos.....	31
5. Otros Requerimientos del Producto.....	33
V. Fase de Desarrollo.....	35
1. Perfiles de Usuario.....	35
2. Modelo Conceptual.....	36
2.1. Diagrama de Paquetes	36
2.2. Diagrama de Casos de Uso	37
2.3. Diagrama de Casos de Uso - Narrativa.....	39
a) Escenarios de Caso de Uso (narrativa).....	39



3. Modelo Lógico.....	70
3.1. Analisis de Objetos.....	70
3.2. Diagrama de Actividades con objetos.....	71
3.3. Diagrama de Procesos.....	72
3.4. Diagrama de Secuencia.....	73
Módulo 1: Gestión de Usuarios (Autenticación y Seguridad).....	73
Módulo 2: Aprendizaje y Navegación.....	77
Módulo 3: Ejecución de Código y Sandbox.....	81
Módulo 4: Evaluación y Retroalimentación.....	85
Módulo 5: Persistencia y Gamificación (Procesos Automáticos).....	86
Módulo 6: Experiencia de Usuario (Adaptabilidad).....	90
Módulo 7: Administración de Contenido (Rol Docente/Admin).....	93
3.5. Diagrama de Clases.....	95
CONCLUSIONES.....	96
RECOMENDACIONES.....	97
REFERENCIAS.....	97
BIBLIOGRAFÍA.....	97



CodeBreaker - Aprende Lenguajes de Programación

INTRODUCCIÓN

1. Proposito

Este SRS (Software Requirements Specification) especifica de manera completa y verificable los requisitos del sistema CodeBreaker, una plataforma web educativa diseñada para la enseñanza de lenguajes de programación (Python y PHP) mediante una arquitectura de ejecución aislada.

El sistema permite la gestión de rutas de aprendizaje, retos de programación con validación automática, gamificación (XP y logros) y un entorno de edición de código en tiempo real (Sandbox) basado en la API de Piston. Este documento servirá como el contrato técnico definitivo entre los stakeholders y el equipo de desarrollo para asegurar la integridad, seguridad y escalabilidad del entorno de ejecución asíncrono desarrollado en Node.js.

2. Alcance

El alcance del sistema comprende los siguientes componentes funcionales:

- Entorno de Desarrollo: Implementación de un editor web funcional (Monaco Editor) con soporte para múltiples lenguajes.
- Seguridad y Aislamiento: Ejecución de scripts del lado del usuario sin comprometer el servidor local, delegando el procesamiento a un entorno controlado.
- Gestión Académica: Registro de progreso del estudiante, niveles bloqueados por jerarquía y retroalimentación inmediata sobre errores de sintaxis o lógica.
- Administración: Panel para que el docente/administrador cargue nuevos desafíos y defina los casos de prueba (inputs/outputs esperados).

3. Contexto del producto

CodeBreaker es una aplicación web responsive diseñada bajo una arquitectura de API REST desacoplada. El sistema utiliza un frontend desarrollado en React.js y un backend en Node.js, comunicándose con una base de datos relacional MySQL para la persistencia de usuarios y retos.

4. Definiciones y acrónimos

- Sandbox (Caja de Arena): Entorno de ejecución aislado y seguro que permite correr código



de programación sin riesgo de afectar el sistema anfitrión o el servidor principal.

- Reto (Challenge): Unidad mínima de aprendizaje consistente en un problema lógico que el usuario debe resolver escribiendo código.
- Caso de Prueba (Test Case): Conjunto de entradas (*inputs*) y salidas esperadas (*outputs*) utilizados para validar automáticamente si el código del estudiante es correcto.
- XP (Experience Points): Puntos de experiencia otorgados al usuario tras completar retos, utilizados para el sistema de niveles y gamificación.
- API Piston: Motor de ejecución de código de código abierto utilizado para procesar scripts de Python y PHP en entornos controlados.
- SPA (Single Page Application): Aplicación de una sola página construida con React.js que ofrece una experiencia de usuario fluida sin recargas completas de navegador.
- JWT (JSON Web Token): Estándar para la autenticación segura de usuarios entre el frontend y el backend de Node.js.
- MVP (Mínimo Viable Producto): Versión inicial de CodeBreaker que incluye el editor funcional, la ejecución de código y el sistema básico de usuarios.
- Latencia p95: Métrica de rendimiento que indica que el 95% de las solicitudes de ejecución de código se completan por debajo de un tiempo determinado (umbral de eficiencia).

I. Generalidades de la Empresa

1. Nombre de la Empresa

CodeBreaker Software Learning Labs

Nombre del sistema: Plataforma Educativa CodeBreaker.

2. Visión

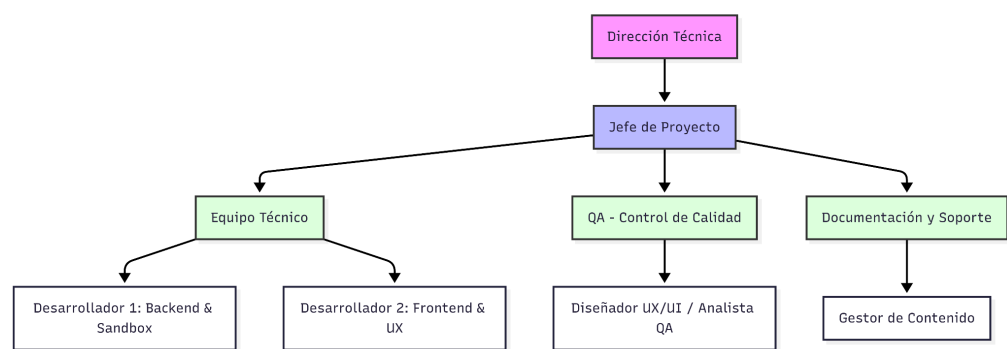
Ser la plataforma líder en la formación de desarrolladores en el sur del Perú, proporcionando un entorno de ejecución de código en la nube que elimine las barreras técnicas de instalación y configuración para estudiantes de ingeniería.



3. Misión

Ofrecer una experiencia de aprendizaje interactiva y segura mediante un sistema de Sandboxing asíncrono, permitiendo que los usuarios resuelvan retos de programación en Python y PHP con retroalimentación instantánea y una progresión gamificada que fomente la excelencia académica.

4. Organigrama



5. Cronograma

Tabla 01

Cronograma de actividades.

CRONOGRAMA			
Fase	Periodo	Duración	Responsable
Inicio y Planificación	16/03/2026 – 31/03/2026	2 semanas	Jefe de Proyecto
Análisis y Diseño de Software (Hito 1)	01/04/2026 – 15/04/2026	2 semanas	Equipo Técnico
Desarrollo del Core (Backend & Sandbox)	16/04/2026 – 10/05/2026	4 semanas	Dev Team (Alex/Jefferson)



Integración Frontend y Gamificación	11/05/2026 – 31/05/2026	3 semanas	Dev Team
Pruebas de Calidad (ISO 25010) y QA	01/06/2026 – 15/06/2026	2 semanas	QA / Tester
Despliegue (CI/CD) y Cierre (Hito Final)	16/06/2026 – 30/06/2026	2 semanas	Dirección Técnica

Nota: Elaboración propia.

II. Visionamiento de la Empresa

1. Descripción del Problema

La enseñanza y práctica de la programación en entornos académicos enfrenta barreras técnicas que limitan el aprendizaje fluido. Actualmente, los estudiantes y la organización experimentan:

- Barreras de Configuración: Pérdida de tiempo significativa instalando compiladores, intérpretes y entornos locales (Python, PHP), lo que desmotiva al principiante.
- Inconsistencia de Entornos: Los errores de código suelen deberse a diferencias de versiones entre el equipo del alumno y el del docente ("en mi máquina sí funciona").
- Falta de Validación Inmediata: Los estudiantes deben esperar la revisión manual del docente para saber si su lógica es correcta, lo que retrasa el ciclo de aprendizaje.
- Ausencia de Trazabilidad: No existe un repositorio centralizado que permita medir el progreso real, los errores comunes y la evolución de las habilidades lógicas del estudiante.
- Riesgos de Seguridad: Ejecutar scripts de estudiantes sin el aislamiento adecuado (Sandboxing) en servidores locales de la facultad es un riesgo de seguridad.

2. Objetivos de Negocios

Los objetivos estratégicos del sistema CodeBreaker para el primer semestre de operación son:



1. Reducción de Barreras: Lograr que el 100% de los usuarios puedan ejecutar su primer script en menos de 30 segundos tras registrarse, sin configuraciones locales.
2. Eficiencia en Evaluación: Automatizar la validación del 90% de los retos propuestos mediante casos de prueba (inputs/outputs) estandarizados.
3. Engagement Académico: Incrementar la resolución de ejercicios voluntarios en un 25% mediante mecánicas de gamificación (XP e insignias).
4. Disponibilidad y Rendimiento: Garantizar una disponibilidad del sistema del 99.5% y una latencia p95 menor a 2 segundos en la respuesta del motor de ejecución de código.
5. Seguridad Técnica: Alcanzar un 0% de incidentes de seguridad en el servidor backend mediante el uso de ejecución aislada en la API de Piston.

3. Objetivos de Diseño

El diseño del sistema CodeBreaker - Aprende Lenguajes de Programación está orientado a los siguientes pilares técnicos y de experiencia:

3.1. Usabilidad y Experiencia del Usuario (UX/UI)

- Minimizar la curva de aprendizaje mediante una interfaz intuitiva, jerarquías visuales claras y flujo guiado (onboarding rápido).
- Diseñar una experiencia responsive compatible con escritorio y dispositivos móviles (Android), asegurando usabilidad con teclados virtuales y navegación táctil.
- Implementar un lenguaje visual consistente (tipografía, contraste, ritmo visual) que refuerce la motivación del estudiante mediante feedback inmediato.
- Integrar el editor Monaco para ofrecer una experiencia tipo IDE: resaltado de sintaxis, autocompletado e indentación.

3.2. Arquitectura Modular y Escalable

- Diseñar una arquitectura modular que permita incorporar nuevas funcionalidades (nuevos retos, módulos educativos, nuevas rutas de aprendizaje) sin afectar el núcleo del sistema.



- Facilitar la evolución del sistema hacia componentes independientes (ej. módulos de contenido, ejecución, progreso y gamificación) y soporte de APIs REST.
- Asegurar mantenibilidad mediante separación de responsabilidades (auth, ejecución, progreso, contenido) y la posibilidad de pruebas unitarias por módulo.

3.3. Seguridad y Protección de Datos

- Proteger credenciales y sesiones mediante hash bcrypt (contraseñas) y tokens JWT para control de acceso.
- Garantizar comunicación cifrada HTTPS/TLS y sanitización de entradas para reducir riesgos de inyección SQL y XSS.
- Registrar trazas de auditoría y eventos críticos (ejecuciones, cambios de progreso, validaciones de retos) para garantizar trazabilidad y soporte pedagógico.
- Ejecutar el código del usuario en un Sandbox aislado mediante Piston API (o mecanismo equivalente), evitando acceso a archivos/red del servidor y controlando timeouts y límites.

3.4. Inteligencia y Automatización

- Automatizar la validación de ejercicios mediante un motor de “juez en línea” que compare la salida obtenida (stdout/stderr) contra resultados esperados.
- Proveer retroalimentación inmediata y descriptiva (mensajes de error y pistas) para mejorar el entendimiento y reducir frustración.
- Mantener un sistema de recomendación/guía pedagógica en función de progreso (niveles completados, errores frecuentes y rendimiento), priorizando rutas de aprendizaje adecuadas.

3.5. Compatibilidad y Portabilidad

- Garantizar compatibilidad con navegadores modernos (Chrome, Edge, Firefox, Safari) y experiencia consistente en Windows, Linux y Android.



- Utilizar estándares abiertos y tecnologías web modernas para facilitar despliegue y migración a entornos cloud.
- Asegurar que el sistema funcione como **SPA** sin depender de instalaciones locales complejas.

3.6. Rendimiento y Optimización

Cumplir con tiempos objetivo para:

- carga inicial,
- latencia de ejecución,
- visualización de resultados en el panel del estudiante,
- transición entre retos.

Optimizar consultas y uso de recursos (persistencia de progreso, XP y logros) mediante un diseño eficiente de base de datos y reglas de negocio del lado del backend.

Integrar respuestas rápidas del motor de ejecución asegurando manejo de errores y tiempos máximos por ejecución.

3.7. Estetica y Consistencia Visual

- Mantener una interfaz moderna y coherente con la identidad del proyecto mediante componentes UI reutilizables (botones, formularios, tableros).
- Usar animaciones suaves para transiciones y retroalimentación (logros, cambios de nivel, estados de ejecución), reforzando engagement sin afectar rendimiento.
- Definir una paleta y estilo consistentes para mejorar comprensión y reducir carga cognitiva del estudiante.

3.8. Accesibilidad e inclusión

- Cumplir al menos con los lineamientos básicos de **WCAG 2.1** (contraste, soporte de navegación por teclado, textos escalables y accesibilidad para lectores de pantalla).
- Diseñar mensajes de validación claros y comprensibles para principiantes, evitando jerga técnica.



- Asegurar que los controles principales sean utilizables mediante interacción táctil sin depender exclusivamente de hover.

3.9. Integración Tecnológica

Integrar y orquestar servicios externos para:

- ejecución aislada de código (Sandbox vía Piston API),
- persistencia de usuarios y progreso (base de datos).

Estandarizar el intercambio de información en JSON entre frontend (React) y backend (Node.js).

3.10. Sostenibilidad y Futuro del Proyecto

- Reducir la dependencia de infraestructura local del estudiante, centralizando la ejecución y la experiencia en la plataforma web.
- Documentar arquitectura, flujos y dependencias para facilitar mantenimiento y escalabilidad.
- Preparar el sistema para crecer hacia nuevas rutas, más lenguajes (además de Python/JS o Python/PHP según roadmap) y mejoras pedagógicas basadas en analítica.

4. Alcance del proyecto

- Editor de Código Web: Integración de un editor profesional (Monaco Editor) con resaltado de sintaxis y autocompletado para Python y PHP.
- Motor de Ejecución (Sandbox): Procesamiento de código en tiempo real mediante la API de Piston, garantizando el aislamiento de procesos y límites de tiempo (*timeouts*).
- Sistema de Rutas de Aprendizaje: Estructura jerárquica de niveles y retos que el usuario debe completar de forma secuencial.
- Validación Automática de Retos: Sistema que compara la salida del código del usuario con los casos de prueba (*inputs/outputs*) definidos por el administrador.
- Gamificación y Progreso: Gestión de puntos de experiencia (XP), barra de progreso por lenguaje y sistema de insignias por logros alcanzados.
- Gestión de Usuarios: Registro, autenticación mediante JWT, recuperación de contraseña y perfil con estadísticas de desempeño.
- Panel de Administración de Retos: Interfaz para que los docentes carguen nuevos desafíos, definan la dificultad y configuren los casos de prueba.



- Reportes y Analíticas Académicas: Visualización de métricas de rendimiento (intentos por reto, tasa de éxito, tiempo promedio de resolución) con exportación en PDF.

Fuera de alcance (Roadmap):

- Aplicación Móvil Nativa: El acceso en dispositivos móviles será exclusivamente vía navegador web (PWA).
- Ejecución de Código Offline: El sistema requiere conexión obligatoria a internet para interactuar con la API de ejecución.
- Soporte para Entornos Gráficos: El sandbox solo soportará ejecución de código por consola (Standard Input/Output), no interfaces gráficas o desarrollo de videojuegos complejos.
- Inteligencia Artificial Generativa: La asistencia de IA para la resolución de retos (tipo Copilot) se considera para versiones posteriores a la 2.0.

5. Funcionalidades Principales

1. Editor de Código Multilenguaje: Interfaz basada en Monaco Editor que permite la escritura de código con resaltado de sintaxis, autocompletado e indentación automática para Python y PHP.
2. Motor de Ejecución Aislada (Sandbox): Ejecución de scripts en tiempo real mediante el consumo de la API de Piston, asegurando que el código del usuario no afecte la integridad del servidor principal.
3. Sistema de Rutas de Aprendizaje: Estructura de niveles jerárquicos donde el acceso a temas avanzados está condicionado a la resolución exitosa de retos básicos.
4. Validación Automática de Resultados: Comparación asíncrona entre el resultado de la ejecución del usuario y los casos de prueba (*inputs/outputs*) configurados para cada reto.
5. Panel de Control de Gamificación: Visualización en tiempo real de puntos de experiencia (XP), nivel actual, insignias obtenidas y posición en el ranking de usuarios.
6. Gestión de Retos (CRUD Administrativo): Herramienta para que el administrador cree, edite y elimine desafíos, asignando niveles de dificultad, lenguajes permitidos y parámetros de validación.



7. Consola de Salida Dinámica: Área de visualización de resultados y errores de compilación/interpretación que proporciona retroalimentación técnica inmediata al estudiante.
8. Gestión de Sesiones y Perfiles: Registro y autenticación segura mediante JWT, permitiendo al usuario guardar su progreso y retomar retos pendientes desde cualquier dispositivo.
9. Analítica Académica: Panel de métricas para visualizar el tiempo promedio de resolución, tasa de error por lenguaje y progreso porcentual de cada curso.
10. Generación de Reportes de Desempeño: Exportación en formato PDF de certificados de nivel y resúmenes de habilidades adquiridas para el portafolio del estudiante.

6. Viabilidad del Sistema

El análisis de viabilidad de CodeBreaker confirma que el proyecto es realizable bajo los estándares de calidad de la Escuela de Ingeniería de Sistemas, basándose en los siguientes pilares:

- Viabilidad Técnica: El equipo cuenta con dominio en el entorno de ejecución Node.js y el framework Express. El riesgo de seguridad asociado a la ejecución de código de terceros se mitiga mediante el uso de Sandboxing a través de la API de Piston, garantizando un aislamiento total de los scripts en Python y PHP.
- Viabilidad Económica: Según el análisis financiero, el proyecto es sostenible bajo un modelo de suscripción o implementación institucional, con indicadores de rentabilidad positivos y un retorno de inversión proyectado dentro del ciclo académico.
- Viabilidad Operativa: La metodología de desarrollo incremental por hitos asegura entregas funcionales en los meses de abril, mayo y junio, permitiendo un control de calidad constante bajo la norma ISO/IEC 25010.

7. Información obtenida del Levantamiento de Información

Métodos usados: Entrevistas semiestructuradas con docentes de programación, encuestas a estudiantes de la facultad (N=18), revisión de errores comunes en laboratorios de los últimos 6 meses, benchmark de 3 plataformas educativas (LeetCode, HackerRank, Exercism) y análisis de puntos de fricción en la configuración de entornos locales.



7.1 Usuarios Clave Identificados

- Estudiante (Visitante/Usuario): Busca un entorno donde pueda escribir código sin instalar nada; valora la retroalimentación inmediata, el sistema de niveles y la claridad en los enunciados de los retos.
- Docente / Administrador de Contenido: Necesita centralizar la gestión de ejercicios, configurar casos de prueba automáticos, monitorear el progreso de los estudiantes y asegurar que el servidor sea inmune a código malicioso.
- Equipo Técnico (Soporte): Requiere un sistema que permita escalar la capacidad de ejecución (API Piston) y monitorear la latencia p95 de las respuestas del backend en Node.js.

Principales tareas por usuario

- Estudiante: 1. Explorar rutas de aprendizaje y retos disponibles. 2. Escribir y ejecutar código en el Sandbox con respuesta en tiempo real. 3. Visualizar progreso (XP, niveles e insignias).
- Docente / Administrador: 1. Crear y configurar nuevos retos (CRUD) con sus respectivos casos de prueba. 2. Moderar y validar el contenido académico subido a la plataforma. 3. Revisar métricas de desempeño y tasas de error de los estudiantes.
- Equipo Técnico: 1. Monitorear la conectividad y el tiempo de respuesta del motor de ejecución. 2. Asegurar la integridad de la base de datos relacional MySQL. 3. Gestionar el despliegue continuo (CI/CD) para actualizaciones de seguridad.

7.2 Requerimientos Funcionales

Código	Nombre	Descripción	Actores	Prioridad	Criterios de Aceptación	Dependencias
RF-01	Registro de usuario	Registro vía email/password u OAuth (Google/GitHub).	Público	Alta	Cuenta creada con éxito y datos persistidos en MySQL tras validación.	-
RF-02	Inicio de sesión	Autenticación mediante tokens JWT.	Público	Alta	Acceso concedido al dashboard y token generado correctamente.	RF-01



RF-03	Cierre de sesión	Invalidación del token JWT activo.	Estudiante	Media	El token queda inutilizable y se redirige al login.	RF-02
RF-04	Perfil de usuario	Muestra de avatar, nivel, XP e insignias.	Estudiante	Media	Los datos visualizados coinciden con el estado actual en la BD.	RF-02, RF-05
RF-05	Guardado de progreso	Persistencia de niveles y XP en la base de datos.	Sistema	Alta	El progreso se guarda automáticamente tras completar un reto.	RF-16
RF-06	Editor integrado	Editor con resaltado y autocompletado (Monaco).	Estudiante	Alta	El editor carga los lenguajes Python/PHP y asiste en la escritura.	-
RF-07	Ejecución en sandbox	Ejecución aislada vía API de Piston.	Estudiante	Alta	El código se ejecuta en la nube sin comprometer el servidor backend.	RF-06
RF-08	Visualización resultados	Muestra de stdout y stderr en panel.	Estudiante	Alta	Los resultados o errores del intérprete son visibles al usuario.	RF-07



RF-09	Límite de ejecución	Corte automático a los 10s o 256MB RAM.	Sistema	Alta	El sistema detiene procesos que excedan los límites de seguridad.	RF-07
RF-10	Soporte multiformato	Compatibilidad con Python 3.x y PHP 8.x.	Estudiante	Alta	El backend selecciona el compilador correcto según el reto.	RF-07
RF-11	Ruta Python	10 niveles iniciales de lógica Python.	Estudiante	Alta	Se visualizan todos los temas desde variables hasta listas.	RF-21
RF-12	Ruta PHP	10 niveles iniciales de lógica PHP.	Estudiante	Alta	Se visualizan todos los temas desde sintaxis hasta formularios.	RF-21
RF-13	Tipos de ejercicio	Variedad: completar, corregir o escribir.	Estudiante	Alta	Cada nivel presenta la interfaz adecuada al tipo de desafío.	RF-14
RF-14	Instrucciones de nivel	Descripción de objetivos y salida esperada.	Estudiante	Media	El usuario visualiza la guía técnica antes de editar código.	-



RF-15	Progresión bloqueada	Desbloqueo secuencial de niveles.	Sistema	Alta	No se puede acceder al nivel "n" si "n-1" no está completado.	RF-16
RF-16	Validación automática	Comparación de output real vs. esperado.	Sistema	Alta	El reto se marca "completado" solo si el output es idéntico.	RF-08
RF-17	Retroalimentación	Entrega de pistas al fallar ejercicios.	Sistema	Alta	El sistema muestra un mensaje de ayuda tras un intento fallido.	RF-16
RF-18	Sistema de XP	Incremento de XP y nivel de perfil.	Sistema	Alta	El usuario sube de nivel al alcanzar el umbral de puntos definido.	RF-16
RF-19	Sistema de logros	Entrega de insignias por hitos alcanzados.	Sistema	Media	Se notifica y guarda la insignia en el perfil al cumplir el hito.	RF-04, RF-18
RF-20	Diseño responsivo	Adaptación a móviles (360px) y PC.	Público	Alta	La UI es fluida y legible en Android y navegadores de escritorio.	-



RF-21	Mapa de progreso	Interfaz visual del árbol de niveles.	Estudiante	Alta	Los estados (libre/bloqueado) se reflejan visualmente en el mapa.	RF-15
RF-22	Navegación táctil	Optimización de botones para interacción táctil.	Estudiante	Alta	Acciones como "Ejecutar" responden al tap sin errores de precisión.	RF-20

7.3 Requerimientos No Funcionales

Código	Nombre	Descripción	Métrica / Condición de Éxito	Prioridad	Notas / Alcance
RNF-01	Tiempo de carga inicial	Velocidad de acceso a la SPA (Single Page Application) desde el navegador.	Carga de la página principal en < 3 segundos con conexión de 10 Mbps.	Alta	Aplica al bundle de React y recursos estáticos (CSS/JS).
RNF-02	Latencia de ejecución	Tiempo de respuesta desde que se envía el código hasta que se recibe el output.	El resultado debe mostrarse en < 5 segundos (Métrica p95).	Alta	Incluye el tiempo de viaje a la API de Piston y retorno.
RNF-03	Aislamiento de ejecución	Seguridad del entorno de procesamiento de scripts.	0% de acceso al sistema de archivos local o red interna del servidor.	Alta	Garantizado mediante el Sandboxing de Piston.
RNF-04	Protección de datos	Cifrado de credenciales de acceso de los estudiantes.	Almacenamiento con Hash bcrypt (cost factor ≥ 10); nunca texto plano.	Alta	Aplica a la tabla de usuarios en la base de datos MySQL.



RNF-05	Comunicación segura	Cifrado de los datos en tránsito entre cliente y servidor.	Uso obligatorio de protocolos HTTPS/TLS 1.2 o superior.	Alta	Todo el tráfico de la API REST debe estar cifrado.
RNF-06	Concurrencia	Capacidad de respuesta del backend ante múltiples usuarios.	Soportar al menos 50 usuarios concurrentes ejecutando código sin degradación.	Media	Escalabilidad horizontal del servicio de Node.js.
RNF-07	Accesibilidad básica	Inclusión de usuarios con discapacidades visuales o motrices.	Cumplimiento del nivel AA de WCAG 2.1 (contrastes y navegación teclado).	Media	Aplicable a toda la interfaz de usuario de CodeBreaker.
RNF-08	Disponibilidad	Tiempo en que el sistema está operativo para los alumnos.	Disponibilidad del 99% durante el periodo académico vigente.	Alta	Excluye ventanas de mantenimiento programadas.
RNF-09	Compatibilidad Android	Adaptabilidad de la interfaz en dispositivos móviles.	Resolución mínima de 360x640px con soporte para teclado virtual.	Alta	Optimización de la UI para smartphones de estudiantes.
RNF-10	Mantenibilidad modular	Estructura del código para permitir futuras mejoras.	Backend con separación de responsabilidades (Auth, Sandbox, Progreso).	Media	Facilita la adición de nuevos lenguajes como Java o Rust.
RNF-11	Sanitización de entradas	Prevención de ataques por inyección o scripts maliciosos.	100% de las entradas validadas y sanitizadas antes de procesar en BD.	Alta	Previene SQL Injection y XSS (Cross-Site Scripting).
RNF-12	Mensajes de error	Claridad pedagógica en las notificaciones de fallo.	Errores en lenguaje humano comprensible, evitando jerga técnica interna.	Media	Mejora la experiencia de aprendizaje del estudiante principiante.



7.4 Oportunidades Detectadas

Durante el proceso de levantamiento de información y análisis del entorno académico, se identificaron diversas oportunidades de mejora e innovación que justifican el desarrollo e implementación del sistema CodeBreaker. Estas oportunidades surgen de las brechas en la enseñanza de la programación y la necesidad de entornos de práctica seguros y accesibles.

1. Modernización de la Enseñanza de Programación

Muchos entornos académicos aún dependen de instalaciones locales complejas que consumen recursos de hardware del estudiante. CodeBreaker representa una oportunidad para centralizar la práctica en una plataforma web, eliminando barreras técnicas de instalación y configuraciones de entorno iniciales.

2. Democratización del Acceso a Herramientas de Desarrollo

El uso de una arquitectura basada en la API de Piston permite que estudiantes con equipos de computación limitados puedan ejecutar lenguajes como Python, PHP o JS de forma remota. El sistema aprovecha el procesamiento en la nube para ofrecer una experiencia de codificación fluida sin importar la potencia del hardware local.

3. Feedback Inmediato y Aprendizaje Autogestionado

Los métodos tradicionales de evaluación de código suelen ser lentos. El sistema brinda la oportunidad de mejorar la curva de aprendizaje mediante la calificación automática y el Sandbox, proporcionando resultados en tiempo real que permiten al estudiante corregir errores de lógica de manera instantánea.

4. Cumplimiento de Estándares de Seguridad Educativa

Existe una necesidad crítica de ejecutar código de terceros de forma segura. La implementación de contenedores aislados (Sandboxing) detectada como oportunidad técnica garantiza que las prácticas de los alumnos no comprometan la integridad de los servidores institucionales, cumpliendo con los pilares de ciberseguridad.

5. Trazabilidad del Progreso Académico

La falta de registros detallados sobre los intentos y la evolución del aprendizaje es una debilidad común. El sistema permitirá recolectar métricas de aprendizaje (KPIs) sobre los lenguajes más usados y las tasas de éxito, ofreciendo a los docentes datos valiosos para ajustar sus sílabos y estrategias pedagógicas.

6. Fomento de la Innovación Tecnológica en la UPT

El proyecto ofrece una oportunidad de colaboración académica directa al ser desarrollado por estudiantes para la propia comunidad universitaria. Sirve como un caso de éxito en la implementación de patrones de software y arquitecturas modernas (Node.js, Express, React), promoviendo el talento local en Tacna.

7. Alineación con la Protección de Datos (Ley 29733)

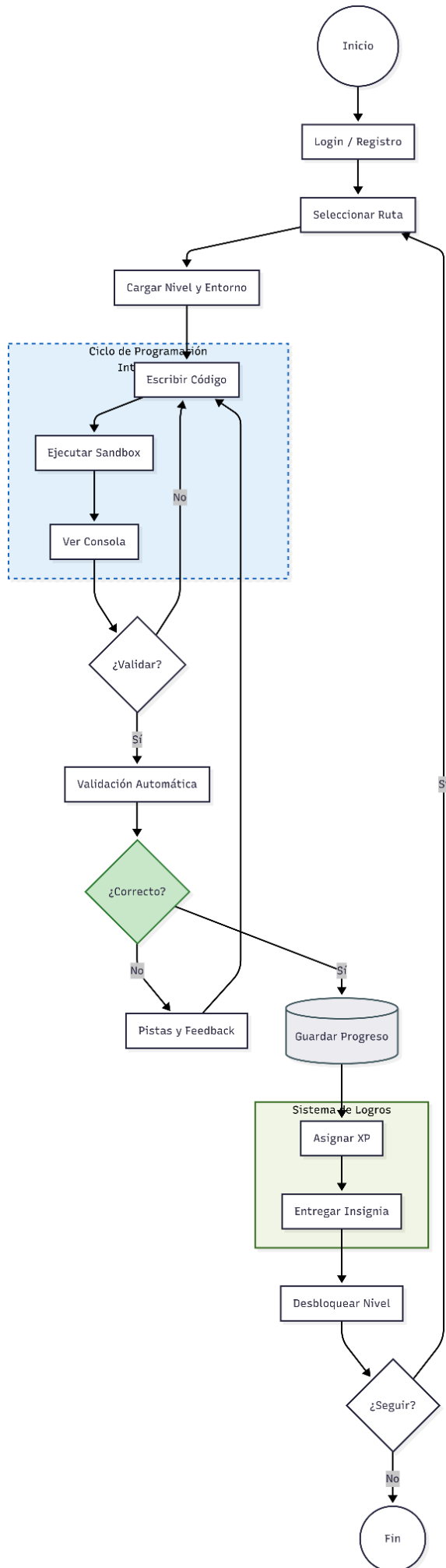
Dada la creciente sensibilidad sobre la privacidad, CodeBreaker se posiciona como una plataforma responsable al integrar desde su diseño los derechos ARCO y la minimización de datos (solo correo UPT), asegurando un entorno digital ético y legal para los estudiantes de ingeniería de sistemas.



III. Análisis de Procesos

- 1. Diagrama del Proceso Actual – Diagrama de Actividades**

- 2. Diagrama del Proceso Propuesto – (CodeBreaker)**





IV. Especificación de Requerimientos de Software

1. Cuadro de requerimientos funcionales Inicial

Código	Nombre	Descripción	Actores	Prioridad	Criterios de Aceptación	Dependencias
RF-01	Registro de usuario	Permitir registrar usuarios con correo/contraseña u OAuth.	Usuario	Alta	Dado que el usuario no está registrado, cuando ingresa sus datos válidos, entonces su cuenta es creada.	-
RF-02	Inicio de sesión	Autenticar usuarios y mantener sesión con JWT.	Usuario	Alta	Dado un usuario registrado, cuando inicia sesión correctamente, entonces accede al sistema.	RF-01
RF-03	Cierre de sesión	Permitir cerrar sesión e invalidar token.	Usuario	Media	Dado un usuario autenticado, cuando cierra sesión, entonces su token queda invalidado.	RF-02
RF-04	Perfil de usuario	Mostrar datos del usuario (nombre, avatar, nivel, XP, logros).	Usuario	Media	Dado un usuario autenticado, cuando accede a su perfil, entonces visualiza su información actual.	RF-02
RF-05	Guardado de progreso	Persistir progreso y sincronizar entre dispositivos.	Sistema	Alta	Dado progreso en curso, cuando el usuario completa un nivel, entonces se guarda correctamente.	RF-02
RF-06	Editor integrado	Editor con resaltado, autocompletado e	Usuario	Alta	Dado un ejercicio, cuando el usuario escribe código,	-



		indentación para Python y PHP.			entonces se visualiza correctamente formateado.	
RF-07	Ejecución en sandbox	Ejecutar código mediante API segura.	Sistema	Alta	Dado código ingresado, cuando se ejecuta, entonces se procesa en entorno aislado sin afectar el servidor.	RF-06
RF-08	Visualización de resultados	Mostrar stdout y stderr.	Usuario	Alta	Dado código ejecutado, cuando finaliza, entonces se muestran resultados o errores.	RF-07
RF-09	Límite de ejecución	Limitar ejecución a 10s y 256MB.	Sistema	Alta	Dado una ejecución, cuando supera límites, entonces se detiene automáticamente.	RF-07
RF-10	Soporte multiformato	Soporte para Python y PHP.	Sistema	Alta	Dado código en lenguaje válido, cuando se ejecuta, entonces es procesado correctamente.	RF-06, RF-07
RF-11	Ruta Python	10 niveles iniciales de Python.	Usuario	Alta	Dado un usuario nuevo, cuando accede a la ruta, entonces visualiza niveles disponibles.	RF-13
RF-12	Ruta PHP	10 niveles iniciales de PHP.	Usuario	Alta	Dado un usuario nuevo, cuando accede a la ruta, entonces visualiza niveles disponibles.	RF-13
RF-13	Tipos de ejercicio	Diferentes tipos de ejercicios.	Sistema	Alta	Dado un nivel, cuando se	-



					presenta, entonces incluye al menos un tipo de ejercicio válido.	
RF-14	Instrucciones de nivel	Mostrar objetivo, ejemplos y resultado esperado.	Usuario	Media	Dado un nivel, cuando inicia, entonces visualiza instrucciones completas.	RF-13
RF-15	Progresión bloqueada	Desbloqueo secuencial de niveles.	Sistema	Alta	Dado un nivel no completado, cuando intenta acceder al siguiente, entonces se bloquea el acceso.	RF-05
RF-16	Validación automática	Comparar salida con resultado esperado.	Sistema	Alta	Dado código ejecutado, cuando coincide con salida esperada, entonces se marca como completado.	RF-08
RF-17	Retroalimentación inmediata	Mostrar errores y pistas.	Sistema	Alta	Dado error en ejercicio, cuando falla, entonces se muestra mensaje descriptivo.	RF-16
RF-18	Sistema de XP y niveles	Ganar XP y subir de nivel.	Sistema	Alta	Dado un nivel completado, cuando finaliza, entonces se otorgan puntos XP.	RF-05
RF-19	Sistema de logros	Insignias por hitos.	Sistema	Media	Dado cumplimiento de condiciones, cuando se alcanza un hito, entonces se otorga un logro.	RF-18
RF-20	Diseño responsivo	Adaptación a escritorio y móviles.	Usuario	Alta	Dado distintos dispositivos, cuando accede, entonces la UI se	-



					adapta correctamente.	
RF-21	Mapa de progreso	Visualización de niveles.	Usuario	Alta	Dado acceso a rutas, cuando navega, entonces visualiza progreso y niveles.	RF-05
RF-22	Navegación táctil	Controles accesibles con tap.	Usuario	Alta	Dado uso en móvil, cuando interactúa, entonces los controles responden correctamente al tacto.	RF-20

2. Cuadro de requerimientos No funcionales

Tabla 06

Código	Nombre	Descripción	Métrica / Condición de Éxito	Prioridad	Notas / Alcance
RNF-01	Tiempo de carga inicial	La app debe cargar la página principal rápidamente.	< 3 segundos en conexión de 10 Mbps	Alta	Aplica a landing y dashboard
RNF-02	Latencia de ejecución	Mostrar resultados de ejecución de código rápidamente.	< 5 segundos en condiciones normales	Alta	Depende del servicio sandbox
RNF-03	Respuesta del editor	El editor debe ser fluido al escribir.	Latencia < 100 ms	Alta	Dispositivos gama media-alta
RNF-04	Carga de niveles	Transición rápida entre niveles.	< 1 segundo tras validación	Alta	UX crítica en aprendizaje
RNF-05	Soporte de concurrencia	Soportar múltiples usuarios simultáneos.	≥ 50 usuarios concurrentes sin degradación	Alta	Backend escalable requerido
RNF-06	Aislamiento de ejecución	Código ejecutado en entorno seguro.	Sin acceso a FS ni red del servidor	Alta	Uso de sandbox tipo Judge0



RNF-07	Protección de contraseñas	Almacenamiento seguro de credenciales.	Hash bcrypt con cost ≥ 10	Alta	Nunca texto plano
RNF-08	Comunicación cifrada	Seguridad en transmisión de datos.	HTTPS/TLS 1.2+ obligatorio	Alta	Certificado SSL válido
RNF-09	Validación de entradas	Prevenir ataques (SQLi, XSS, etc.).	Sanitización completa de inputs	Alta	Backend y frontend
RNF-10	Onboarding intuitivo	Inicio rápido para nuevos usuarios.	Usuario inicia ejercicio en < 3 min	Media	Incluye UX/UI inicial
RNF-11	Accesibilidad básica	Cumplir estándares de accesibilidad.	WCAG 2.1 nivel AA	Media	Incluye contraste y teclado
RNF-12	Mensajes de error claros	Errores comprensibles.	Lenguaje simple, sin jerga técnica	Media	UX educativo
RNF-13	Consistencia visual	UI coherente en toda la app.	Uso consistente de componentes y estilos	Media	Basado en design system
RNF-14	Arquitectura modular	Separación de responsabilidades.	Módulos independientes (auth, ejecución, etc.)	Alta	Facilita mantenimiento
RNF-15	Contenido configurable	Niveles editables sin cambiar código.	Definidos en BD o archivos config	Alta	Escalable en contenido
RNF-16	Escalabilidad horizontal	Capacidad de escalar servicios.	Soporte para múltiples instancias	Alta	Infraestructura cloud
RNF-17	Disponibilidad del servicio	Alta disponibilidad de la plataforma.	$\geq 99\%$ uptime	Alta	Periodo académico
RNF-18	Compatibilidad navegadores	Funcionar en navegadores modernos.	Chrome, Firefox, Edge ≥ 110 ; Safari ≥ 15	Media	Escritorio y móvil
RNF-19	Compatibilidad Android	Optimización en dispositivos Android.	Android 10+, resolución mínima 360x640	Media	UX táctil y teclado virtual



3. Cuadro de Requerimientos funcionales Final

Tabla 07

Código	Nombre	Descripción	Actores	Prioridad	Criterios de Aceptación	Dependencias
Código	Nombre	Descripción	Actores	Prioridad	Criterios de Aceptación	Dependencias
RF-01	Registro de usuario	Permitir registrar usuarios con correo/contraseña u OAuth.	Usuario	Alta	Dado que el usuario no está registrado, cuando ingresa sus datos válidos, entonces su cuenta es creada.	-
RF-02	Inicio de sesión	Autenticar usuarios y mantener sesión con JWT.	Usuario	Alta	Dado un usuario registrado, cuando inicia sesión correctamente, entonces accede al sistema.	RF-01
RF-03	Cierre de sesión	Permitir cerrar sesión e invalidar token.	Usuario	Media	Dado un usuario autenticado, cuando cierra sesión, entonces su token queda invalidado.	RF-02
RF-04	Perfil de usuario	Mostrar datos del usuario (nombre, avatar, nivel, XP, logros).	Usuario	Media	Dado un usuario autenticado, cuando accede a su perfil, entonces visualiza su información actual.	RF-02



RF-05	Guardado de progreso	Persistir progreso y sincronizar entre dispositivos.	Sistema	Alta	Dado progreso en curso, cuando el usuario completa un nivel, entonces se guarda correctamente.	RF-02
RF-06	Editor integrado	Editor con resaltado, autocompletado e indentación para Python y PHP.	Usuario	Alta	Dado un ejercicio, cuando el usuario escribe código, entonces se visualiza correctamente formateado.	-
RF-07	Ejecución en sandbox	Ejecutar código mediante API segura.	Sistema	Alta	Dado código ingresado, cuando se ejecuta, entonces se procesa en entorno aislado sin afectar el servidor.	RF-06
RF-08	Visualización de resultados	Mostrar stdout y stderr.	Usuario	Alta	Dado código ejecutado, cuando finaliza, entonces se muestran resultados o errores.	RF-07
RF-09	Límite de ejecución	Limitar ejecución a 10s y 256MB.	Sistema	Alta	Dado una ejecución, cuando supera límites, entonces se detiene automáticamente.	RF-07
RF-10	Soporte multiformato	Soporte para Python y PHP.	Sistema	Alta	Dado código en lenguaje válido, cuando se ejecuta, entonces es procesado correctamente.	RF-06, RF-07



RF-11	Ruta Python	10 niveles iniciales de Python.	Usuario	Alta	Dado un usuario nuevo, cuando accede a la ruta, entonces visualiza niveles disponibles.	RF-13
RF-12	Ruta PHP	10 niveles iniciales de PHP.	Usuario	Alta	Dado un usuario nuevo, cuando accede a la ruta, entonces visualiza niveles disponibles.	RF-13
RF-13	Tipos de ejercicio	Diferentes tipos de ejercicios.	Sistema	Alta	Dado un nivel, cuando se presenta, entonces incluye al menos un tipo de ejercicio válido.	
RF-14	Instrucciones de nivel	Mostrar objetivo, ejemplos y resultado esperado.	Usuario	Media	Dado un nivel, cuando inicia, entonces visualiza instrucciones completas.	RF-13
RF-15	Progresión bloqueada	Desbloqueo secuencial de niveles.	Sistema	Alta	Dado un nivel no completado, cuando intenta acceder al siguiente, entonces se bloquea el acceso.	RF-05
RF-16	Validación automática	Comparar salida con resultado esperado.	Sistema	Alta	Dado código ejecutado, cuando coincide con salida esperada, entonces se marca como completado.	RF-08



RF-17	Retroalimentación inmediata	Mostrar errores y pistas.	Sistema	Alta	Dado error en ejercicio, cuando falla, entonces se muestra mensaje descriptivo.	RF-16
RF-18	Sistema de XP y niveles	Ganar XP y subir de nivel.	Sistema	Alta	Dado un nivel completado, cuando finaliza, entonces se otorgan puntos XP.	RF-05
RF-19	Sistema de logros	Insignias por hitos.	Sistema	Media	Dado cumplimiento de condiciones, cuando se alcanza un hito, entonces se otorga un logro.	RF-18
RF-20	Diseño responsivo	Adaptación a escritorio y móviles.	Usuario	Alta	Dado distintos dispositivos, cuando accede, entonces la UI se adapta correctamente.	-
RF-21	Mapa de progreso	Visualización de niveles.	Usuario	Alta	Dado acceso a rutas, cuando navega, entonces visualiza progreso y niveles.	RF-05
RF-22	Navegación táctil	Controles accesibles con tap.	Usuario	Alta	Dado uso en móvil, cuando interactúa, entonces los controles responden correctamente al tacto.	RF-20



4. Reglas de Negocio

Tabla 08

Reglas de Negocio del sistema

RB	Nombre	Regla / Política	Aplica a	Disparador	Comportamiento del sistema	RF/RNF
RB-01	Aislamiento de Código	Todo código enviado debe ejecutarse en un entorno aislado (Sandbox).	Ejecución	Envío de código	Enviar petición a API Piston; retornar salida estándar o error de compilación.	RF-07, RNF-06
RB-02	Tiempo de Ejecución	El tiempo máximo de ejecución por script no debe exceder los 5 segundos.	Ejecución	Inicio de proceso	Cancelar ejecución si excede el tiempo y devolver error de "Timeout".	RF-09, RNF-02
RB-03	Roles y Acceso	Solo el Admin gestiona ejercicios; el Estudiante solo resuelve y ve su progreso.	Seguridad	Inicio de sesión	Renderizar dashboard según rol (RBAC); denegar rutas administrativas.	RF-02, RF-06
RB-04	Validación de Resultados	Un ejercicio se marca como "Completado" solo si la salida coincide con el caso de prueba.	Evaluación	Salida de código	Comparar String de salida vs Valor esperado (ignorar espacios en blanco).	RF-16
RB-05	Privacidad de Datos (ARCO)	Los datos personales deben tratarse bajo la Ley 29733 (Perú).	Privacidad	Registro	Solicitar consentimiento explícito; habilitar eliminación de cuenta.	RNF-04, RNF-05
RB-06	Unicidad de Usuario	No se permiten dos cuentas con el mismo correo institucional (UPT).	Registro	Guardar usuario	Validar existencia previa; rechazar con mensaje de error si ya existe.	RF-01



Nota:
Creación
propia

RB-07	Límite de Intentos	No hay límite de intentos para resolver ejercicios, pero se registra cada envío.	Evaluación	Envío de código	Guardar historial de envíos para análisis de aprendizaje.	RF-05 , RF-07
RB-08	Disponibilidad de Lenguajes	Solo se permite ejecutar lenguajes soportados por la configuración activa (Python, PHP, JS).	Ejecución	Selección de lenguaje	Filtrar lenguajes disponibles según la configuración del módulo.	RF-10
RB-09	Integridad de Contenido	Los ejercicios eliminados por el Admin pasan a estado "Inactivo" para no romper el historial.	Contenido	Borrar ejercicio	Cambio de estado lógico (Soft Delete); el ejercicio ya no aparece en el catálogo.	RF-03
RB-10	Seguridad de API	Las solicitudes a la API de Piston deben incluir una API Key válida y rotativa.	Seguridad	Request externo	Validar credenciales antes de la conexión externa.	RNF-04

4.1 Mapa de Trazabilidad de Requisitos

Tabla 09

Trazabilidad de Requisitos

RF	Nombre	Objetivo de negocio relacionado	Actores	Caso(s) de uso (CU)	Reglas de negocio (RB)	Criterios de aceptación (resumen)	Métrica / Validación	Estado
RF-01	Autenticación y Roles	Seguridad y personalización de la experiencia académica.	Estudiante / Admin	CU-01 Registro / CU-02 Login	RB-03, RB-06	Acceso seguro con JWT y validación de correo institucional (UPT).	100% de usuarios autenticados.	MVP
RF-04	Entorno Multi-lenguaje	Brindar versatilidad en el aprendizaje técnico.	Estudiante	CU-03 Selección de Lenguaje	RB-08	Disponibilidad de Python, PHP y JS según configuración del módulo.	Carga de entorno < 1.5s.	MVP
RF-05	Ejecución en Sandbox	Garantizar la integridad del servidor mediante aislamiento.	Sistema	CU-04 Ejecución de Código	RB-01, RB-02, RB-10	Procesamiento vía API de Piston con timeout automático de 5s.	Aislamiento verificado (0 escapes).	MVP



RF-06	Calificación Automática	Feedback inmediato sobre la lógica de programación.	Sistema	CU-05 Validación de Solución	RB-04	Comparación de salida estándar vs. valor esperado definido por el Admin.	Precisión del 100% en el test.	MVP
RF-07	Historial de Intentos	Persistencia y seguimiento del aprendizaje.	Estudiante	CU-06 Consulta de Progreso	RB-07	Registro íntegro de códigos enviados y resultados obtenidos.	Persistencia en base de datos.	MVP
RF-11	Gestión de Retos (CRUD)	Mantener el repositorio educativo actualizado.	Admin	CU-07 Gestión de Contenido	RB-03, RB-09	Creación de retos con casos de prueba; borrado lógico (Soft Delete).	Integridad referencial.	MVP
RF-20	Auditoría del Sistema	Control de seguridad y cumplimiento normativo.	Admin	CU-08 Auditoría de Logs	RB-10	Registro detallado de peticiones a la API externa (IP, payload, timestamp).	Trazabilidad completa.	MVP
RF-23	Recuperación Segura	Garantizar la disponibilidad de acceso al usuario.	Público	CU-09 Reset Password	RB-05*	Token de uso único enviado por email con expiración programada.	Token expira en 30 min.	MVP
RF-30	Gestión de Privacidad	Cumplimiento estricto de la Ley 29733 (Perú).	Estudiante	CU-10 Derechos ARCO	RB-05	Funciones de exportación y eliminación de datos personales (derecho al olvido).	Respuesta inmediata a solicitud.	MVP

Nota: Creación Propia

5. Otros Requerimientos del Producto

Tabla 10

Requerimientos de Producto

Nota: Creación Propia

Categoría	Requerimiento	Criterio medible / Condición de aceptación	Evidencia / Verificación
Legales	Cumplimiento Ley N° 29733 (Perú)	Consentimiento explícito en registro, cláusulas ARCO operativas y registro de finalidad.	Política publicada, logs de solicitudes ARCO, checklist legal.



Legales	Términos de uso Académicos	Aceptación obligatoria de normas de uso del sandbox al registrarse (checkbox).	Registro de aceptación con marca temporal en BD.
Seguridad	Aislamiento de Ejecución	Todo código de usuario debe correr en contenedores aislados (Sandbox) vía API Piston.	Pruebas de penetración al sandbox; logs de la API externa.
Seguridad	Control de acceso (RBAC)	Rutas de gestión de ejercicios inaccesibles a Estudiantes (Error 403).	Tests de autorización; auditoría de intentos de acceso denegados.
Seguridad	Gestión de sesiones	Uso de JWT con expiración de 2 horas; cookies httpOnly y SameSite.	Inspección de headers; pruebas OWASP ASVS.
Privacidad	Minimización de datos	Solo se recogen datos académicos: nombre, correo UPT y código de estudiante.	Revisión de esquemas de base de datos y formularios.
Técnico	Interoperabilidad (API)	API REST con documentación OpenAPI/Swagger (v1) para el motor de ejecución.	Especificación Swagger accesible; contratos de API validados.
Técnico	Tiempo de Respuesta	Ejecución de scripts y retorno de salida estándar en menos de 5 segundos.	Logs de tiempo de respuesta (latency) en el backend.
Calidad (UX)	Accesibilidad Educativa	Cumplimiento de WCAG 2.1 nivel AA para el editor de código y dashboard.	Informe de auditoría (Lighthouse / Axe DevTools).
Calidad	Resiliencia (Timeout)	El sistema debe cancelar ejecuciones infinitas o pesadas que superen los 256MB de RAM.	Pruebas de carga; logs de errores "Resource Limit Exceeded".
Comunicación	Notificaciones Académicas	Correos de confirmación de registro y recuperación de cuenta parametrizados.	Catálogo de plantillas y reportes de entregabilidad SMTP.



Compatibilidad	Navegadores Modernos	Soporte completo en Chrome, Firefox y Edge para el resaltado de sintaxis del editor.	Matriz de compatibilidad; pruebas de renderizado del editor.
Monitoreo	Observabilidad	Dashboards de errores de compilación, consumo de API Piston y latencia p95.	Panel de métricas (Grafana/Prometheus o similar).
Backups	Recuperación de Datos	Respaldo diario de ejercicios y progreso de usuarios; RTO $\leq$ 4h.	Logs de backups; acta de prueba de restauración trimestral.
Anti-abuso	Rate Limiting	Límite de 10 ejecuciones de código por minuto por usuario para evitar spam a la API.	Configuración de WAF/Middleware; logs de bloqueo por exceso.
Datos	Retención y Purga	Logs de ejecución: 30 días; historial de código: 1 semestre; derecho al olvido.	Política de retención y scripts de purga programada.
Despliegue	CI/CD	Pipeline automatizado con pruebas unitarias antes del despliegue en producción.	Evidencia de paso de tests en GitHub Actions / Pipeline.
Contenido	Gestión de Casos de Prueba	El Admin define entradas/salidas; el sistema valida integridad del test (no nulos).	Validaciones en el módulo de gestión de ejercicios.
Localización	Idiomas Técnico	Interfaz en Español (ES) con opción a Inglés (EN) para términos técnicos.	Archivos de traducción i18n validados.
Analítica	Métricas de Aprendizaje	KPIs: tasa de éxito por ejercicio, lenguajes más usados, tiempo promedio de resolución.	Dashboard de analíticas con exportación para el docente.



V. Fase de Desarrollo

1. Perfiles de Usuario

Tabla 11

Perfil de Usuario

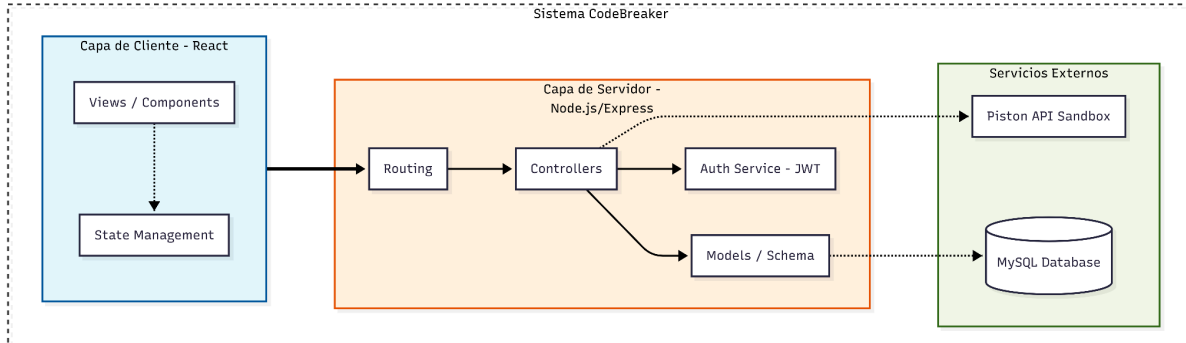
Perfil	Objetivo	Tareas clave	Permisos (resumen)	Datos que maneja	Vistas/Pantallas	Métricas de éxito	Riesgos / RNF clave
Público (Visitante)	Explorar la oferta educativa y registrarse para practicar.	Ver landing page, lenguajes disponibles (Python, PHP, JS) y registrarse con correo UPT.	Lectura de página principal y términos de uso; acceso a formulario de registro.	Nombre, correo institucional, contraseña (hash).	Landing Page, Catálogo de cursos, Formulario de Registro/Login.	Tasa de conversión de visitante a registrado.	Abandono en registro. RNF: Rendimiento (RNF-001), SEO (RNF-013).
Estudiante (Usuario)	Aprender a programar mediante la resolución de retos en un entorno seguro.	Escribir código, ejecutar scripts en el Sandbox, validar resultados y ver progreso académico.	Ejecución de código (limitada por RB-02), gestión de perfil propio, consulta de historial de intentos.	Código fuente enviado, resultados de tests, puntaje, logs de intentos.	Dashboard Estudiante, Editor de Código, Historial de Intentos, Mi Perfil.	% de ejercicios completados; tiempo promedio de resolución.	Abuso de recursos en el sandbox. RNF: Seguridad (RNF-004), Rate Limiting (RNF-020).
Administrador (Docente)	Gestionar el contenido educativo y garantizar la seguridad del sistema.	CRUD de ejercicios y casos de prueba; monitorear logs de ejecución; gestionar usuarios y reportes.	Acceso total a módulos administrativos: gestión de retos, auditoría de logs, control de usuarios y API.	Repositorio de retos, casos de prueba (input/output), logs de sistema, métricas globales.	Gestión de Ejercicios, Panel de Auditoría (Logs Piston), Reportes Académicos, Gestión de Usuarios.	Disponibilidad del servicio (SLA); integridad del catálogo educativo.	Fallas en la conexión con la API de Piston. RNF: Observabilidad (RNF-009), Backups (RNF-010).

Nota: Creación Propia

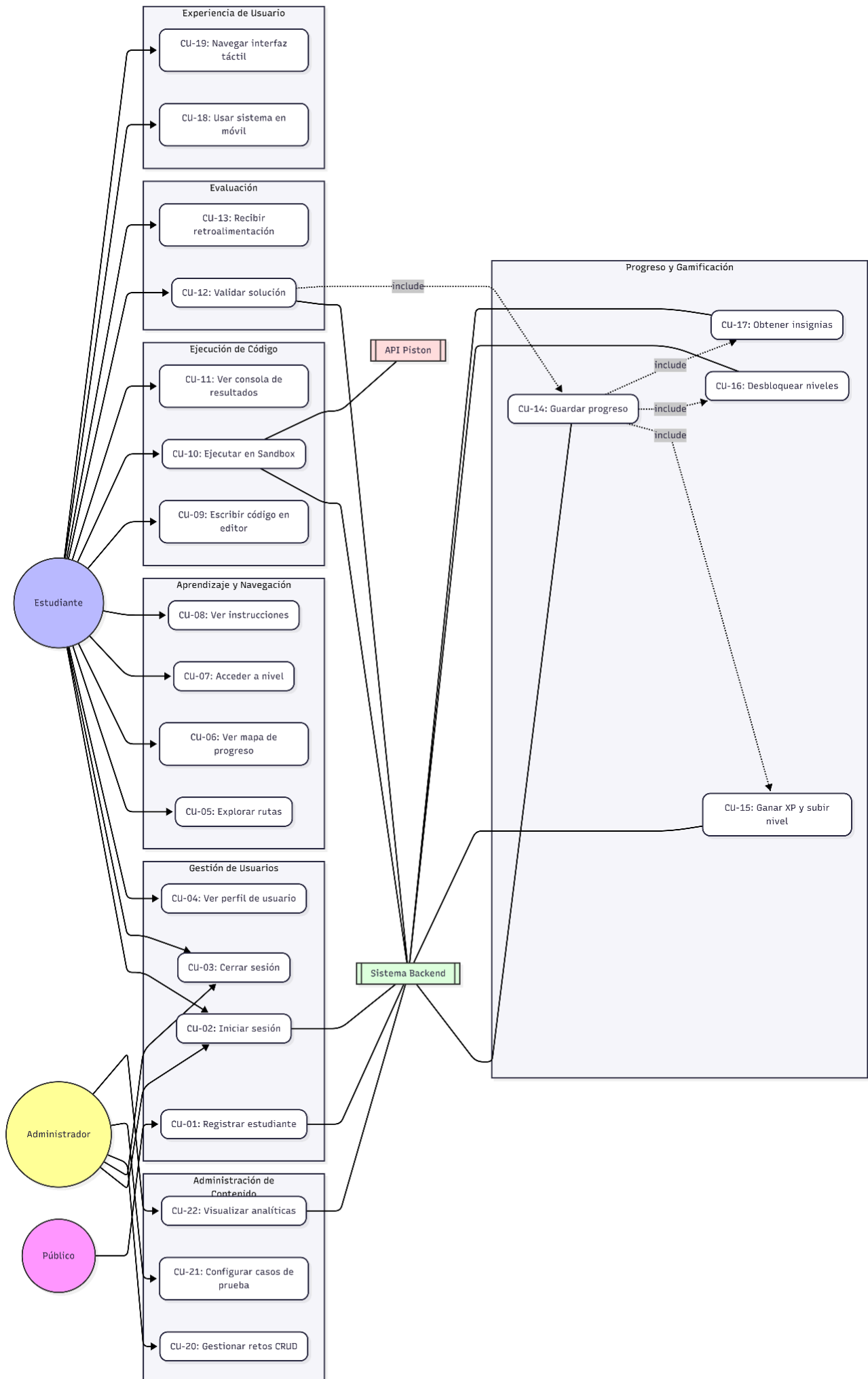


2. Modelo Conceptual

2.1. Diagrama de Paquetes



2.2. Diagrama de Casos de Uso





2.3. Diagrama de Casos de Uso - Narrativa

a) Escenarios de Caso de Uso (narrativa)

Caso de uso	Módulo	ID	Nombre
	Gestión de Usuario	CU-01	Registrar nuevo estudiante
Descripción del caso de uso	Permite a un nuevo usuario crear una identidad digital en la plataforma CodeBreaker para iniciar su formación en programación, asegurando la persistencia de su progreso y el cumplimiento de la normativa de protección de datos.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario académico que busca aprender lenguajes de programación.
	Sistema	Secundario	Componente backend encargado de la validación, cifrado y persistencia.
Precondición	El estudiante debe contar con acceso a su correo institucional de la UPT y no poseer una cuenta activa vinculada a dicho correo.		
Entradas	Nombre de usuario, correo electrónico institucional (@upt.pe), contraseña (mínimo 8 caracteres, incluye mayúsculas y números).		
Salidas	Registro exitoso en la base de datos; perfil de usuario inicializado (Nivel 1, 0 XP); Token de sesión generado; Redirección automática al Dashboard.		
Reglas implicadas	RB-06 (Unicidad): No se permiten duplicados de correo.		
	RNF-04 (Seguridad): Uso de algoritmo Bcrypt para el hash de contraseñas.		
	RNF-07 (Onboarding): El proceso debe completarse en menos de 3 minutos.		
Criterios de aceptación (QA)	El sistema bloquea el registro si el dominio del correo no es @upt.pe.		
	El sistema impide el envío del formulario si no se marca el check de "Aceptación de Políticas de Privacidad (Ley 29733)".		
	La contraseña debe pasar el validador de complejidad antes de habilitar el botón de registro.		
Flujo principal	Paso	Acción	
	1.1	El estudiante selecciona la opción "Registrarse" en la interfaz de inicio.	
	1.2	El estudiante completa el formulario con sus datos personales y credenciales.	
	1.3	El estudiante acepta los Términos y Condiciones y la Política de Protección de Datos Personales.	



	1.4	El sistema valida en tiempo real la sintaxis del correo y la fortaleza de la contraseña.
	1.5	El sistema verifica la inexistencia del correo en la base de datos y procesa el hash de la contraseña.
	1.6	El sistema crea el registro, inicializa los valores de gamificación (XP y Logros) y otorga acceso a la Ruta 1.
Postcondición	El estudiante queda autenticado en el sistema y visualiza el Mapa de Niveles con el primer ejercicio desbloqueado.	
Excepciones	Paso	Acción
	1.5.a	Correo duplicado: El sistema notifica que el usuario ya existe y sugiere recuperar contraseña.
	1.4.a	Dominio inválido: El sistema muestra un mensaje de advertencia: "Solo se permiten correos institucionales @upt.pe".
	1.4.b	Contraseña débil: El sistema indica los requisitos faltantes (ej. "Falta un número").
	1.3.a	Falta consentimiento: El sistema inhabilita el botón de registro hasta que se acepten las políticas legales.

Caso de uso	Módulo	ID	Nombre
	Gestión de Usuario	CU-02	Iniciar sesión
Descripción del caso de uso	Permite a un estudiante registrado autenticarse en la plataforma para retomar su progreso, acceder a sus rutas de aprendizaje y sincronizar su experiencia (XP) acumulada.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario que busca retomar sus sesiones de aprendizaje.
	Sistema	Secundario	Encargado de validar credenciales, generar tokens de acceso y recuperar el estado del usuario.
Precondición	El estudiante debe estar previamente registrado en el sistema (CU-01).		
Entradas	Correo electrónico institucional (@upt.pe), contraseña.		
Salidas	Token de autenticación (JWT) almacenado localmente; Redirección al último nivel visitado o al Dashboard; Carga de estadísticas personales (XP, nivel, insignias).		
Reglas implicadas	RF-02 (Autenticación): Uso de tokens JWT para mantener la sesión activa.		
	RNF-04 (Seguridad): Validación de hash de contraseña.		
	RNF-08 (Compatibilidad): El login debe ser fluido en dispositivos móviles y escritorio.		



Criterios de aceptación (QA)	• El sistema debe impedir el acceso tras 5 intentos fallidos (prevención de fuerza bruta).	
	• La sesión debe persistir aunque el usuario cierre el navegador, a menos que cierre sesión explícitamente.	
	• El tiempo de respuesta de la autenticación debe ser menor a 2 segundos.	
Flujo principal	Paso	Acción
	1.1	El estudiante accede a la pantalla de inicio de sesión.
	1.2	El estudiante ingresa sus credenciales (correo y contraseña).
	1.3	El sistema valida que el formato del correo sea correcto y que los campos no estén vacíos.
	1.4	El sistema busca el usuario en la BD y compara el hash de la contraseña ingresada con el almacenado.
	1.5	El sistema genera un Token de Acceso seguro y recupera el último estado de progreso (XP y niveles).
	1.6	El sistema redirige al usuario a su panel principal personalizado.
Postcondición	El estudiante está autenticado con éxito y sus datos de gamificación están listos para actualizarse tras resolver retos.	
Excepciones	Paso	Acción
	1.4.a	Credenciales incorrectas: El sistema muestra "Correo o contraseña incorrectos" sin especificar cuál falló por seguridad.
	1.4.b	Usuario no registrado: El sistema sugiere al usuario crear una cuenta nueva.
	1.5.a	Token expirado/inválido: El sistema solicita al usuario loguearse nuevamente si intenta acceder a una ruta protegida.
	1.6.a	Error de conexión: El sistema notifica que el servicio de autenticación no está disponible temporalmente.

Caso de uso	Módulo	ID	Nombre
	Gestión de Usuario	CU-03	Cerrar sesión
Descripción del caso de uso	Permite al estudiante finalizar su sesión de forma segura, garantizando que el token de acceso sea invalidado y que nadie más pueda acceder a su progreso desde el mismo dispositivo.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario que desea salir del sistema.
	Sistema	Secundario	Encargado de limpiar el almacenamiento local y



			revocar el acceso en el servidor.
Precondición	El estudiante debe tener una sesión activa en el sistema (CU-02).		
Entradas	Acción de clic en el botón "Cerrar sesión".		
Salidas	Token de acceso eliminado; Redirección a la pantalla de inicio (Landing Page/Login).		
Reglas implicadas	RF-03 (Cierre de sesión): Invalidación inmediata del token activo.		
	RNF-04 (Seguridad): Limpieza de datos sensibles en el LocalStorage o Cookies.		
Criterios de aceptación (QA)	• Tras cerrar sesión, intentar navegar a una ruta protegida (ej. /dashboard) debe redireccionar al Login.		
	• El sistema debe confirmar visualmente que la sesión ha finalizado.		
Flujo principal	Paso	Acción	
	1.1	El estudiante selecciona la opción "Cerrar sesión" en el menú de perfil o navegación.	
	1.2	El sistema solicita confirmación al usuario (opcional, según diseño de UI).	
	1.3	El sistema elimina el Token de Acceso (JWT) almacenado en el navegador (Local Storage o Cookies).	
	1.4	El sistema envía una señal al backend para invalidar el token en la lista de sesiones activas (si aplica).	
	1.5	El sistema redirige al usuario a la pantalla principal de la plataforma.	
Postcondición	La sesión queda terminada y el acceso a las rutas de aprendizaje queda bloqueado hasta un nuevo inicio de sesión.		
Excepciones	Paso	Acción	
	1.3.a	Error de red: Si no hay internet para avisar al backend, el sistema prioriza borrar el token local para asegurar el dispositivo del usuario.	
	1.5.a	Sesión ya expirada: Si el token ya no era válido, el sistema simplemente redirige al usuario al inicio sin mostrar errores.	

Caso de uso	Módulo	ID	Nombre
	Gestión de Usuario	CU-04	Ver perfil de usuario
Descripción del caso de uso	Permite al estudiante visualizar su identidad dentro de la plataforma, incluyendo sus estadísticas de aprendizaje (XP), su nivel actual y la colección de insignias obtenidas por sus logros académicos.		
Actores	Actor	Tipó	Descripción



	Estudiante	Primario	Usuario que desea monitorear su avance y estatus en la plataforma.
	Sistema	Secundario	Encargado de recuperar y calcular en tiempo real las métricas de progreso desde la base de datos.
Precondición	El estudiante debe haber iniciado sesión correctamente (CU-02).		
Entradas	Selección del icono de perfil o sección "Mi Perfil" en el menú de navegación.		
Salidas	Interfaz visual con: Nombre, avatar, nivel, barra de experiencia (XP), total de niveles completados y tablero de logros/insignias.		
Reglas implicadas	RF-04 (Perfil de usuario): Despliegue de datos de gamificación.		
	RF-05 (Guardado de progreso): Sincronización de datos persistentes.		
	RNF-13 (Consistencia visual): Diseño coherente con el sistema de diseño de la app.		
Criterios de aceptación (QA)	• Los datos mostrados deben coincidir exactamente con el último estado guardado en la BD.		
	• La barra de progreso debe reflejar el porcentaje de XP necesario para alcanzar el siguiente nivel.		
	• El tiempo de carga del perfil no debe superar los 1.5 segundos.		
Flujo principal	Paso	Acción	
	1.1	El estudiante hace clic en su avatar o en la opción "Perfil" desde cualquier pantalla del sistema.	
	1.2	El sistema solicita al backend los datos actualizados del usuario (XP, nivel, lista de insignias).	
	1.3	El sistema recupera la información de la base de datos y calcula la posición del usuario en el mapa de progreso.	
	1.4	El sistema renderiza la interfaz del perfil mostrando los logros obtenidos y los retos pendientes.	
	1.5	El estudiante visualiza su estatus y puede navegar de regreso al mapa de rutas.	
Postcondición	El usuario conoce su estado actual en la plataforma, lo cual incentiva la continuidad del aprendizaje.		
Excepciones	Paso	Acción	
	1.3.a	Error de sincronización: Si los datos no cargan, se muestra un mensaje de "Error al recuperar estadísticas" con un botón de reintentar.	
	1.5.a	Sin logros: Si el usuario es nuevo, el sistema muestra mensajes motivacionales para completar su primer nivel.	

	Módulo	ID	Nombre
--	--------	----	--------

Caso de uso



	Aprendizaje / Navegación	CU-05	Explorar rutas de aprendizaje
Descripción del caso de uso	Permite al estudiante navegar a través de los diferentes lenguajes disponibles (Python, PHP) y visualizar su progreso actual mediante un mapa de niveles interactivo.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario que busca seleccionar un tema o nivel para practicar.
	Sistema	Secundario	Encargado de filtrar los niveles disponibles según el progreso guardado del usuario.
Precondición	El estudiante debe estar autenticado (CU-02).		
Entradas	Selección de la ruta (Python o PHP) desde el menú principal.		
Salidas	Mapa visual de niveles (Ruta de aprendizaje) con indicadores de estado: Completado, Disponible o Bloqueado.		
Reglas implicadas	RF-11/RF-12 (Rutas): Existencia de al menos 10 niveles por lenguaje.		
	RF-15 (Bloqueo secuencial): Un nivel solo se habilita si el anterior fue completado.		
	RF-21 (Mapa visual): Interfaz gráfica tipo "camino" o "grafo".		
Criterios de aceptación (QA)	• Los niveles bloqueados deben mostrar un icono de candado y no permitir el acceso.		
	• El mapa debe ser responsivo y permitir navegación táctil en móviles (RNF-19).		
	• El estado de cada nivel debe actualizarse inmediatamente tras completar un reto.		
Flujo principal	Paso	Acción	
	1.1	El estudiante accede a la sección "Rutas" o "Dashboard".	
	1.2	El sistema muestra las tarjetas de los lenguajes disponibles (Python 3.x y PHP 8.x).	
	1.3	El estudiante selecciona una ruta específica (ej. Python).	
	1.4	El sistema consulta en la BD el progreso del usuario para esa ruta.	
	1.5	El sistema renderiza el mapa de niveles, resaltando el nivel actual donde se quedó el estudiante.	
	1.6	El estudiante selecciona un nivel "Disponible" para comenzar a practicar.	
Postcondición	El estudiante es dirigido a la interfaz del editor de código (CU-06) del nivel seleccionado.		
Excepciones	Paso	Acción	
	1.3.a	Ruta no disponible: Si una ruta está en desarrollo, el sistema muestra el mensaje "Próximamente".	



	1.5.a	Acceso a nivel bloqueado: Si el usuario intenta forzar la entrada a un nivel superior sin terminar los previos, el sistema muestra una alerta: "Debes completar los retos anteriores".
	1.4.a	Error de carga: Si no se recupera el progreso, el sistema muestra los niveles como bloqueados por seguridad y solicita refrescar la página.

Caso de uso	Módulo	ID	Nombre
	Aprendizaje / Interfaz	CU-06	Visualizar mapa de progreso
Descripción del caso de uso	Permite al estudiante observar de forma gráfica su avance dentro de una ruta tecnológica (Python o PHP), identificando los retos completados, el reto actual y los niveles que aún permanecen bloqueados.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario que consulta su ubicación en la ruta de aprendizaje.
	Sistema	Secundario	Encargado de renderizar la interfaz visual basada en el historial del usuario.
Precondición	El estudiante debe estar autenticado y haber seleccionado una ruta de aprendizaje específica (CU-04).		
Entradas	Acción de seleccionar una ruta o ingresar al Dashboard principal.		
Salidas	Interfaz de "Roadmap" o Grafo con estados visuales: Superado (check), Actual (resaltado) y Bloqueado (candado).		
Reglas implicadas	RF-15 (Desbloqueo): Solo se habilita el nivel $n+1$ si el nivel n ha sido validado con éxito.		
	RF-21 (Mapa Visual): Despliegue de niveles en formato de mapa interactivo.		
	RF-22 (Navegación Táctil): El mapa debe permitir desplazamiento <i>scroll/swipe</i> en dispositivos móviles.		
Criterios de aceptación (QA)	El sistema debe posicionar automáticamente la vista en el nivel más alto alcanzado por el usuario.		
	Los niveles bloqueados no deben responder a eventos de clic o pulsación.		
	El tiempo de carga de los nodos del mapa debe ser inferior a 1 segundo para garantizar fluidez.		
Flujo principal	Paso	Acción	
	1.1	El estudiante accede a la vista de la Ruta seleccionada.	



	1.2	El sistema solicita al backend el estado de todos los niveles vinculados a dicha ruta para el ID del usuario.
	1.3	El sistema procesa la lógica de bloqueo: marca como "disponible" el primer nivel no completado y como "bloqueados" todos los posteriores.
	1.4	El sistema renderiza el camino de aprendizaje con iconos diferenciados por color y estado.
	1.5	El estudiante interactúa con el mapa para seleccionar el nivel al que desea ingresar.
Postcondición	El estudiante selecciona un reto disponible y es redirigido al editor de código (CU-06).	
Excepciones	Paso	Acción
	1.2.a	Sin datos de progreso: Si es un usuario nuevo, el sistema desbloquea automáticamente el "Nivel 0: Introducción" y bloquea el resto.
	1.4.a	Error de renderizado: Si falla la carga de imágenes o iconos, el sistema muestra una lista de texto alternativa para no interrumpir el acceso.
	1.5.a	Nivel restringido: Si el estudiante intenta acceder a un nivel bloqueado, el sistema muestra un mensaje flotante (Tooltip): "Completa el nivel anterior para desbloquear este reto".

Caso de uso	Módulo	ID	Nombre
	Aprendizaje / Contenido	CU-07	Acceder a nivel
Descripción del caso de uso	Permite al estudiante ingresar al entorno de resolución de un reto específico, cargando las instrucciones, el código inicial y las herramientas de ejecución correspondientes.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario que decide iniciar un ejercicio de programación.
	Sistema	Secundario	Encargado de verificar permisos y servir el contenido del nivel desde la base de datos.
Precondición	El estudiante debe haber seleccionado un nivel que figure como "Disponible" en su Mapa de Progreso (CU-05).		
Entradas	Clic o pulsación sobre un nodo de nivel habilitado.		
Salidas	Interfaz del Editor cargada con: Título del reto, narrativa/instrucciones, plantilla de código base y consola de salida.		
Reglas	RF-14 (Instrucciones): Despliegue de objetivos y ejemplos de salida.		



implicadas	RF-15 (Bloqueo): Verificación de que los requisitos previos (niveles anteriores) estén cumplidos.	
	RNF-15 (Contenido configurable): Carga dinámica del ejercicio desde la BD sin recargar la aplicación.	
Criterios de aceptación (QA)	El sistema no debe permitir la carga de un nivel si el ID del usuario no tiene registrado el nivel anterior como "Completado".	
	El entorno debe cargar el lenguaje correcto (Python o PHP) según la ruta seleccionada.	
	Si el estudiante ya había iniciado el nivel pero no lo terminó, se debe cargar su último código guardado.	
Flujo principal	Paso	Acción
	1.1	El estudiante selecciona un nivel disponible en el Mapa de Progreso.
	1.2	El sistema valida en el backend que el usuario tiene los permisos para acceder a ese ID de nivel.
	1.3	El sistema recupera de la base de datos la configuración del reto: lenguaje, descripción, código inicial y casos de prueba.
	1.4	El sistema inicializa el Editor Integrado (Monaco/CodeMirror) con el resaltado de sintaxis correspondiente.
	1.5	El sistema presenta al estudiante la vista de "Área de Trabajo" con las instrucciones visibles en un panel lateral.
Postcondición	El estudiante se encuentra dentro del entorno de codificación listo para realizar el Caso de Uso: CU-08 (Escribir y ejecutar código).	
Excepciones	Paso	Acción
	1.2.a	Intento de salto de nivel: Si el usuario intenta acceder por URL a un nivel bloqueado, el sistema lo redirige al Mapa con un mensaje de error.
	1.3.a	Fallo de carga de activos: Si no se recuperan las instrucciones, el sistema muestra un botón de "Recargar Nivel".
	1.4.a	Lenguaje no soportado: Si el nivel requiere un motor de ejecución no configurado, el sistema deshabilita el botón de ejecución.

Caso de uso	Módulo	ID	Nombre
	Aprendizaje / Contenido	CU-08	Visualizar instrucciones del reto
Descripción del caso de uso	Permite al estudiante consultar la narrativa pedagógica, los objetivos técnicos, los ejemplos de entrada/salida y las restricciones de un nivel para comprender qué solución debe programar.		
Actores	Actor	Tipó	Descripción



	Estudiante	Primario	Usuario que requiere entender el problema para proponer una solución.
	Sistema	Secundario	Encargado de presentar el contenido educativo de forma clara y accesible.
Precondición	El estudiante debe haber accedido con éxito a un nivel específico (CU-07).		
Entradas	Carga automática al entrar al nivel o clic en el icono de "Ayuda/Instrucciones".		
Salidas	Panel lateral o modal con: Título del reto, descripción del problema, ejemplos de código (entrada vs salida esperada) y tips opcionales.		
Reglas implicadas	RF-14 (Instrucciones): El sistema debe mostrar instrucciones claras y ejemplos de ejecución.		
	RNF-12 (Lenguaje comprensible): El contenido debe evitar jerga técnica innecesaria para principiantes.		
	RNF-11 (Accesibilidad): El texto debe tener contraste adecuado para lectura prolongada.		
Criterios de aceptación (QA)	• Las instrucciones deben ser visibles sin obstruir el editor de código principal.		
	• Los ejemplos de entrada y salida deben estar claramente diferenciados (usualmente en bloques de código monospaciados).		
	• El panel de instrucciones debe poder colapsarse para maximizar el área de trabajo en pantallas pequeñas (RNF-19).		
Flujo principal	Paso	Acción	
	1.1	El sistema detecta la carga exitosa del nivel y despliega el panel de instrucciones automáticamente.	
	1.2	El estudiante lee la descripción del reto (ej. "Crea un bucle que imprima los números del 1 al 10").	
	1.3	El estudiante revisa la sección de "Ejemplos" para entender el formato exacto de salida que espera el validador.	
	1.4	El estudiante consulta las restricciones del nivel (ej. "No usar librerías externas").	
	1.5	El estudiante colapsa o mantiene abierto el panel mientras comienza a escribir su solución en el editor.	
Postcondición	El estudiante comprende el objetivo del reto y está listo para ejecutar y validar su código (CU-09 y CU-10).		
Excepciones	Paso	Acción	
	1.2.a	Contenido vacío: Si por un error de base de datos no hay texto, el sistema muestra un mensaje de "Instrucciones no disponibles" y un botón para reportar el error.	
	1.3.a	Dificultad de lectura: El estudiante puede ajustar el tamaño de fuente del panel de instrucciones para mejorar la legibilidad.	



	1.4.a	Confusión de lenguaje: Si el usuario cambió de ruta (ej. de Python a PHP), el sistema actualiza automáticamente los ejemplos al lenguaje correspondiente.
--	-------	--

Caso de uso	Módulo	ID	Nombre
	Ejecución de Código	CU-09	Escribir código en editor
Descripción del caso de uso	Permite al estudiante redactar su solución técnica utilizando un editor de texto enriquecido que facilita la programación mediante herramientas de asistencia visual y sintáctica.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario que traduce la lógica del problema a código fuente.
	Sistema	Secundario	Provee la interfaz del editor (basado en Monaco o CodeMirror) y gestiona la asistencia en tiempo real.
Precondición	El estudiante debe haber accedido a un nivel específico (CU-07) y seleccionado el lenguaje de la ruta (Python/PHP).		
Entradas	Caracteres de texto, comandos de indentación (Tab), atajos de teclado (Ctrl+C/V).		
Salidas	Código fuente estructurado y resaltado visualmente en el área de trabajo.		
Reglas implicadas	RF-06 (Editor integrado): Resaltado de sintaxis y autocompletado.		
	RF-10 (Soporte multiformato): Adaptación del editor a Python 3.x o PHP 8.x.		
Criterios de aceptación (QA)	El editor debe aplicar el resaltado de colores correcto según las palabras reservadas del lenguaje seleccionado.		
	La indentación automática debe activarse tras presionar "Enter" después de una estructura de control (como un if o for).		
	El código escrito debe persistir localmente de forma temporal para evitar pérdidas por desconexión accidental.		
Flujo principal	Paso	Acción	
	1.1	El sistema carga la plantilla de código inicial (Boilerplate) en el área del editor.	
	1.2	El estudiante posiciona el cursor en el editor e inicia la redacción de su script.	
	1.3	El sistema aplica resaltado de sintaxis en tiempo real para diferenciar variables, funciones y strings.	



	1.4	El estudiante consulta las restricciones del nivel (ej. "No usar librerías externas").
	1.5	El estudiante colapsa o mantiene abierto el panel mientras comienza a escribir su solución en el editor.
Postcondición	El estudiante comprende el objetivo del reto y está listo para ejecutar y validar su código (CU-09 y CU-10).	
Excepciones	Paso	Acción
	1.2.a	Contenido vacío: Si por un error de base de datos no hay texto, el sistema muestra un mensaje de "Instrucciones no disponibles" y un botón para reportar el error.
	1.3.a	Dificultad de lectura: El estudiante puede ajustar el tamaño de fuente del panel de instrucciones para mejorar la legibilidad.
	1.4.a	Confusión de lenguaje: Si el usuario cambió de ruta (ej. de Python a PHP), el sistema actualiza automáticamente los ejemplos al lenguaje correspondiente.

Caso de uso	Módulo	ID	Nombre
	Ejecución de Código	CU-010	Ejecutar código en Sandbox
Descripción del caso de uso	Permite al estudiante enviar el código fuente escrito en el editor hacia un entorno de ejecución aislado (Sandbox) en la nube, procesarlo en tiempo real y visualizar los resultados o errores sin comprometer la seguridad del servidor principal.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario que desea probar si la lógica de su código funciona.
	Sistema	Secundario	Backend que actúa como intermediario seguro (Proxy) entre el cliente y la API.
	API Piston	Secundario	Motor externo que provee los contenedores efímeros para la compilación/ejecución.
Precondición	El estudiante debe estar dentro del editor de un nivel activo (CU-09) y haber redactado su propuesta de solución.		
Entradas	Código fuente (String), Identificador y versión del lenguaje (ej. python-3.10 o php-8.2), Acción de clic en el botón "Ejecutar".		
Salidas	Impresión en la consola virtual: Salida estándar (stdout), errores (stderr) y métricas básicas (tiempo de ejecución en milisegundos).		
Reglas implicadas	RF-07 (Sandbox): Uso de Piston para aislamiento.		
	RF-08 (Visualización): Mostrar resultados en un panel inferior.		



	RF-09 / RB-02 (Límites): Tope estricto de 5 a 10 segundos de cómputo y 256 MB de RAM.	
	RNF-20 (Anti-abuso): Rate limiting de 10 ejecuciones por minuto por usuario.	
Criterios de aceptación (QA)	El sistema no debe bloquearse ni reiniciarse si el estudiante envía un código con un bucle infinito (while True).	
	El tiempo total desde que se presiona el botón hasta que se muestra el resultado no debe superar los 5 segundos (RNF-02).	
	El backend debe ocultar la <i>API Key</i> de Piston, evitando que el usuario la vea en las peticiones de red del navegador (RB-10).	
Flujo principal	Paso	Acción
	1.1	El estudiante presiona el botón "Ejecutar" en la interfaz del editor.
	1.2	El frontend empaqueta el código y el lenguaje seleccionado, enviándolos al backend institucional.
	1.3	El backend valida que el usuario no haya excedido el límite de ejecuciones por minuto (Rate Limit).
	1.4	El backend inyecta sus credenciales de seguridad y redirige el paquete hacia la API de Piston.
	1.5	Piston levanta un contenedor efímero, ejecuta el código aislado y captura la salida de la consola.
	1.6	Piston retorna la respuesta (stdout o stderr) al backend, quien la reenvía al frontend.
	1.7	El sistema muestra los resultados en el panel de la consola en la pantalla del estudiante.
Postcondición	El estudiante visualiza el comportamiento de su código para decidir si lo corrige o lo envía para la validación final (Calificación automática).	
Excepciones	Paso	Acción
	1.3.a	Abuso de peticiones (Spam): Si el usuario supera el límite de intentos, el sistema bloquea el botón y muestra: "Has alcanzado el límite. Espera 60 segundos antes de volver a ejecutar".
	1.5.a	Time-Out (Bucle Infinito): Si el código excede el tiempo límite (5s), el sistema corta la ejecución forzosamente y muestra en consola: Error: Execution Timed Out.
	1.5.b	Fallo de Memoria (OOM): Si el código satura la RAM (excede 256 MB), el sistema devuelve: Error: Memory Limit Exceeded.
	1.6.a	Servicio Piston caído: Si el motor de ejecución externo falla, el sistema notifica: "El motor de ejecución no está disponible en este momento. Inténtalo más tarde".



Caso de uso	Módulo	ID	Nombre
	Ejecución de Código	CU-11	Visualizar resultados en consola
Descripción del caso de uso	Permite al estudiante visualizar la salida estándar (stdout) o los mensajes de error (stderr) generados por su script en un panel de consola integrado, lo que le ayuda a depurar su lógica antes de la validación final.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario que analiza la salida de su código para verificar si funciona como esperaba.
	Sistema	Secundario	Encargado de formatear y renderizar los datos crudos devueltos por el Sandbox en una interfaz legible.
Precondición	El estudiante debe haber ejecutado su código (CU-10) y el frontend debe haber recibido la respuesta del servidor.		
Entradas	Paquete de datos (JSON) proveniente del backend que contiene: stdout (salida normal), stderr (errores) y código de estado.		
Salidas	Texto renderizado en el panel inferior del editor ("Consola de resultados").		
Reglas implicadas	RF-08 (Visualización de resultados): El sistema debe mostrar stdout y stderr claramente diferenciados .		
	RNF-02 (Latencia de ejecución): Todo el proceso de muestra del resultado debe finalizar en menos de 5 segundos.		
Criterios de aceptación (QA)	• Los mensajes de error (stderr) deben mostrarse en un color de advertencia (ej. texto rojo) para contrastar visualmente con la salida estándar (stdout).		
	• El panel de consola debe tener un comportamiento de desplazamiento automático (auto-scroll) hacia la última línea si la salida de texto es muy extensa.		
	• El sistema debe sanitizar la salida para prevenir inyección de código (XSS) en caso de que el usuario intente imprimir etiquetas HTML maliciosas.		
Flujo principal	Paso	Acción	
	1.1	El sistema recibe la carga útil (payload) de respuesta desde el motor de ejecución.	
	1.2	El sistema analiza el payload para determinar si la ejecución fue exitosa (stdout) o si arrojó una excepción (stderr).	
	1.3	El sistema procesa y formatea el texto, preservando los saltos de línea y tabulaciones originales del código impreso.	
	1.4	El sistema inyecta el texto formateado en el panel visual de la consola en la interfaz del estudiante.	
	1.5	El estudiante lee el resultado devuelto para comparar mentalmente si coincide con la salida esperada del ejercicio.	



Postcondición	El estudiante cuenta con la información necesaria para decidir si modifica su código (regresando al CU-09) o si procede a presionar el botón de "Validar" para la calificación oficial (Siguiendo CU).	
Excepciones	Paso	Acción
	1.2.a	Ejecución sin salida: Si el código es válido pero no contiene instrucciones de impresión (ej. falta un comando print o echo), el sistema muestra un mensaje neutral: "Ejecución finalizada con éxito. No se detectó salida en consola".
	1.3.a	Salida truncada: Si el script genera una cantidad excesiva de texto (ej. miles de líneas), el sistema trunca la visualización por seguridad y añade al final: "... [Salida truncada por límite de visualización]".
	1.4.a	Panel minimizado: Si el estudiante tenía el panel de resultados colapsado, el sistema lo expande automáticamente o muestra un indicador visual (ej. un punto rojo) para avisar que hay un nuevo resultado disponible.

Caso de uso	Módulo	ID	Nombre
	Evaluación	CU-12	Validar solución
Descripción del caso de uso	Permite al sistema evaluar de manera automatizada el código final del estudiante, comparando su salida de ejecución con los casos de prueba esperados para determinar si el nivel ha sido superado o si requiere correcciones.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario que envía su código finalizado para obtener una calificación oficial.
	Sistema	Secundario	Encargado de procesar la lógica de comparación y emitir el veredicto.
Precondición	El estudiante debe haber escrito una solución en el editor (CU-09) y encontrarse dentro de un nivel activo.		
Entradas	Acción de clic en "Enviar Solución / Validar", Salida estándar generada por el Sandbox (stdout), ID del nivel.		
Salidas	Veredicto de evaluación (Aprobado o Fallido); Mensaje de retroalimentación o pista; Actualización de estado en BD.		
Reglas implicadas	RF-16 (Validación automática): Comparación directa de la salida del usuario vs salida esperada.		
	RF-17 (Retroalimentación): Ofrecer pistas si el código es incorrecto.		
	RB-04 (Criterio de éxito): Se aprueba solo si las salidas coinciden textualmente.		



Criterios de aceptación (QA)	El comparador de texto debe ser insensible a espacios en blanco o saltos de línea innecesarios al final de la cadena (función trim()).	
	El sistema debe distinguir entre mayúsculas y minúsculas (<i>case-sensitive</i>) si el ejercicio lo requiere.	
	El veredicto debe mostrarse en pantalla casi instantáneamente (latencia menor a 1 segundo) tras procesar la ejecución.	
Flujo principal	Paso	Acción
	1.1	El estudiante presiona el botón "Validar" en la interfaz del editor.
	1.2	El sistema ejecuta el código de forma oculta en el Sandbox y captura la salida final (stdout).
	1.3	El sistema consulta en la base de datos el "Caso de Prueba Esperado" asociado al ID del nivel actual.
	1.4	El sistema limpia ambas cadenas de texto (salida del alumno y salida esperada) eliminando espacios residuales en los extremos.
	1.5	El sistema compara ambas cadenas mediante igualdad estricta.
	1.6	Al coincidir al 100%, el sistema marca el reto con el estado "Completado".
	1.7	El sistema despliega un mensaje de éxito ("¡Excelente trabajo!") y dispara el proceso de gamificación (asignación de XP y desbloqueo del siguiente nivel).
Postcondición	El nivel actual queda registrado como superado en el historial del estudiante, y este queda habilitado para avanzar en la ruta.	
Excepciones	Paso	Acción
	1.2.a	Salida incorrecta: Si las cadenas no coinciden, el sistema detiene el avance, marca el intento como fallido y muestra un mensaje de retroalimentación: "Tu salida no es la esperada. Pista: [Pista predefinida del nivel]".
	1.3.a	Error de sintaxis durante validación: Si el código enviado contiene errores crudos (stderr), el sistema no realiza la comparación lógica y notifica: "Corrige los errores de tu código antes de validarlo".
	1.4.a	Caso de prueba no configurado: Si la base de datos no tiene una respuesta esperada configurada (error administrativo), el sistema bloquea la validación y sugiere reportar el nivel.

Caso de uso	Módulo	ID	Nombre
	Evaluación / Pedagógico	CU-13	Recibir retroalimentación



Descripción del caso de uso	Permite al estudiante obtener mensajes de orientación y pistas específicas tras un intento fallido de validación, facilitando la identificación de errores lógicos o sintácticos en su código.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario que requiere guía para corregir su solución.
	Sistema	Secundario	Encargado de seleccionar y mostrar el mensaje de ayuda adecuado según el tipo de error detectado.
Precondición	El sistema debe haber procesado una validación (CU-12) cuyo resultado fue "Fallido" o con errores de ejecución.		
Entradas	Resultado de la comparación (CU-12), Tipo de error detectado (Sintaxis vs. Lógica), ID del nivel.		
Salidas	Mensaje de retroalimentación en pantalla; Pista visual (opcional); Resaltado de líneas con error (si aplica).		
Reglas implicadas	RF-17 (Retroalimentación): El sistema debe ofrecer pistas basadas en el error cometido.		
	RNF-12 (Lenguaje claro): Los mensajes deben ser comprensibles para principiantes, evitando códigos de error crudos del sistema.		
Criterios de aceptación (QA)	• El mensaje de retroalimentación no debe revelar la solución exacta del reto, solo guiar al usuario.		
	• Si el error es de sintaxis, el sistema debe indicar la línea probable del error.		
	• El panel de retroalimentación debe ser claramente visible (ej. un cuadro amarillo de "Tip" o "Pista").		
Flujo principal	Paso	Acción	
	1.1	El sistema detecta que la salida del código del estudiante no coincide con la esperada (Resultado de CU-12).	
	1.2	El sistema analiza el tipo de fallo: ¿Es un error de compilación/interpretación o una salida de texto incorrecta?	
	1.3	El sistema recupera de la base de datos la "Pista Pedagógica" configurada para ese nivel específico.	
	1.4	El sistema formatea el mensaje de error técnico convirtiéndolo a un lenguaje amigable (ej. "Olvidaste cerrar un paréntesis" en lugar de SyntaxError: unexpected EOF).	
	1.5	El sistema muestra la retroalimentación en un panel destacado cerca del editor de código.	
	1.6	El estudiante lee la pista y procede a realizar los ajustes necesarios en su código.	
Postcondición	El estudiante cuenta con información cualitativa para realizar un nuevo intento de resolución.		
Excepciones	Paso	Acción	



	1.2.a	Error desconocido: Si el error no está catalogado, el sistema muestra un mensaje genérico: "Revisa tu lógica e intenta comparar tu salida con el ejemplo de las instrucciones".
	1.3.a	Múltiples errores: El sistema prioriza mostrar primero el error de sintaxis (que impide la ejecución) antes que los errores de lógica (salida incorrecta).
	1.5.a	Límite de pistas: Si el usuario falla repetidamente (más de 3 veces), el sistema puede mostrar una "Pista Reforzada" o un enlace a la documentación del lenguaje.

Caso de uso	Módulo	ID	Nombre
	Progreso / Gamificación	CU-14	Guardar progreso
Descripción del caso de uso	Permite al sistema registrar de forma permanente los logros, la experiencia (XP) obtenida y el estado de completitud de los niveles, asegurando que el estudiante pueda retomar su aprendizaje desde cualquier dispositivo.		
Actores	Actor	Tipó	Descripción
	Sistema	Primario	Actúa automáticamente tras una validación exitosa para persistir los datos.
	Estudiante	Secundario	Beneficiario de la persistencia de su avance académico.
Precondición	El estudiante debe haber superado la validación de un reto (CU-12) y tener una sesión activa.		
Entradas	ID de usuario, ID de nivel superado, puntos de XP asignados al reto.		
Salidas	Registro actualizado en la BD; Confirmación de "Progreso guardado" en la interfaz.		
Reglas implicadas	RF-05 (Guardado de progreso): Los niveles, XP y logros deben persistir y sincronizarse.		
	RF-18 (Ganar XP): Asignación de puntos tras completar retos.		
	RB-07 (Registro de intentos): Historial de cuántas veces se intentó el reto antes de superarlo.		
Criterios de aceptación (QA)	El sistema debe realizar la actualización de forma atómica (si falla el guardado de XP, no se marca el nivel como completado).		
	El progreso debe estar disponible inmediatamente tras iniciar sesión en un dispositivo diferente (Sincronización).		



	No se deben sumar puntos de XP duplicados por completar un nivel que ya fue superado previamente.	
Flujo principal	Paso	Acción
	1.1	El sistema detecta que el veredicto del CU-12 fue "Correcto".
	1.2	El sistema verifica en la BD si es la primera vez que el estudiante completa dicho nivel.
	1.3	El sistema calcula la cantidad de XP a otorgar (basado en la dificultad del nivel y bonos por primer intento).
	1.4	El sistema actualiza la tabla de progreso del estudiante, marcando el nivel actual como "Superado".
	1.5	El sistema incrementa el contador de XP total en el perfil del usuario.
	1.6	El sistema dispara la señal de "Éxito" al frontend para actualizar la barra de progreso visual.
Postcondición	Los datos del usuario están actualizados en la base de datos y se habilitan los procesos de desbloqueo de niveles (CU-15).	
Excepciones	Paso	Acción
	1.2.a	Nivel ya completado: Si el usuario repite el nivel, el sistema guarda el intento pero no suma XP adicional ni cambia el estado de completitud.
	1.3.a	Fallo de conexión con BD: El sistema intenta reintentar la operación 3 veces. Si persiste, notifica al usuario: "Progreso no guardado. Verifica tu conexión".
	1.4.a	Error de integridad: Si el XP calculado es negativo o incoherente, el sistema cancela la transacción y registra un log de error técnico.

Caso de uso	Módulo	ID	Nombre
	Progreso / Gamificación	CU-15	Ganar XP y subir de nivel
Descripción del caso de uso	Permite al sistema calcular la experiencia obtenida por el estudiante tras resolver retos y, en caso de alcanzar el umbral necesario, incrementar su nivel de rango dentro de la plataforma.		
Actores	Actor	Tipó	Descripción
	Sistema	Primario	Realiza el cálculo matemático y actualiza los estados del perfil.
	Estudiante	Secundario	Recibe el reconocimiento visual y el aumento de estatus académico.
Precondición	El estudiante debe haber completado con éxito un reto (CU-12) y el sistema debe haber registrado el progreso (CU-14).		



Entradas	XP base del nivel superado, multiplicadores por desempeño (si aplica), XP actual del usuario.	
Salidas	Valor de XP actualizado; Nuevo nivel de usuario (si se supera el umbral); Notificación visual de "Level Up".	
Reglas implicadas	RF-18 (Ganar XP): Los puntos se otorgan proporcionalmente a la dificultad del reto.	
	RF-19 (Subir nivel): El sistema debe manejar una escala de niveles (ej. cada 1000 XP).	
	RB-08 (Escalabilidad): La cantidad de XP necesaria aumenta progresivamente en cada nivel.	
Criterios de aceptación (QA)	El cálculo debe ser exacto y reflejarse inmediatamente en la interfaz del estudiante.	
	Si el usuario sube de nivel, se debe disparar una animación o efecto visual (feedback positivo).	
	El sistema debe validar que el usuario no gane XP por el mismo nivel más de una vez (anti-farreo).	
Flujo principal	Paso	Acción
	1.1	El sistema recupera el XP base definido para el ejercicio recientemente aprobado.
	1.2	El sistema suma el XP obtenido al total acumulado en el perfil del estudiante.
	1.3	El sistema consulta la Tabla de Umbrales de Nivel para verificar si el nuevo total supera el requisito del siguiente nivel.
	1.4	Condición: Si el total supera el umbral, el sistema incrementa el atributo nivel_id en la cuenta del usuario.
	1.5	El sistema registra el evento en el historial de actividades del estudiante.
	1.6	El sistema envía una señal al frontend para mostrar la barra de progreso llenándose y, si aplica, el banner de "Nuevo Nivel Alcanzado".
Postcondición	El perfil del estudiante muestra su nueva jerarquía y experiencia, lo que aumenta su competitividad y compromiso con la plataforma.	
Excepciones	Paso	Acción
	1.3.a	Umbral no alcanzado: El sistema solo actualiza la barra de progreso y el número de XP, manteniendo al usuario en el nivel actual.
	1.4.a	Nivel Máximo: Si el usuario ya se encuentra en el nivel más alto disponible, el sistema sigue acumulando XP pero no incrementa el número de nivel.
	1.2.a	Error de cálculo: Si los datos recibidos son inconsistentes (ej. XP negativo), el sistema aborta la operación y mantiene los valores previos.

Caso de uso	Módulo	ID	Nombre
-------------	--------	----	--------



	Aprendizaje / Progresión	CU-16	Desbloquear niveles
Descripción del caso de uso	Permite habilitar el acceso al siguiente reto de una ruta de aprendizaje (Python o PHP) una vez que el estudiante ha demostrado competencia en el nivel inmediatamente anterior.		
Actores	Actor	Tipó	Descripción
	Sistema	Primario	Actúa de forma automática al validar la progresión del estudiante.
	Estudiante	Secundario	Usuario que recibe el acceso a nuevo contenido educativo.
Precondición	El estudiante debe haber completado con éxito la validación del nivel actual (CU-12) y el guardado de progreso (CU-14).		
Entradas	ID del nivel completado, ID del estudiante, ID de la ruta tecnológica.		
Salidas	Estado del nivel \$n+1\$ actualizado a "Disponible" en la base de datos; Notificación visual en el mapa.		
Reglas implicadas	RF-15 (Bloqueo secuencial): No se puede acceder al nivel \$n\$ si el nivel \$n-1\$ no está marcado como completado.		
	RF-11/RF-12 (Rutas): Cada ruta debe tener una secuencia lógica de 10 niveles.		
Criterios de aceptación (QA)	• El sistema no debe desbloquear niveles de una ruta diferente (ej. terminar Python 1 no desbloquea PHP 2).		
	• El cambio de estado debe ser persistente (si el usuario recarga la página, el nivel debe seguir desbloqueado).		
	• El mapa de progreso (CU-05) debe actualizarse visualmente eliminando el icono de candado del nuevo nivel.		
Flujo principal	Paso	Acción	
	1.1	El sistema confirma que el registro del nivel actual ha sido marcado como "Completado" en la tabla de progreso.	
	1.2	El sistema identifica el ID del siguiente nivel dentro del orden secuencial de la ruta activa.	
	1.3	El sistema verifica si existe un nivel posterior; si existe, crea o actualiza un registro de permiso para el usuario.	
	1.4	El sistema cambia el atributo de acceso de Bloqueado a Disponible para el nivel identificado.	
	1.5	El sistema envía un mensaje al frontend notificando que un nuevo reto ha sido habilitado.	
	1.6	El sistema actualiza la vista del mapa de niveles para que el estudiante pueda hacer clic en el nuevo reto.	
Postcondición	El estudiante tiene acceso interactivo al siguiente reto de su formación técnica.		
Excepciones	Paso	Acción	
	1.3.a	Fin de la ruta: Si el usuario completó el último nivel disponible (ej. Nivel 10), el	



		sistema no intenta desbloquear nada y felicita al usuario por terminar la ruta.
	1.4.a	Error de base de datos: Si falla la actualización del permiso, el sistema reintenta la operación para evitar que el usuario se quede "atascado" en el progreso.
	1.5.a	Nivel ya desbloqueado: Si el usuario repite un nivel que ya había superado, el sistema ignora el flujo de desbloqueo para evitar redundancia de datos.

Caso de uso	Módulo	ID	Nombre
	Gamificación	CU-17	Obtener logros e insignias
Descripción del caso de uso	Permite al sistema otorgar reconocimientos visuales (insignias) al estudiante cuando cumple hitos específicos, como completar su primera ruta, mantener una racha de días o resolver retos complejos, enriqueciendo su perfil público.		
Actores	Actor	Tipó	Descripción
	Sistema	Primario	Monitoriza los eventos y dispara la adjudicación de insignias.
	Estudiante	Secundario	Usuario que recibe el reconocimiento por su desempeño.
Precondición	El estudiante debe estar autenticado y haber realizado una acción que dispare un hito (ej. CU-12 exitoso o inicio de sesión consecutivo).		
Entradas	ID de usuario, Evento disparador (ej. ruta_completada, primer_intento_exitoso), Catálogo de insignias.		
Salidas	Insignia agregada a la colección del perfil; Notificación tipo <i>Pop-up</i> de logro obtenido.		
Reglas implicadas	RF-19 (Logros): El sistema debe manejar un catálogo de al menos 5 insignias iniciales.		
	RF-04 (Visualización): Las insignias deben ser visibles en el perfil del usuario.		
	RB-09 (Unicidad): Cada logro solo puede ser obtenido una vez por cuenta.		
Criterios de aceptación (QA)	La adjudicación del logro debe ser automática y no requerir intervención manual.		
	Al obtener una insignia, el sistema debe actualizar el contador de logros en el perfil de forma inmediata.		
	El diseño de la insignia debe cambiar de "escala de grises" a "color" en la galería del usuario tras ser obtenida.		
Flujo principal	Paso	Acción	
	1.1	El sistema detecta un evento clave (ej. el estudiante acaba de completar el nivel 10 de Python).	



	1.2	El sistema consulta la tabla Logros_Usuario para verificar si el estudiante ya posee la insignia asociada a ese evento.
	1.3	Si no la posee, el sistema inserta un nuevo registro vinculando el ID_Insignia con el ID_Usuario.
	1.4	El sistema registra la fecha y hora exacta de la obtención para el historial de trofeos.
	1.5	El sistema envía una notificación push o un banner animado a la interfaz del estudiante con el mensaje: "¡Nuevo Logro Desbloqueado: Maestro de Python!".
	1.6	El sistema actualiza la galería de insignias en la vista de Perfil (CU-04).
Postcondición	El estudiante cuenta con un nuevo activo digital en su perfil que valida sus competencias adquiridas.	
Excepciones	Paso	Acción
	1.2.a	Logro ya obtenido: Si el sistema detecta que el usuario ya tiene la insignia, ignora el evento y no realiza ninguna acción para evitar duplicidad.
	1.5.a	Error de carga de imagen: Si la imagen de la insignia no carga, el sistema muestra el nombre del logro y una descripción textual como respaldo.
	1.3.a	Logros ocultos: Algunos logros solo se revelan una vez obtenidos; en este caso, el sistema actualiza la base de datos sin previo aviso hasta el momento de la notificación.

Caso de uso	Módulo	ID	Nombre
	Experiencia de Usuario / Interfaz	CU-18	Usar sistema en móvil / responsive
Descripción del caso de uso	Permite al estudiante acceder a todas las funcionalidades de la plataforma (mapa de niveles, editor de código y perfil) desde dispositivos móviles o tablets, garantizando una navegación fluida y una visualización óptima de los contenidos.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario que interactúa con la plataforma mediante gestos táctiles en pantallas pequeñas.
	Sistema (Frontend)	Secundario	Encargado de detectar la resolución de pantalla y reorganizar los componentes visuales dinámicamente.
Precondición	El dispositivo debe contar con un navegador web moderno y conexión a internet.		



Entradas	Gestos táctiles (tap, swipe, scroll), ingreso de texto mediante teclado virtual.	
Salidas	Interfaz adaptada: Menú colapsable (hamburguesa), paneles apilados verticalmente y controles de editor optimizados.	
Reglas implicadas	RF-20 (Interfaz Responsive): Adaptación total a diferentes resoluciones.	
	RF-22 (Navegación táctil): Soporte para desplazamientos y toques en dispositivos móviles.	
	RNF-08 (Compatibilidad): Funcionamiento en navegadores móviles (Chrome/Safari).	
Criterios de aceptación (QA)	No debe existir desbordamiento horizontal (<i>scroll</i> lateral) en la página principal.	
	El editor de código debe ser funcional y permitir la edición de texto sin que el teclado virtual bloquee la visibilidad del botón "Ejecutar".	
	Los elementos interactivos (botones/iconos) deben tener un área de toque mínima de 44x44 píxeles para evitar errores de navegación.	
Flujo principal	Paso	Acción
	1.1	El estudiante accede a la URL de CodeBreaker desde su smartphone o tablet.
	1.2	El sistema detecta el tamaño de la pantalla (Viewport) y aplica las consultas de medios (Media Queries).
	1.3	El sistema oculta la barra de navegación lateral y la reemplaza por un Menú Hamburguesa colapsable.
	1.4	El sistema reorganiza el área de aprendizaje: las instrucciones y el editor, que antes estaban en paralelo, ahora se muestran de forma apilada (uno debajo del otro).
	1.5	El estudiante utiliza gestos de deslizamiento (<i>swipe</i>) para explorar el Mapa de Niveles (CU-05).
	1.6	El estudiante pulsa el editor para redactar código, activándose el teclado virtual con ajustes de indentación asistida.
Postcondición	El estudiante completa sus retos académicos con la misma efectividad técnica que en una computadora de escritorio.	
Excepciones	Paso	Acción
	1.2.a	Resolución no soportada: En pantallas extremadamente pequeñas (menores a 320px), el sistema muestra un aviso sugiriendo rotar el dispositivo.
	1.4.a	Conflicto con teclado virtual: Si el teclado oculta la consola de resultados, el sistema permite minimizar el teclado mediante un botón de "Cerrar" en la barra de herramientas del editor.
	1.6.a	Error de carga de fuentes: Si los iconos no cargan en el móvil, el sistema utiliza etiquetas de texto para asegurar que los botones sigan siendo identificables.

Caso de uso	Módulo	ID	Nombre
-------------	--------	----	--------



	Experiencia de Usuario / Interfaz	CU-19	Navegar con interfaz táctil
Descripción del caso de uso	Permite al estudiante desplazarse por la aplicación utilizando gestos naturales (toque, deslizamiento, pellizco), facilitando la exploración del mapa de niveles y la interacción con el código sin necesidad de periféricos externos.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario que utiliza un dispositivo con pantalla sensible al tacto.
	Sistema (Frontend)	Secundario	Encargado de interpretar los eventos de contacto (Touch Events) y transformarlos en acciones dentro de la UI.
Precondición	El dispositivo debe poseer una pantalla táctil y el sistema debe haber detectado una resolución móvil o tablet (CU-18).		
Entradas	Gestos: <i>Swipe</i> (deslizar), <i>Tap</i> (tocar), <i>Long Press</i> (presión prolongada) y <i>Pinch-to-zoom</i> (pellizcar).		
Salidas	Desplazamiento fluido del mapa, apertura de menús contextuales y escalado de elementos visuales.		
Reglas implicadas	RF-22 (Soporte táctil): Optimización de la respuesta de los componentes.		
	RNF-19 (Adaptabilidad): Los elementos interactivos deben responder a eventos de puntero táctil sin retrasos significativos (latencia baja).		
Criterios de aceptación (QA)	• El mapa de niveles debe permitir un desplazamiento inercial (que siga moviéndose brevemente tras soltar el dedo).		
	• El sistema debe evitar el "zoom accidental" del navegador al tocar repetidamente botones de control.		
	• Los tooltips (información flotante) deben activarse mediante un toque simple o presión prolongada, ya que no existe el evento "hover" del mouse.		
Flujo principal	Paso	Acción	
	1.1	El estudiante utiliza un gesto de deslizamiento vertical u horizontal para explorar la ruta de aprendizaje en el mapa.	
	1.2	El sistema desplaza la vista del mapa de forma fluida, manteniendo los iconos de los niveles alineados.	
	1.3	El estudiante realiza un Tap sobre un nivel disponible para ver un resumen rápido del reto.	
	1.4	El estudiante utiliza el gesto de pellizco (Pinch) para alejar la vista y ver el progreso general de la ruta de un vistazo.	
	1.5	Dentro del editor, el estudiante toca una línea de código y el sistema posiciona el cursor y despliega el teclado virtual automáticamente.	
	1.6	El estudiante desliza el panel lateral para ocultar las instrucciones y ganar más espacio de trabajo.	
Postcondición	El estudiante navega de forma intuitiva, reduciendo la fricción de aprendizaje en entornos móviles.		
Excepciones	Paso	Acción	



	1.1.a	Gesto no reconocido: Si el usuario realiza un gesto inválido, el sistema no realiza ninguna acción y mantiene la vista actual.
	1.3.a	Toque accidental: Si el estudiante toca un nivel bloqueado mientras se desplazaba, el sistema ignora la pulsación para evitar interrupciones innecesarias.
	1.5.a	Selección de texto difícil: Si el estudiante tiene problemas para seleccionar una palabra específica, el sistema habilita "manecillas" de selección (handles) de alta precisión.



Caso de uso	Módulo	ID	Nombre
	Administración de Contenido	CU-20	Gestionar retos (CRUD)
Descripción del caso de uso	Permite al administrador realizar el mantenimiento integral de los ejercicios (Crear, Leer, Actualizar y Eliminar), definiendo la narrativa, el lenguaje, el código inicial y la respuesta esperada para cada nivel.		
Actores	Actor	Tipó	Descripción
	Estudiante	Primario	Usuario con permisos elevados encargado de la gestión académica.
	Sistema	Secundario	Gestiona la persistencia de los cambios en la base de datos y actualiza el mapa de progreso para los estudiantes.
Precondición	El usuario debe haber iniciado sesión con una cuenta de rol "Administrador".		
Entradas	Título, descripción/instrucciones, lenguaje (Python/PHP), código base (boilerplate), salida esperada (regex o string), XP otorgada y número de orden en la ruta.		
Salidas	Lista de retos actualizada; Mensajes de confirmación de operación exitosa.		
Reglas implicadas	RF-23 (Panel Administrativo): Interfaz exclusiva para gestión.		
	RB-11 (Integridad de Ruta): No se puede eliminar un nivel si tiene registros de progreso de estudiantes asociados (integridad referencial).		
	RB-12 (Consistencia): Cada nivel debe tener obligatoriamente una salida esperada para poder ser publicado.		
Criterios de aceptación (QA)	Las modificaciones en un reto existente deben reflejarse en tiempo real para los estudiantes que aún no lo han completado.		
	El sistema debe validar que el "Número de Nivel" no se duplique dentro de la misma ruta tecnológica.		
	Al eliminar un reto, el sistema debe solicitar una doble confirmación para evitar pérdidas accidentales de datos.		
Flujo principal	Paso	Acción	
	1.1	El administrador accede al "Panel de Control" y selecciona la opción "Gestionar Retos".	
	1.2	El sistema muestra una tabla con todos los retos existentes, filtrados por lenguaje y nivel.	
	1.3	Para Crear: El admin pulsa "Nuevo Reto", completa el formulario con la metadata técnica y pedagógica, y presiona "Guardar".	
	1.4	Para Editar: El admin selecciona un reto de la lista, modifica los campos necesarios (ej. corregir una pista o ajustar la salida esperada) y presiona "Actualizar".	
	1.5	Para Eliminar: El admin presiona el icono de papelera y confirma la acción en el cuadro de diálogo emergente.	



	1.6	El sistema valida los datos, actualiza la base de datos y refresca la lista visual.
Postcondición	El repositorio de retos se actualiza, permitiendo que los estudiantes visualicen el contenido nuevo o corregido en sus respectivas rutas.	
Excepciones	Paso	Acción
	1.3.a	Campos incompletos: Si faltan datos obligatorios, el sistema resalta los campos en rojo y bloquea el botón de guardado.
	1.5.a	Reto con dependencias: Si el administrador intenta borrar un reto que ya ha sido completado por estudiantes, el sistema sugiere "Deshabilitar" en lugar de "Eliminar" para preservar el historial académico.
	1.6.a	Error de sintaxis en el código base: El sistema valida que el código inicial proporcionado sea sintácticamente correcto para el lenguaje elegido antes de permitir el guardado.



Caso de uso	Módulo	ID	Nombre
	Administración de Contenido / Evaluación	CU-21	Configurar casos de prueba
Descripción del caso de uso	Permite al administrador definir los conjuntos de entradas y salidas esperadas que el motor de evaluación utilizará para validar el código enviado por el estudiante.		
Actores	Actor	Tipó	Descripción
	Administrador	Primario	Usuario que diseña la lógica de evaluación del reto.
	Sistema	Secundario	Almacena y aplica los criterios de comparación durante el proceso de calificación.
Precondición	El administrador debe estar creando o editando un reto (CU-20).		
Entradas	Entradas simuladas (stdin), Salida esperada (stdout), Tipo de validación (Exacta o por Expresión Regular).		
Salidas	Matriz de pruebas asociada al reto; Indicador de "Configuración válida".		
Reglas implicadas	RF-16 (Validación automática): El sistema debe comparar el resultado del usuario contra estos casos.		
	RB-04 (Criterios de éxito): Un reto se considera superado solo si pasa el 100% de los casos de prueba configurados.		
	RB-13 (Visibilidad): Los casos de prueba pueden marcarse como "Públicos" (vistos por el alumno) o "Ocultos" (para evitar el hardcoding).		
Criterios de aceptación (QA)	• El sistema debe permitir al menos un caso de prueba por reto.		
	• Se debe validar que la salida esperada no sea una cadena vacía.		
	• El administrador debe poder ejecutar una "Prueba de Integridad" para verificar que su propia solución maestra pase los casos configurados.		
Flujo principal	Paso	Acción	
	1.1	El administrador selecciona la pestaña "Configuración de Pruebas" dentro del editor de retos.	
	1.2	El administrador añade un nuevo registro de prueba.	
	1.3	El administrador define la entrada simulada (ej. si el reto pide leer dos números, ingresa 5\n10).	
	1.4	El administrador define la salida esperada (ej. 15).	
	1.5	El administrador selecciona si el caso será Visible para el estudiante (como ejemplo) u Oculto .	
	1.6	El sistema valida que los datos no tengan conflictos de formato y los vincula al ID del reto.	
Postcondición	El reto queda listo para ser publicado y evaluado automáticamente por el motor de CodeBreaker.		



Excepciones	Paso	Acción
	1.3.a	Entrada mal formateada: Si el administrador ingresa caracteres no soportados por el lenguaje (ej. caracteres especiales en Python), el sistema muestra una advertencia.
	1.4.a	Salida ambigua: Si se elige validación exacta, el sistema advierte que cualquier espacio o salto de línea extra en el código del alumno provocará un fallo.
	1.6.a	Fallo de integridad: Si el administrador intenta guardar sin definir al menos un caso de prueba, el sistema bloquea la publicación del reto.



Caso de uso	Módulo	ID	Nombre
	Administración / Reportes	CU-22	Visualizar analíticas académicas
Descripción del caso de uso	Permite al administrador o docente visualizar indicadores clave de rendimiento (KPIs) sobre el progreso de los estudiantes, niveles de dificultad real de los retos y tasas de abandono en las rutas de aprendizaje.		
Actores	Actor	Tipó	Descripción
	Administrador	Primario	Usuario que requiere supervisar el éxito académico de la plataforma.
	Sistema	Secundario	Encargado de procesar grandes volúmenes de datos para generar gráficos y tablas resumen.
Precondición	El usuario debe estar autenticado con rol de Administrador y debe existir actividad previa de estudiantes en la BD.		
Entradas	Filtros de búsqueda (Rango de fechas, Lenguaje, Nivel específico, ID de Estudiante).		
Salidas	Dashboard visual con: Gráficos de barras (niveles completados), tasa de éxito por reto y lista de alumnos destacados (Top XP).		
Reglas implicadas	RF-24 (Reportes): El sistema debe generar reportes exportables.		
	RNF-02 (Rendimiento): Las consultas analíticas no deben degradar el desempeño del sistema para los estudiantes activos.		
Criterios de aceptación (QA)	Las gráficas deben ser interactivas (permitir ver el valor exacto al pasar el cursor).		
	El sistema debe permitir exportar los datos en formato PDF o Excel para auditorías externas.		
	Los datos mostrados deben estar actualizados al menos hasta la última hora de actividad.		
Flujo principal	Paso	Acción	
	1.1	El administrador ingresa a la sección "Analíticas" del Panel de Control.	
	1.2	El sistema realiza consultas agregadas a las tablas de Progreso, Intentos y Usuarios.	
	1.3	El sistema renderiza un Dashboard General mostrando el total de estudiantes activos y el porcentaje global de niveles completados.	
	1.4	El administrador aplica un filtro para ver qué reto de Python tiene la mayor tasa de errores (identificación de cuellos de botella).	
	1.5	El sistema actualiza los gráficos para mostrar el "Top 5 de retos más difíciles" basado en la relación intentos/éxitos.	



	1.6	El administrador genera un reporte en PDF de los 10 estudiantes con mayor XP para su reconocimiento académico.
Postcondición	El administrador obtiene una visión clara del estado educativo de la plataforma para ajustar los retos o apoyar a estudiantes rezagados.	
Excepciones	Paso	Acción
	1.2.a	Base de datos vacía: Si no hay actividad registrada, el sistema muestra el mensaje: "Sin datos suficientes para generar analíticas".
	1.3.a	Tiempo de espera agotado: Si la consulta es muy pesada, el sistema muestra un spinner de carga y utiliza una versión "cacheada" de los datos para no bloquear la interfaz.
	1.6.a	Fallo en exportación: Si no se puede generar el PDF, el sistema permite copiar los datos al portapapeles en formato CSV como alternativa.

3. Modelo Lógico

El modelo lógico del sistema CRM-DJ describe la estructura conceptual y el comportamiento dinámico del software.

Incluye el análisis de objetos, la interacción entre ellos y el flujo de procesos dentro del sistema, asegurando la coherencia entre los requerimientos funcionales y su implementación.

3.1. Analisis de Objetos

Durante el análisis se identificaron los principales objetos del sistema y sus relaciones.

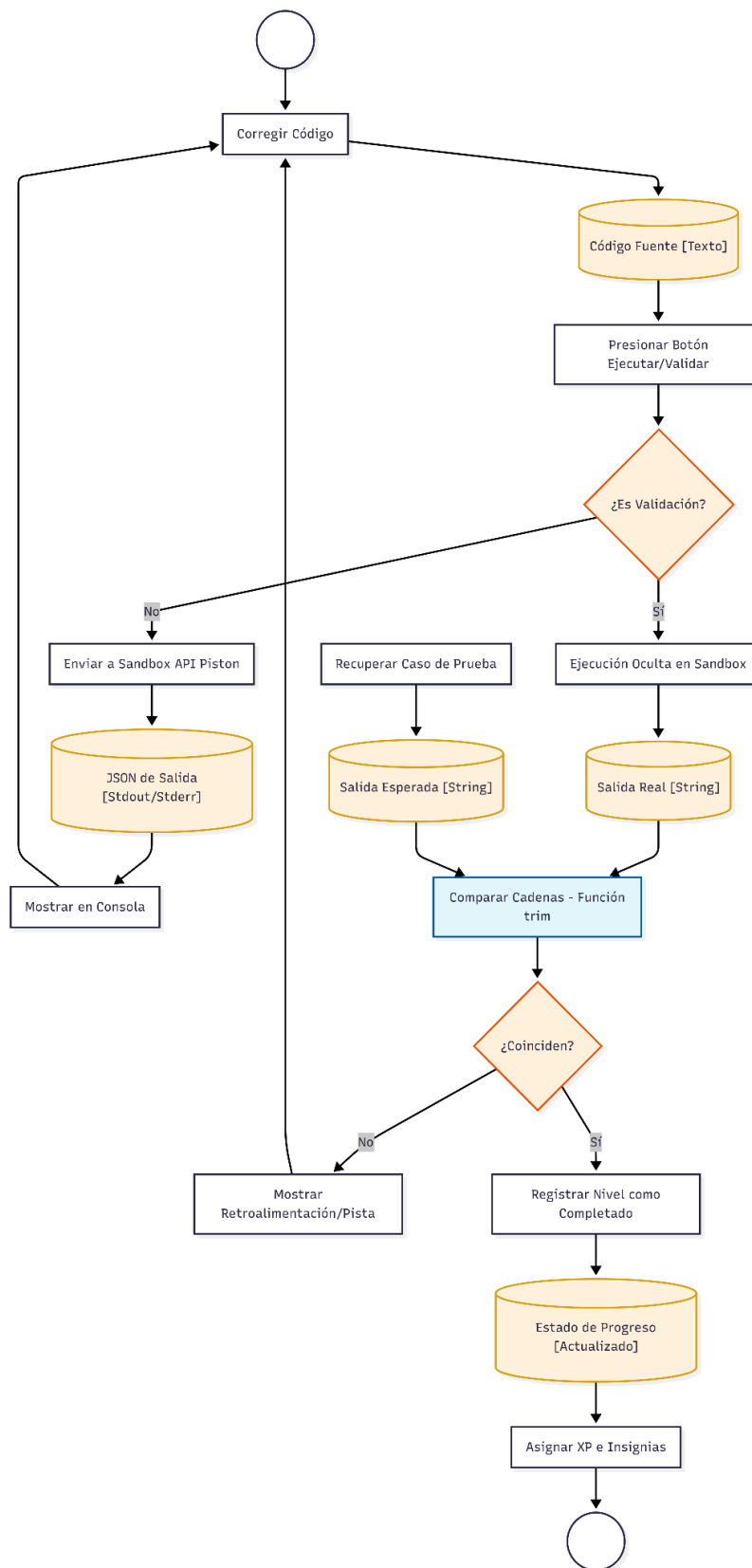
Estos objetos representan las entidades clave del negocio y los elementos que participan en los procesos de reserva, gestión de eventos y pagos.

Objeto	Descripción	Atributos Principales
Usuario	Representa a DJs, clientes o administradores.	idUsuario, nombre, correo, rol, contraseña
Evento	Define el evento o servicio solicitado.	idEvento, nombre, fecha, lugar, tipo, estado
Reserva	Controla la relación entre usuario y evento.	idReserva, idUsuario, idEvento, fechaReserva, estado
Pago	Registra la transacción económica asociada a una reserva.	idPago, idReserva, monto, método, fechaPago
Notificación	Gestiona alertas o mensajes automáticos.	idNotificacion, idUsuario, tipo, mensaje, estado

Estos objetos colaboran entre sí para mantener la **integridad de la información** y permitir la trazabilidad de todas las operaciones.

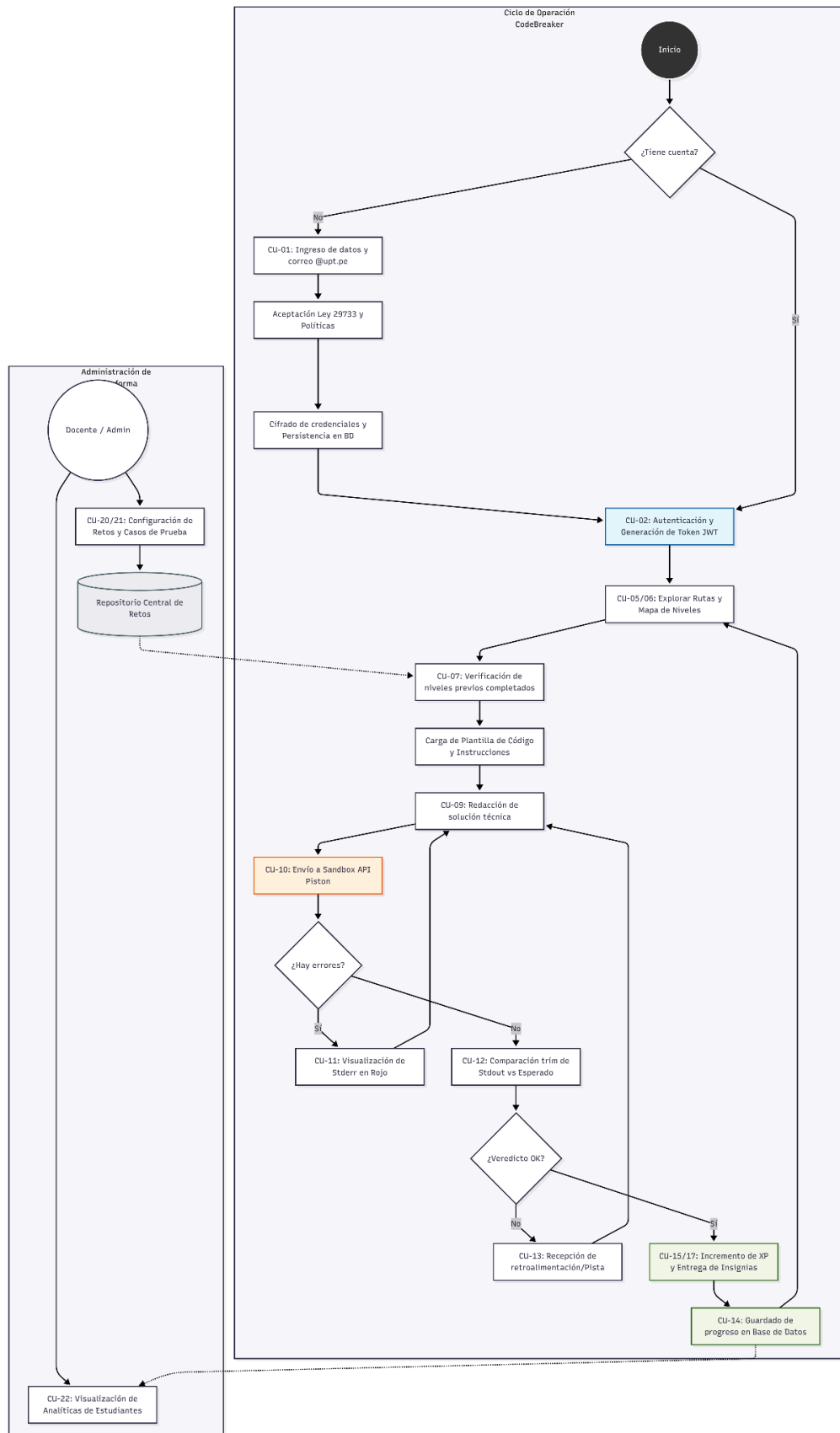


3.2. Diagrama de Actividades con objetos





3.3. Diagrama de Procesos

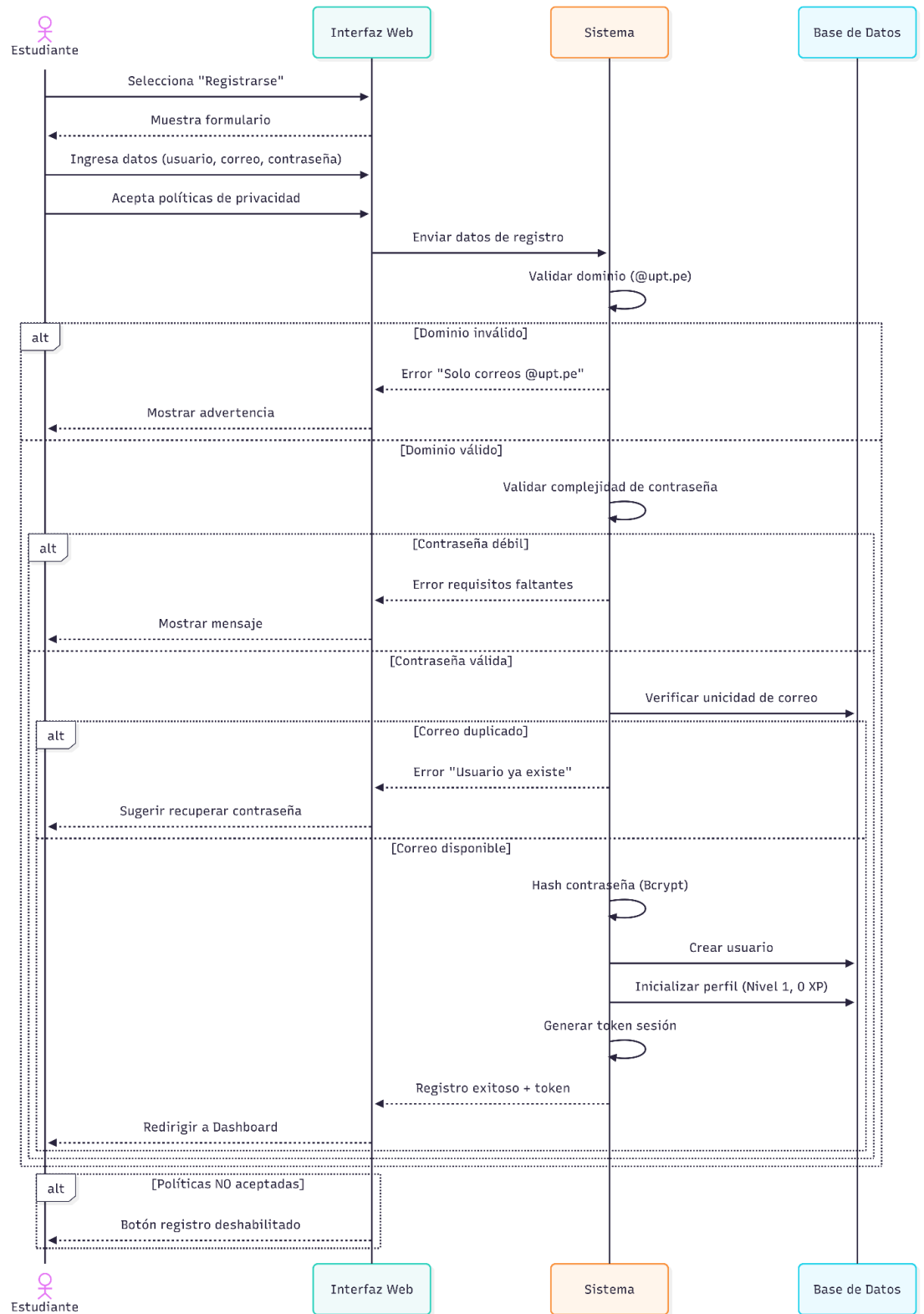




3.4. Diagrama de Secuencia

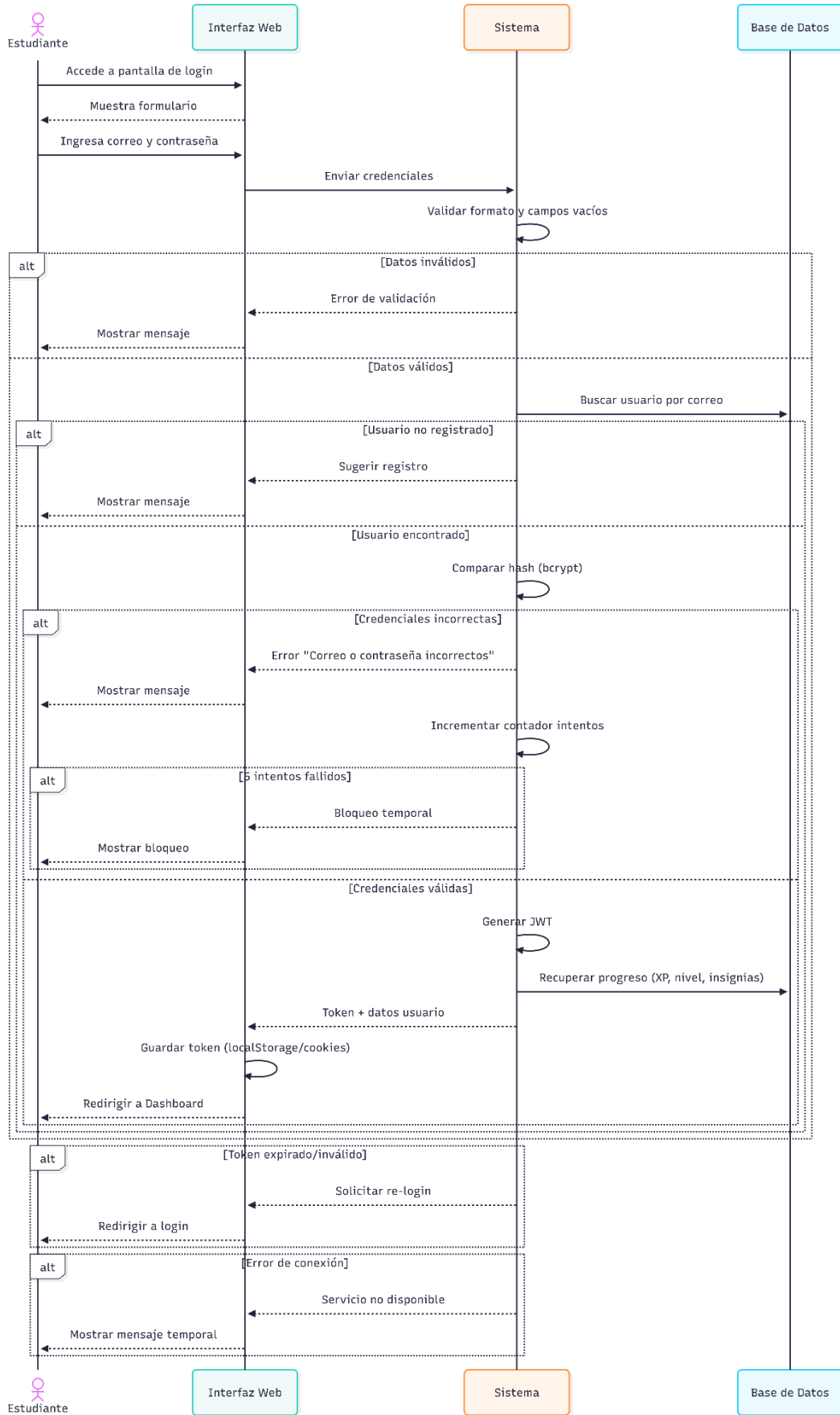
Módulo 1: Gestión de Usuarios (Autenticación y Seguridad)

PR-001 - Registro de Estudiante (CU-01)



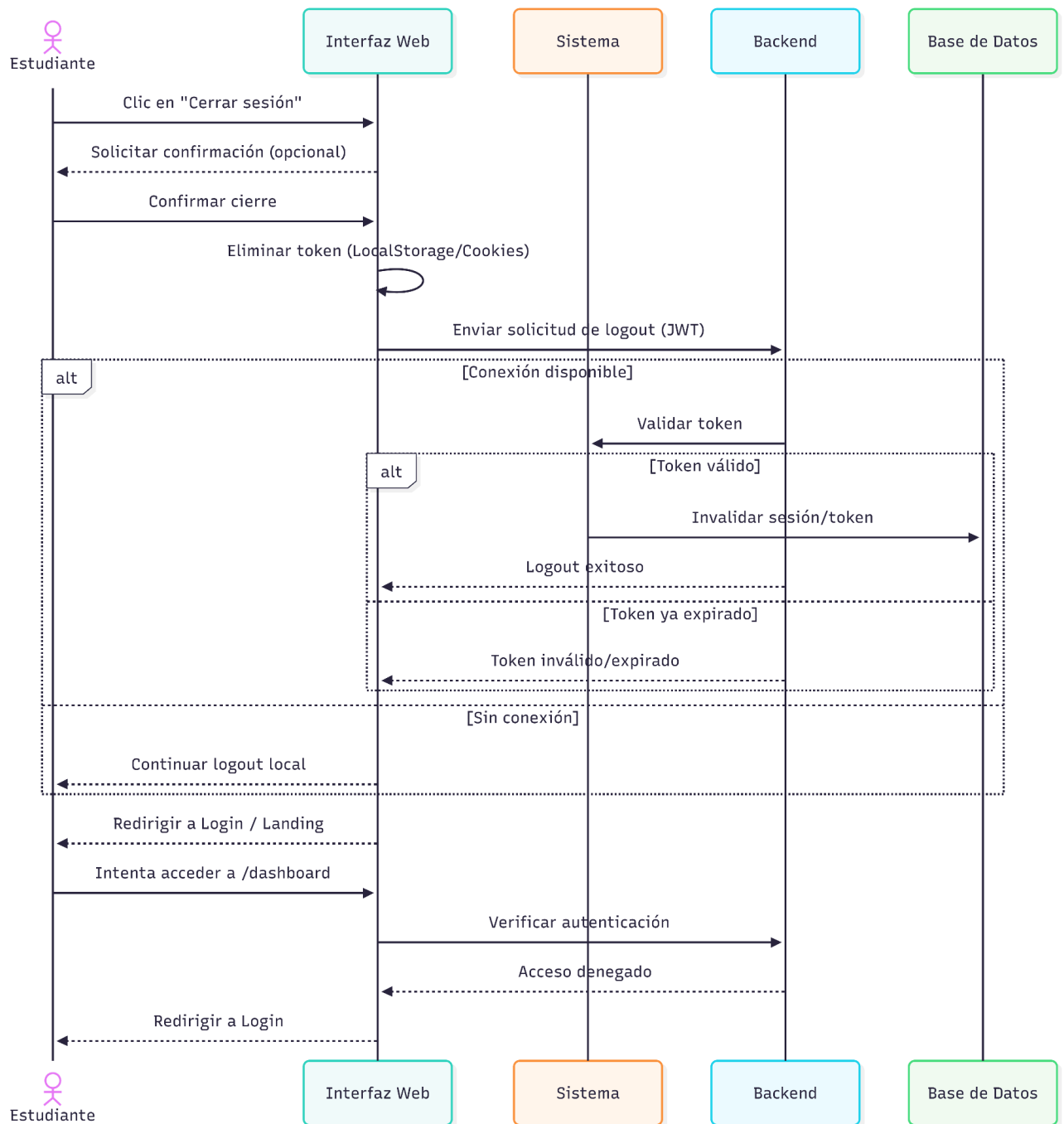


CU-02: Iniciar sesión (generación de token JWT).

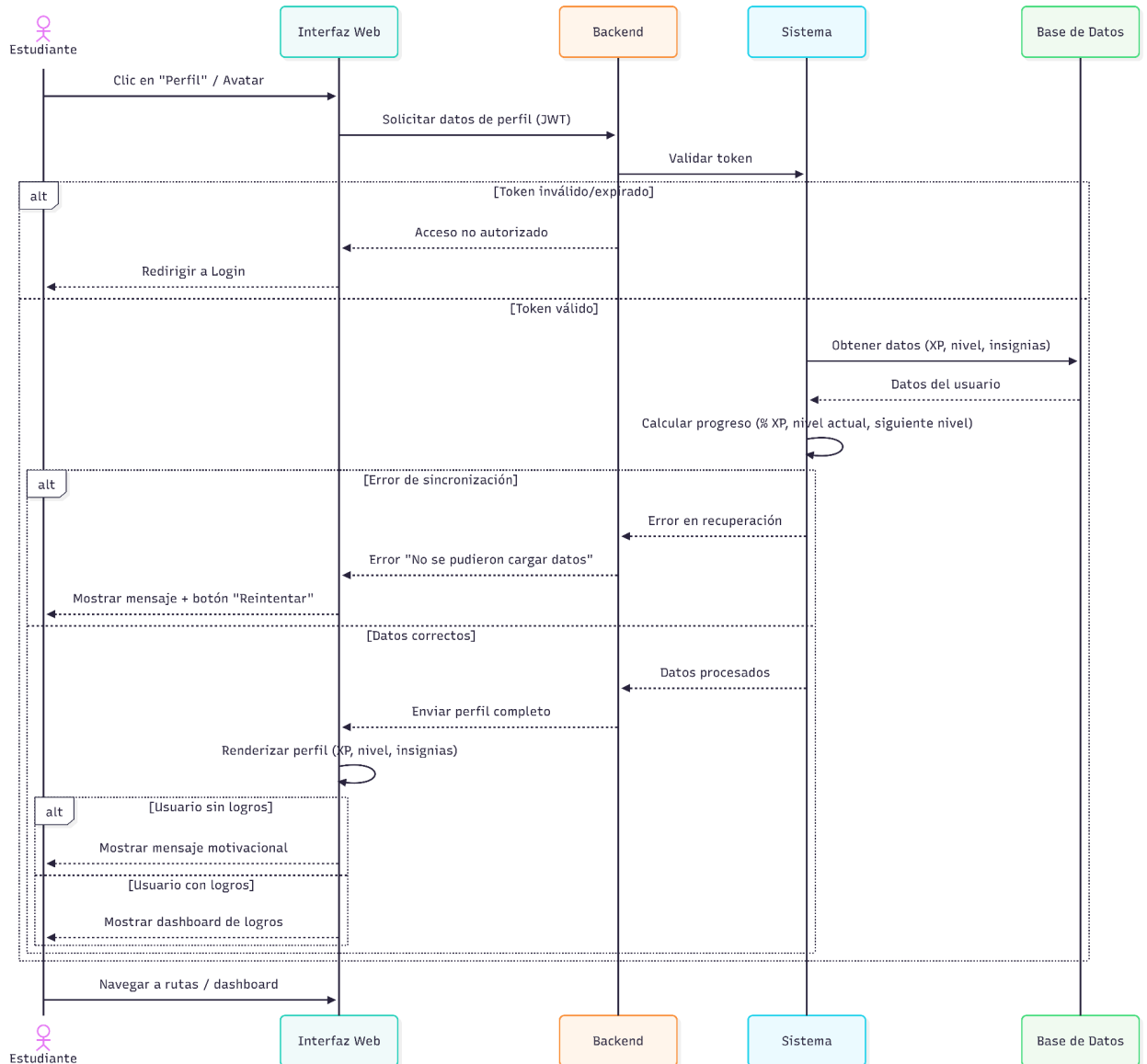




CU-03: Cerrar sesión (invalidación de token).

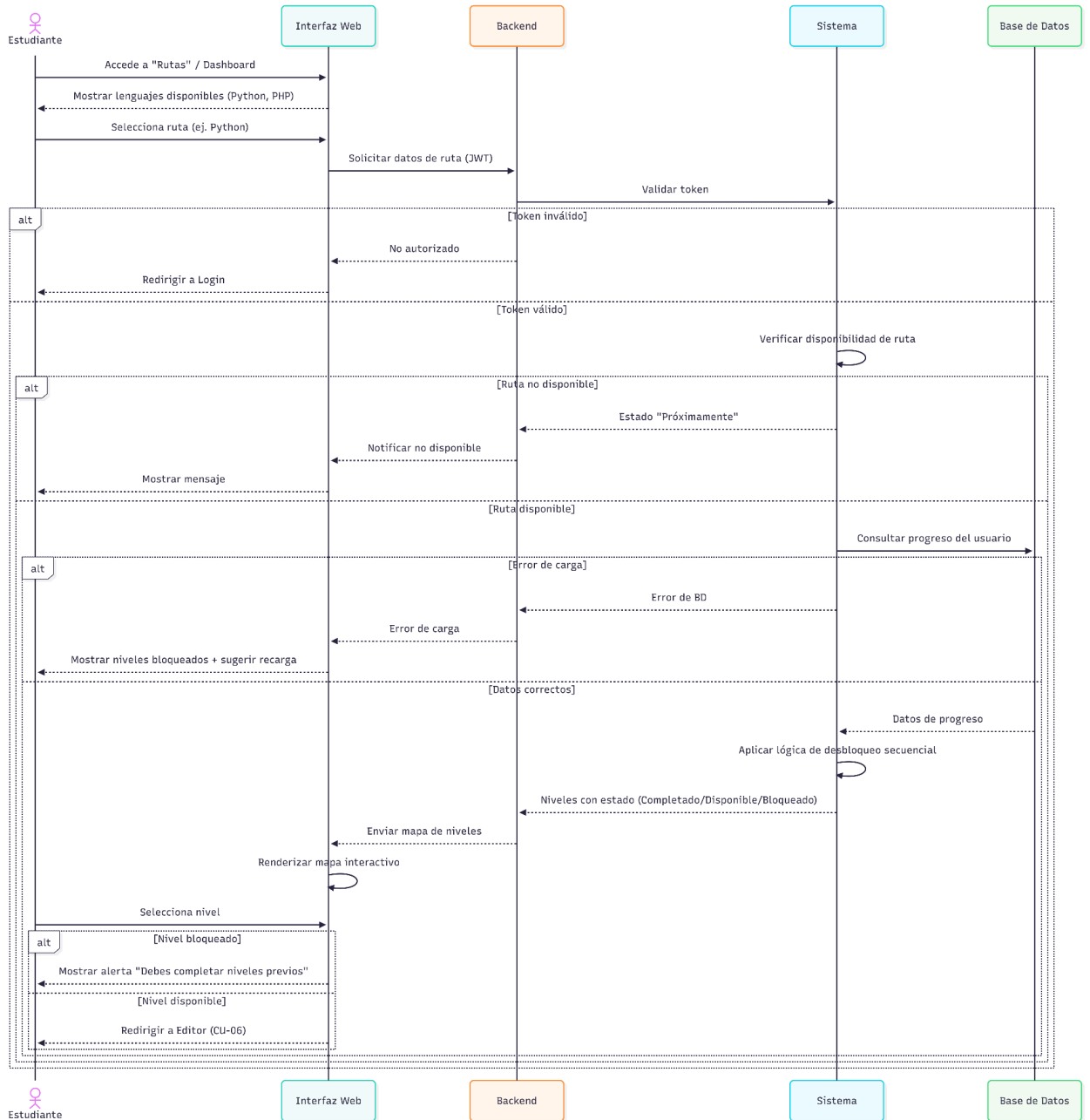


CU-04: Ver perfil de usuario (visualización de estadísticas y avatar).

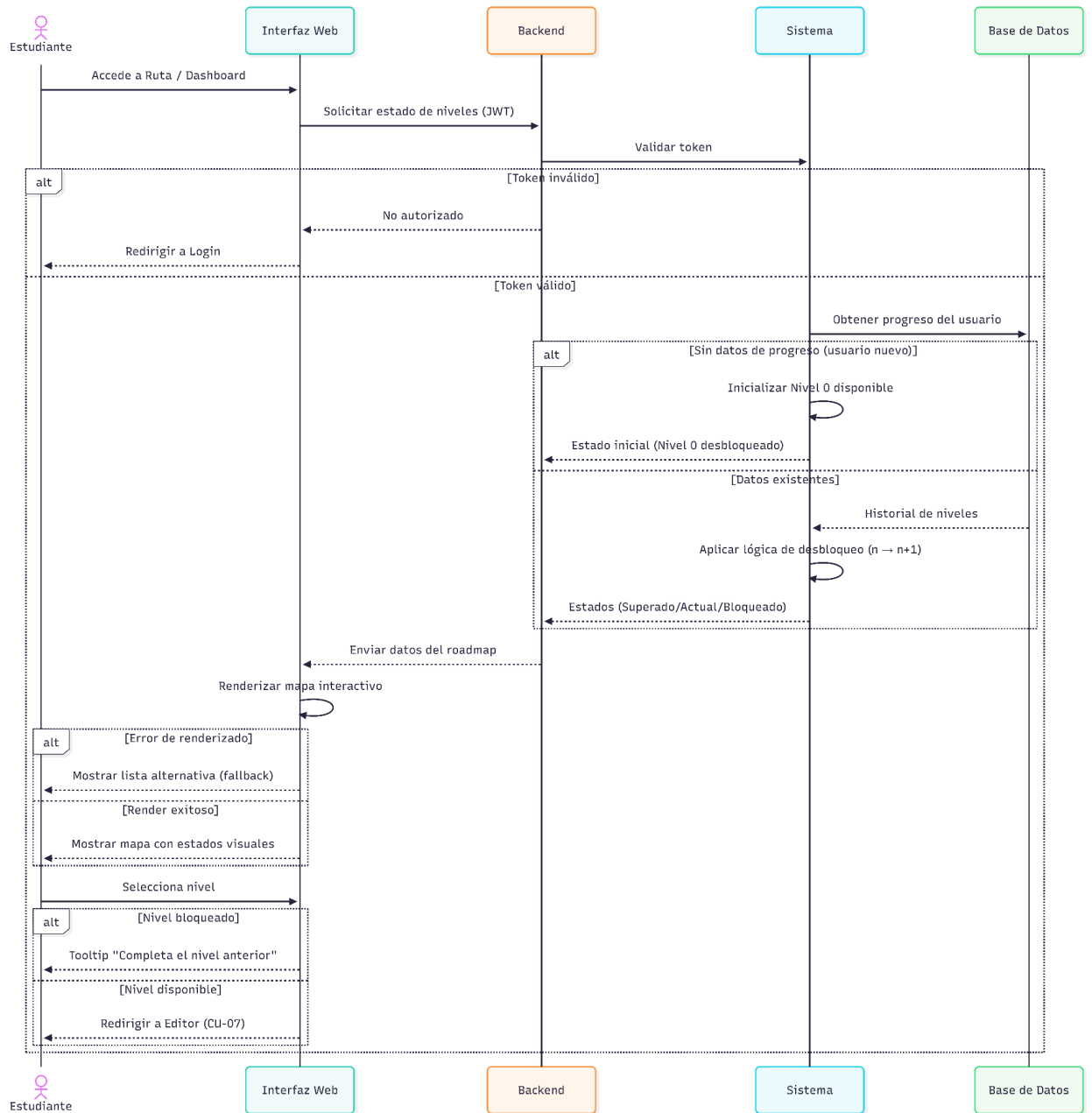


Módulo 2: Aprendizaje y Navegación

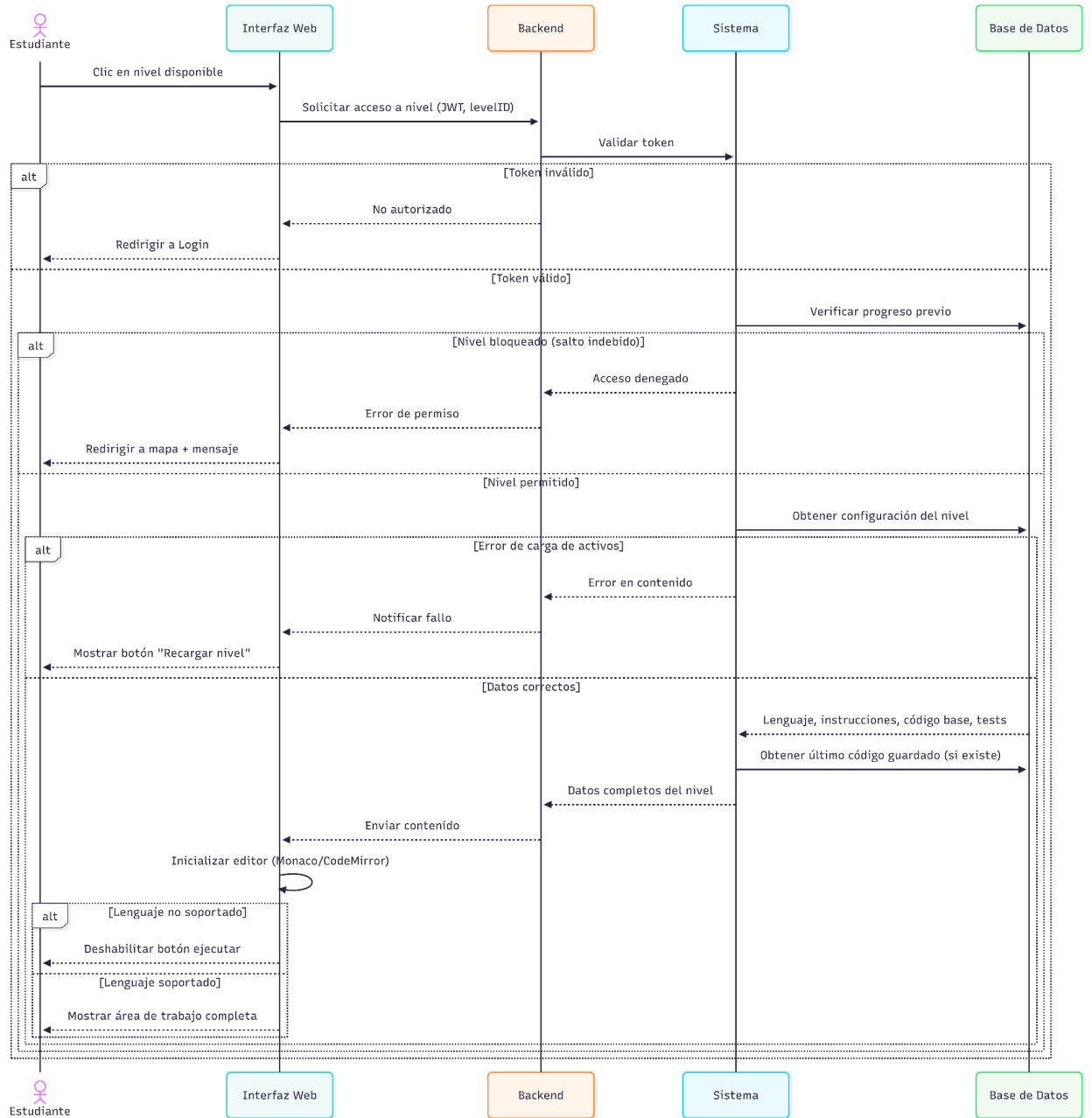
CU-05: Explorar rutas de aprendizaje (selección entre Python y PHP).



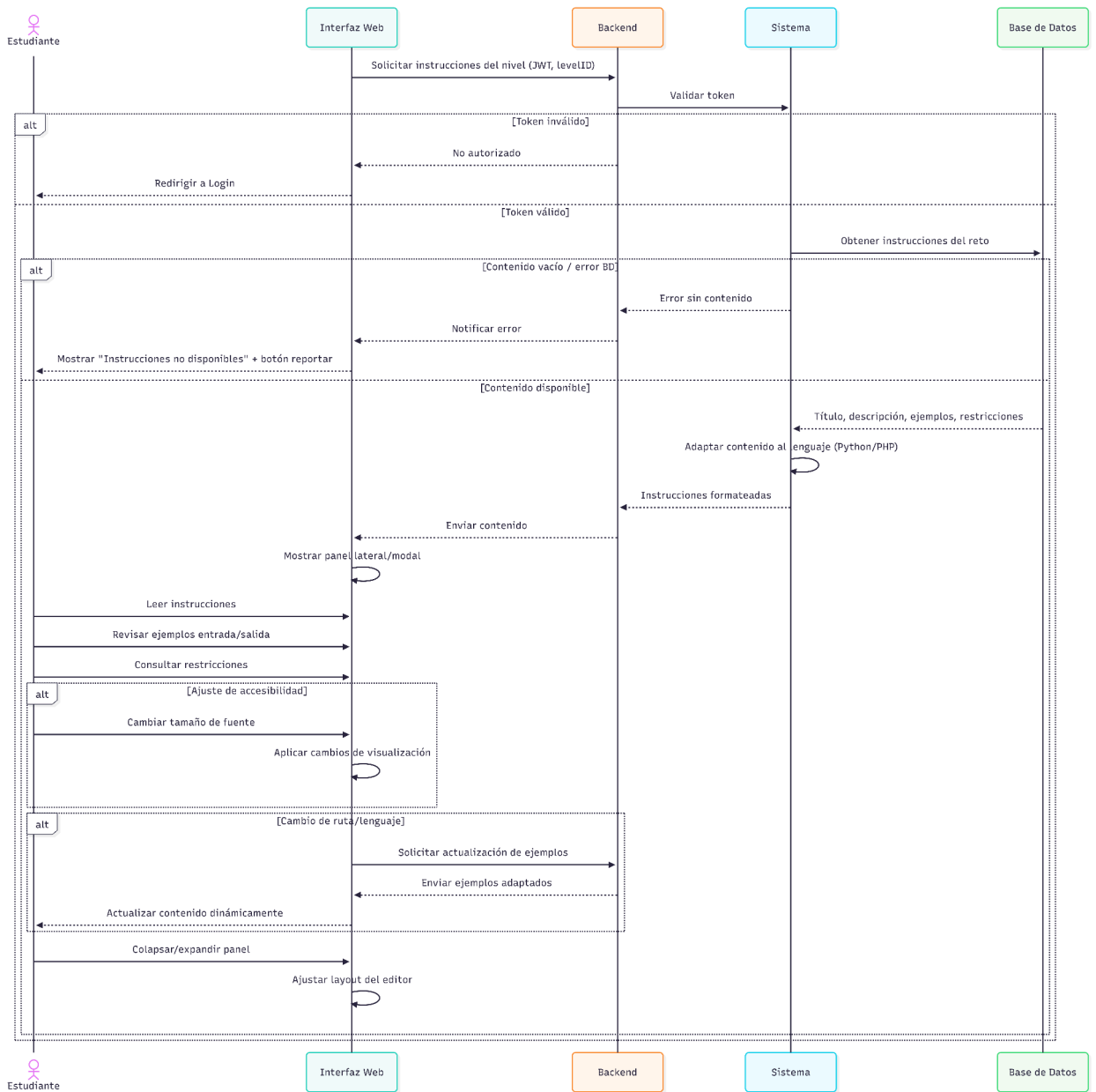
CU-06: Visualizar mapa de progreso (Roadmap interactivo de niveles).



CU-07: Acceder a nivel (carga de entorno de trabajo según requisitos previos).

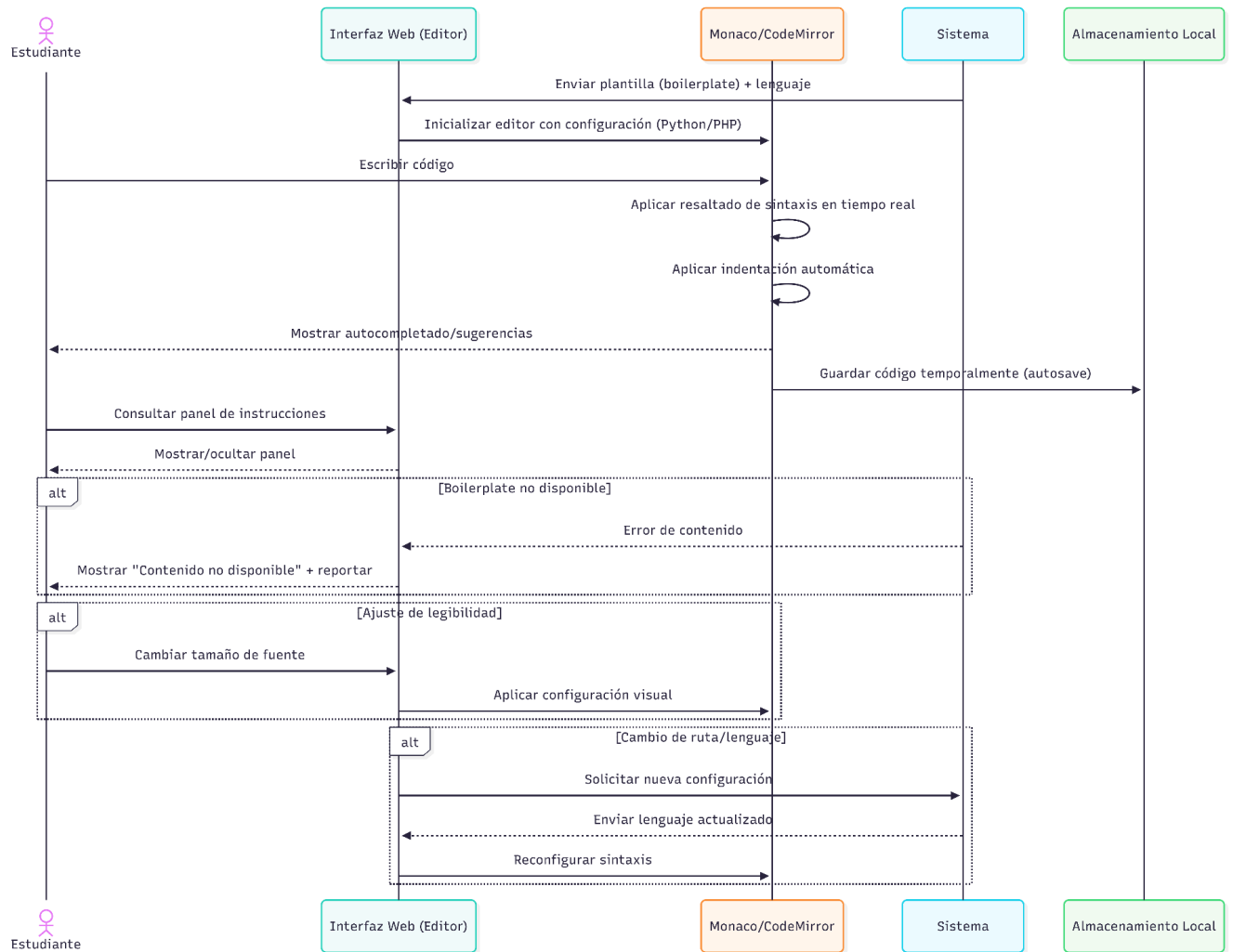


CU-08: Visualizar instrucciones del reto (narrativa pedagógica y ejemplos).

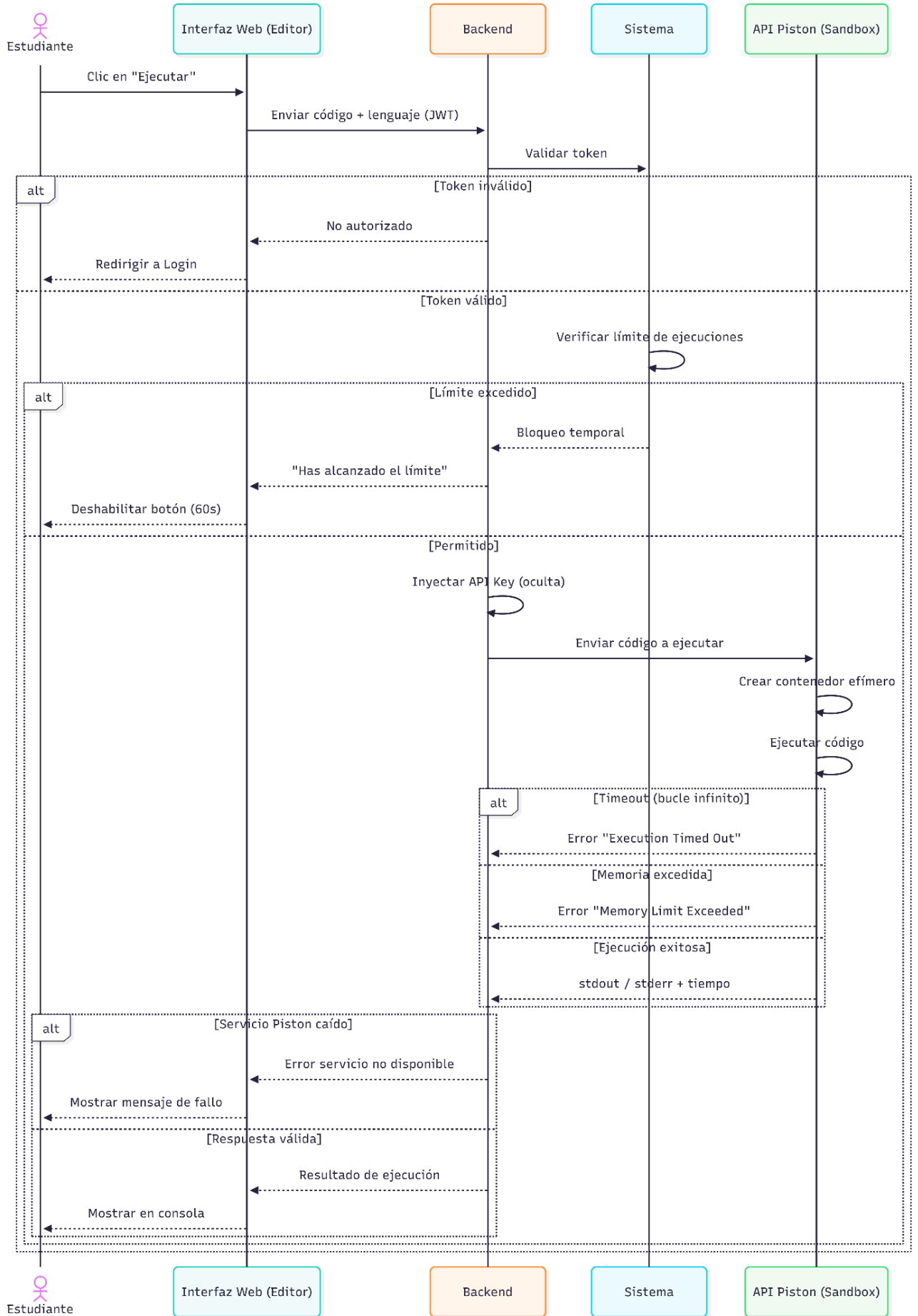


Módulo 3: Ejecución de Código y Sandbox

CU-09: Escribir código en editor (uso de editor enriquecido Monaco/CodeMirror).

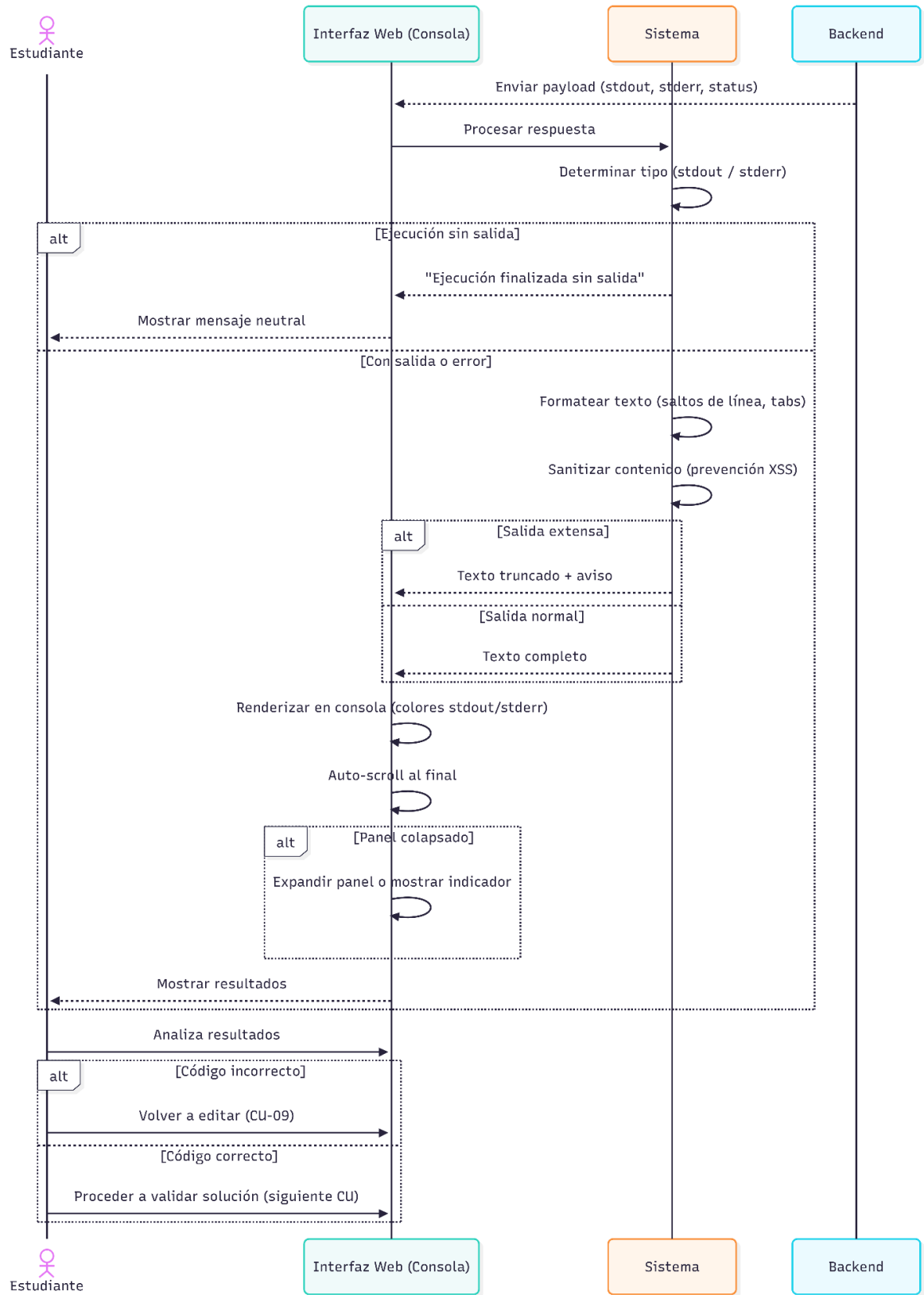


CU-10: Ejecutar código en Sandbox (conexión segura con API Piston).





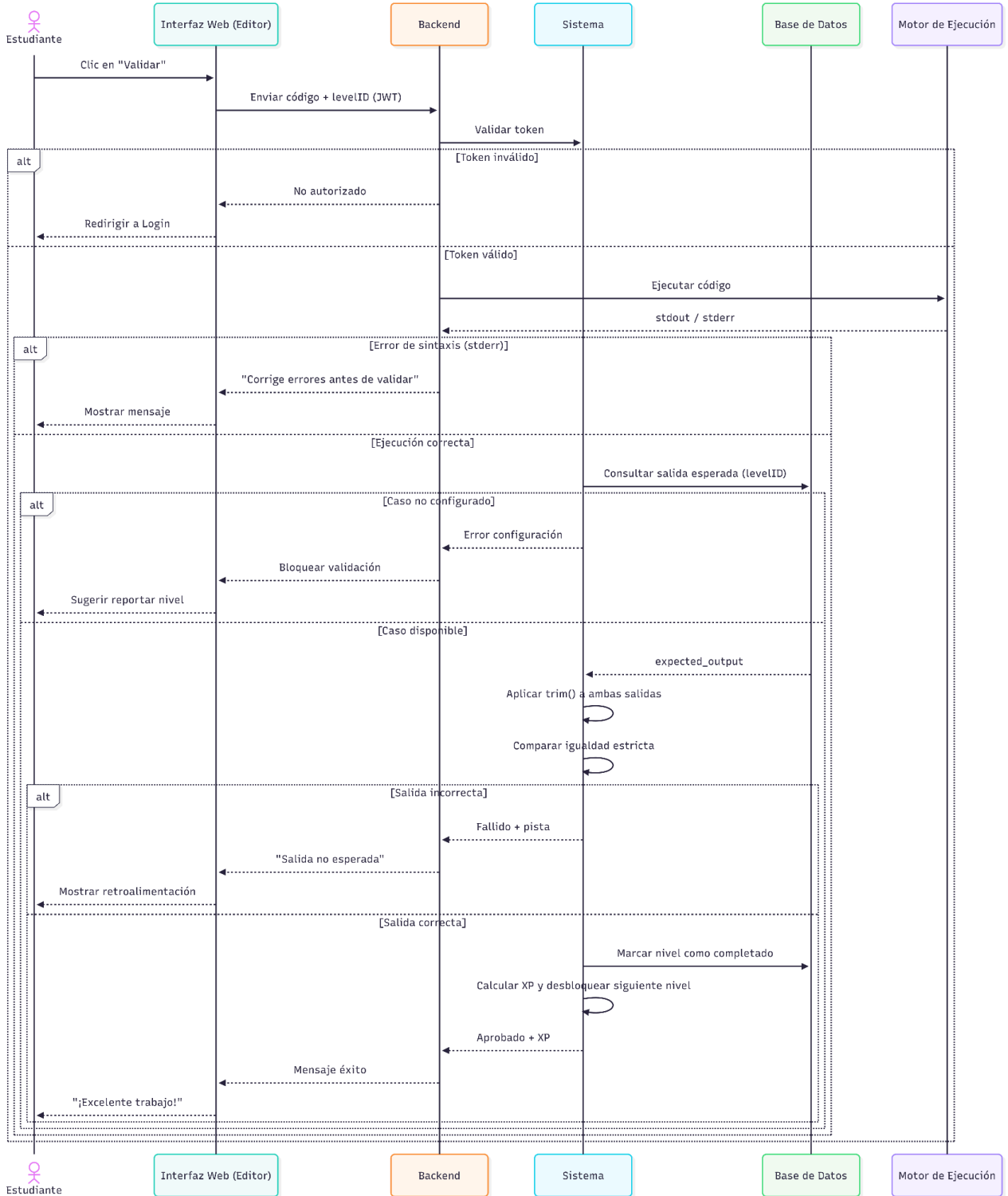
CU-11: Visualizar resultados en consola (diferenciación entre stdout y stderr).





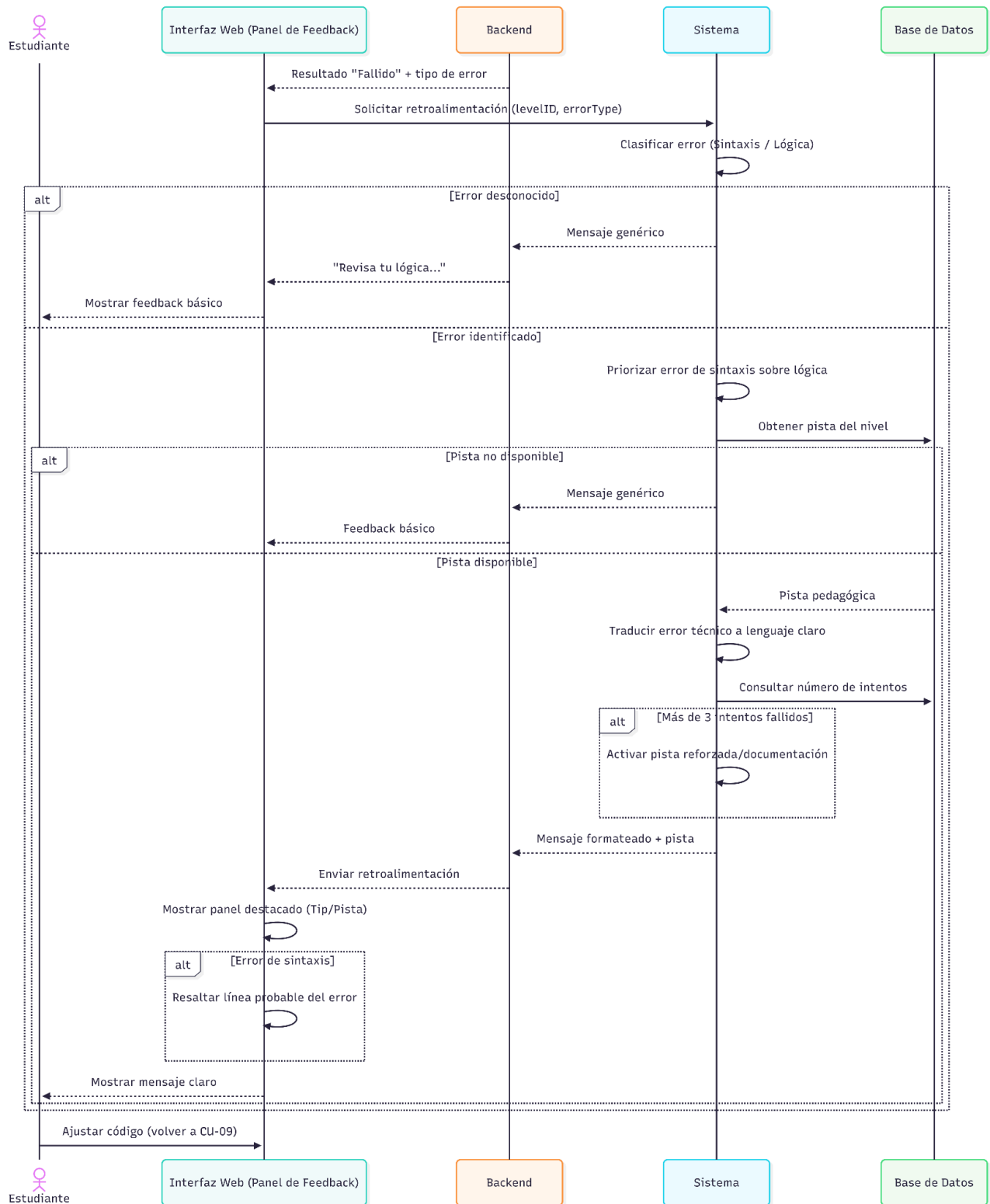
Módulo 4: Evaluación y Retroalimentación

CU-12: Validar solución (comparación automática con casos de prueba).





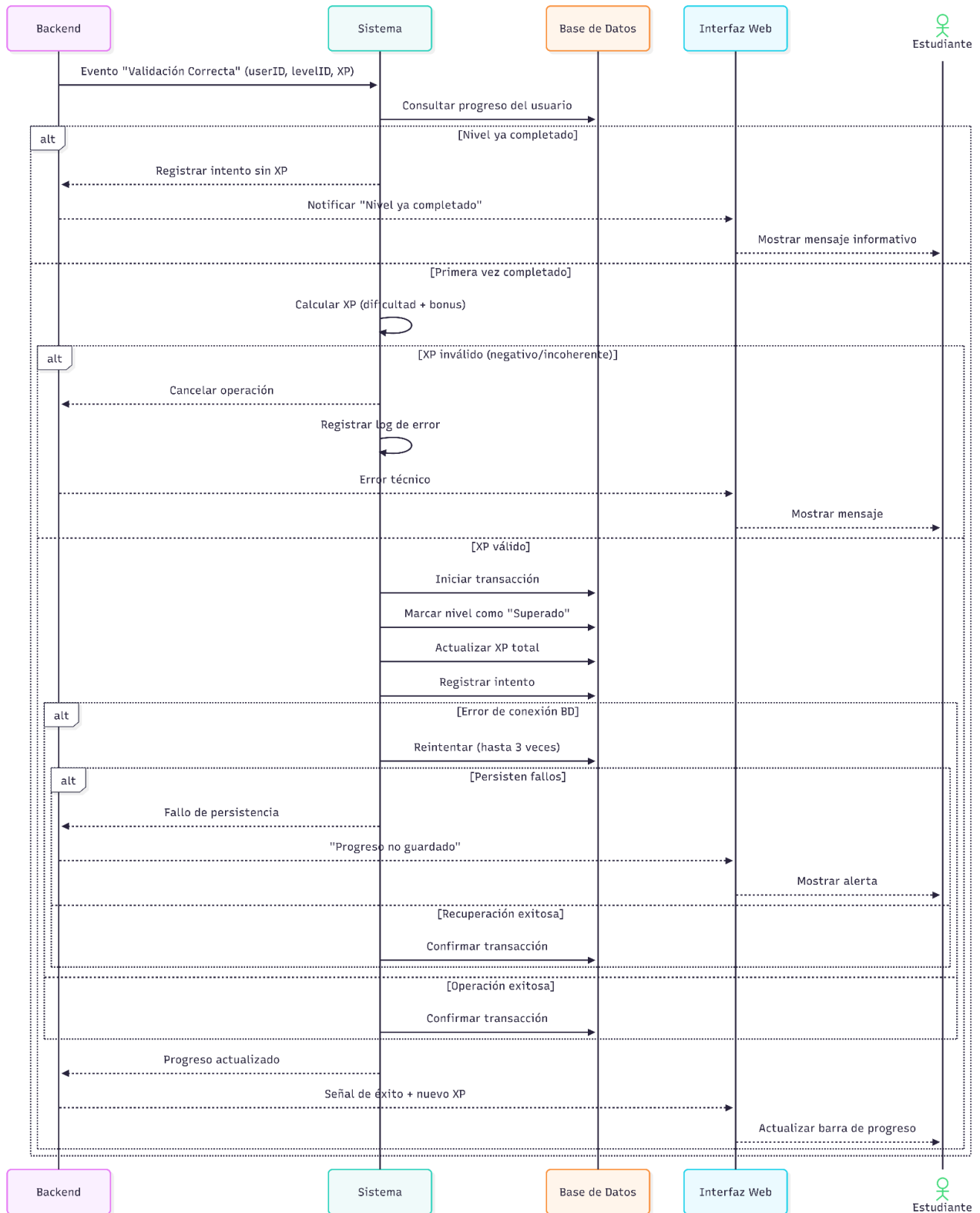
CU-13: Recibir retroalimentación (pistas dinámicas en caso de error).



Módulo 5: Persistencia y Gamificación (Procesos Automáticos)

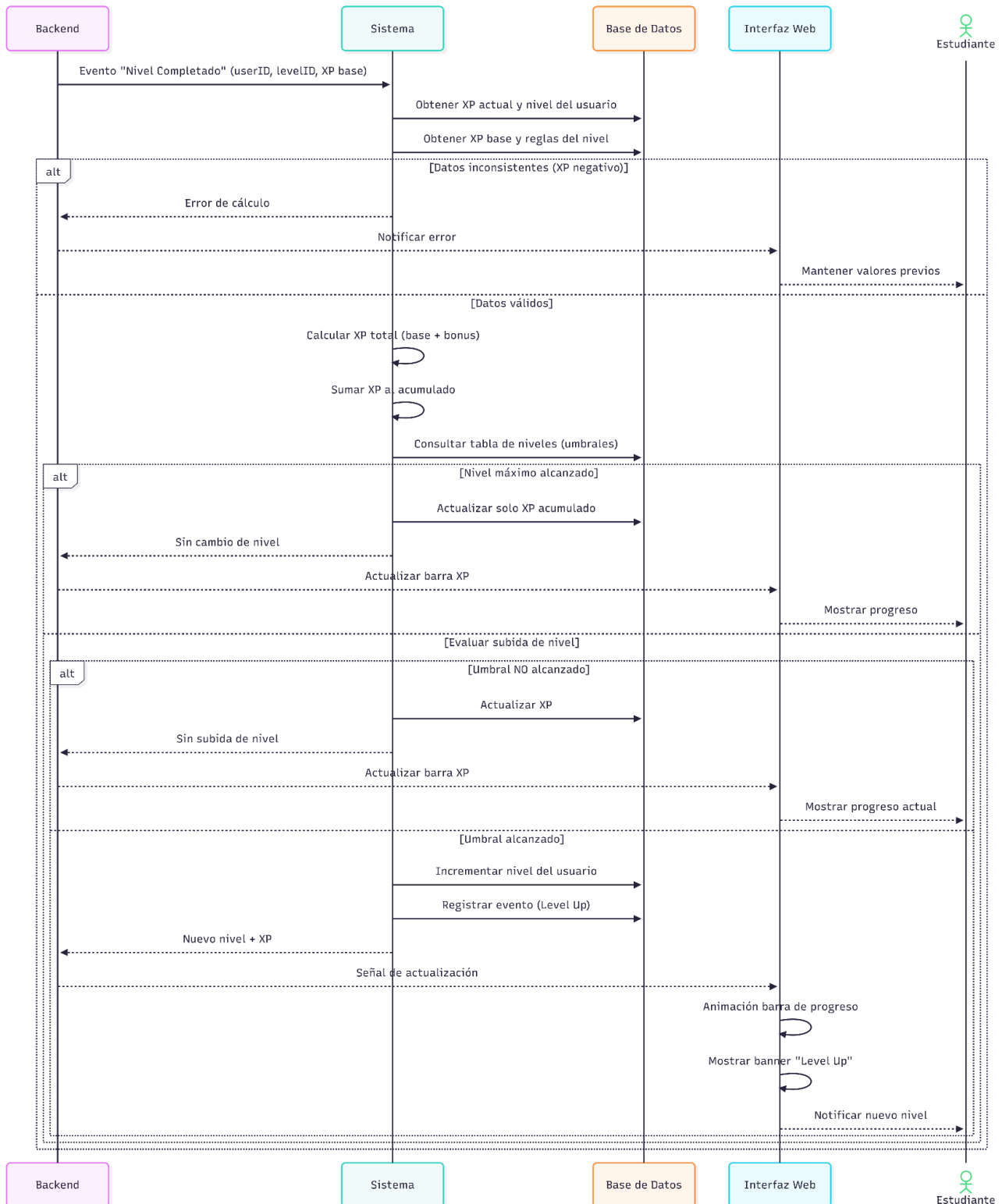


CU-14: Guardar progreso (sincronización con la base de datos MySQL).

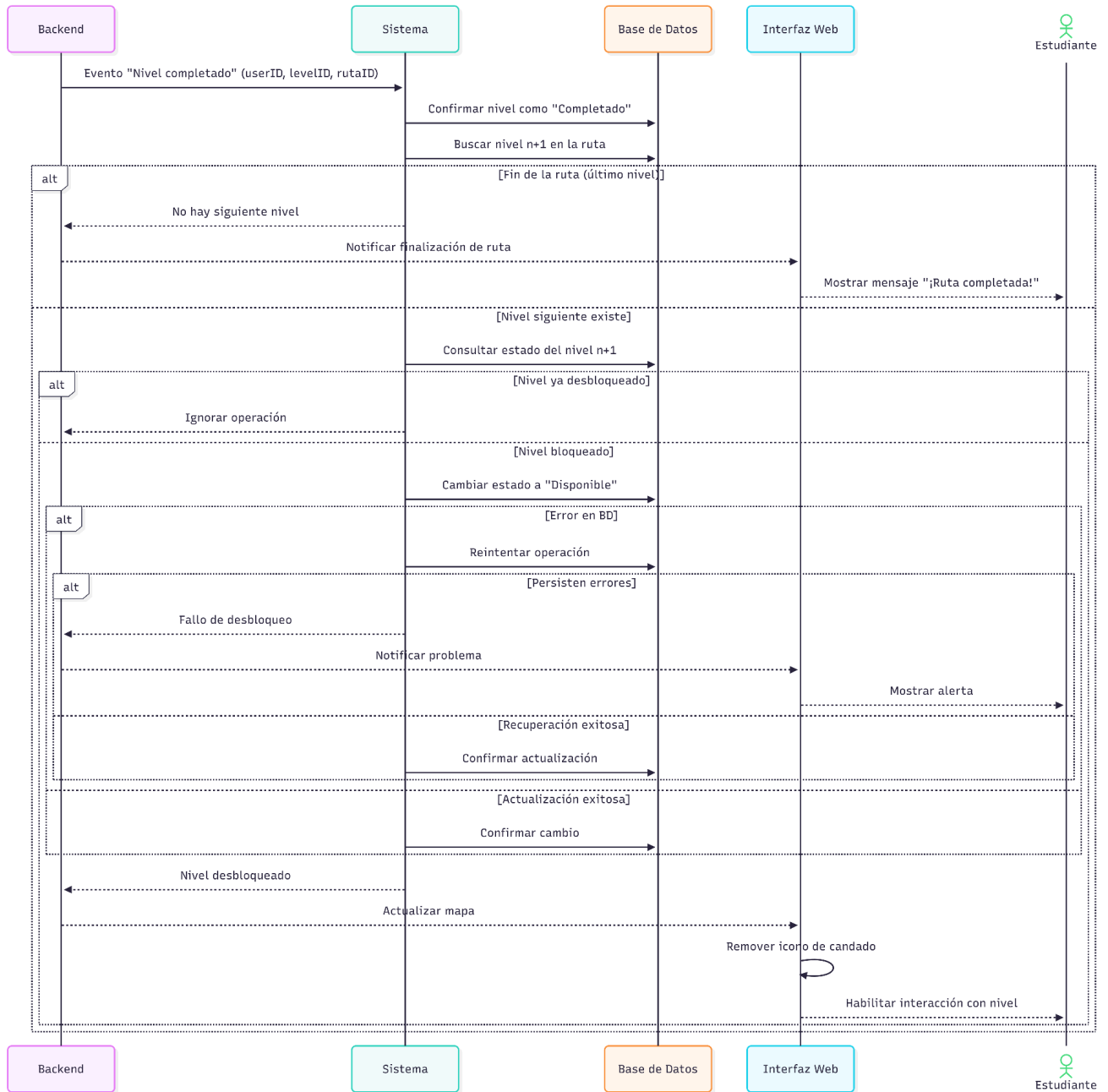




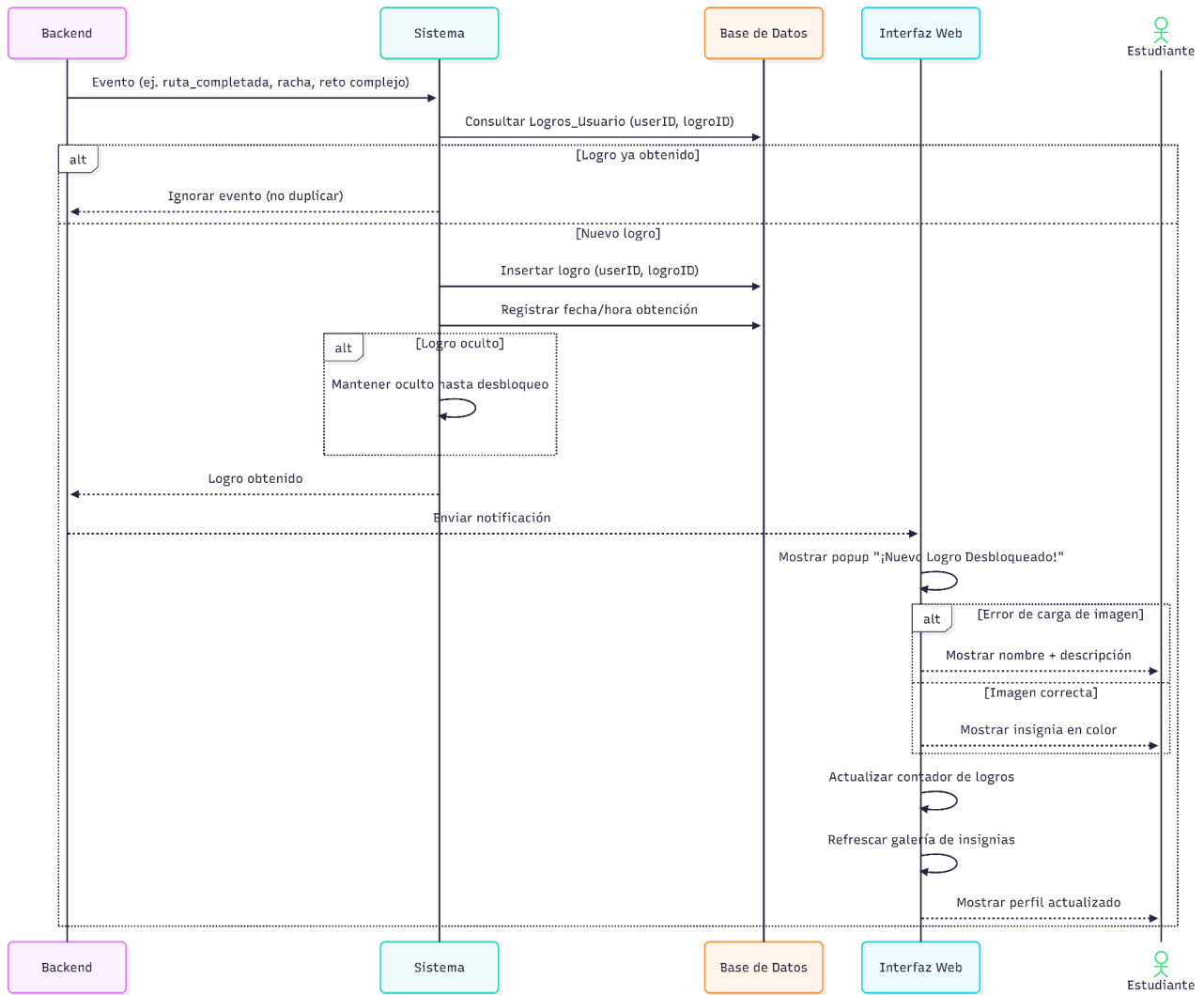
CU-15: Ganar puntos de experiencia (XP) y subir de nivel.



CU-16: Desbloquear niveles secuenciales.

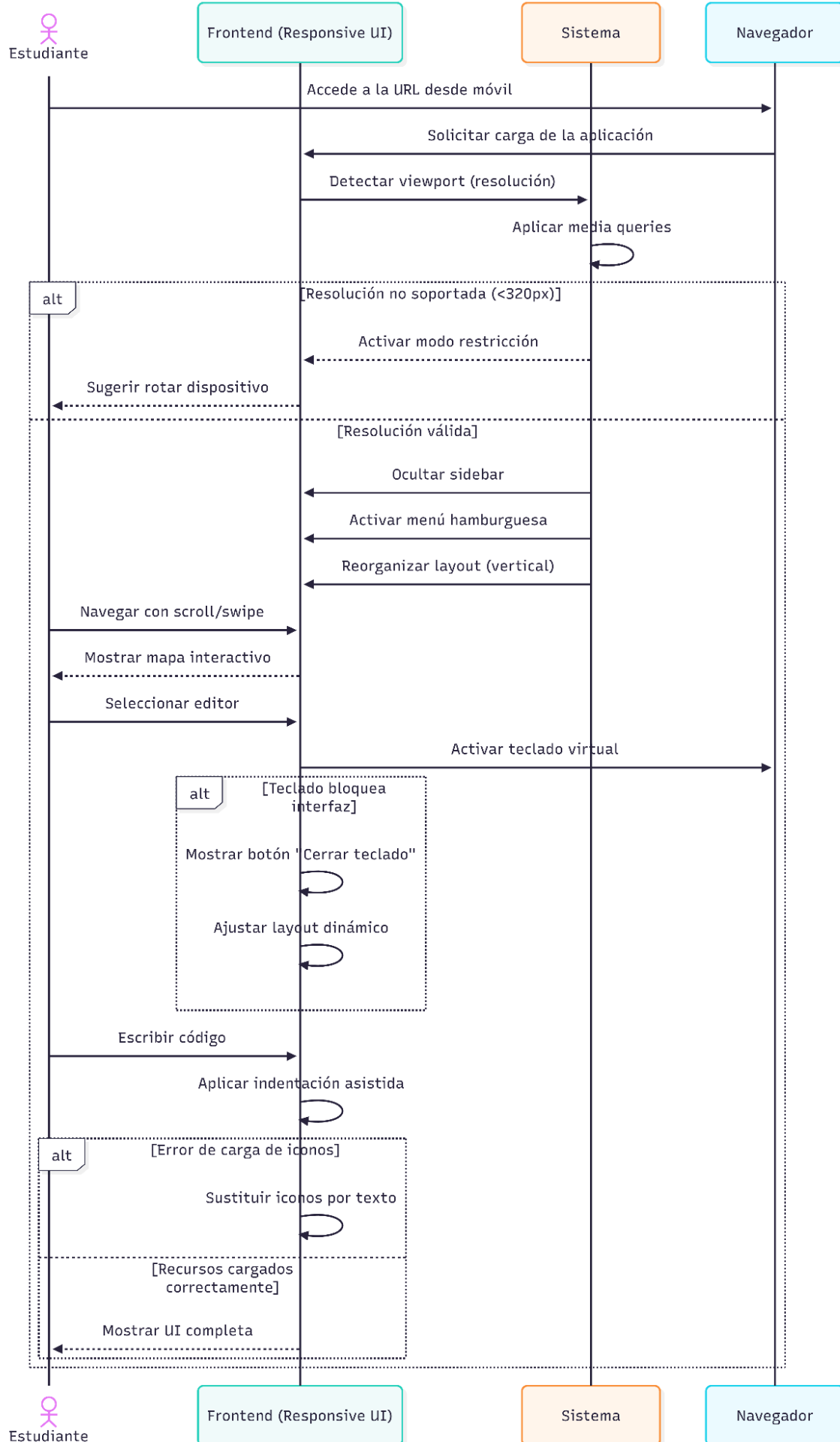


CU-17: Obtener insignias por logros.



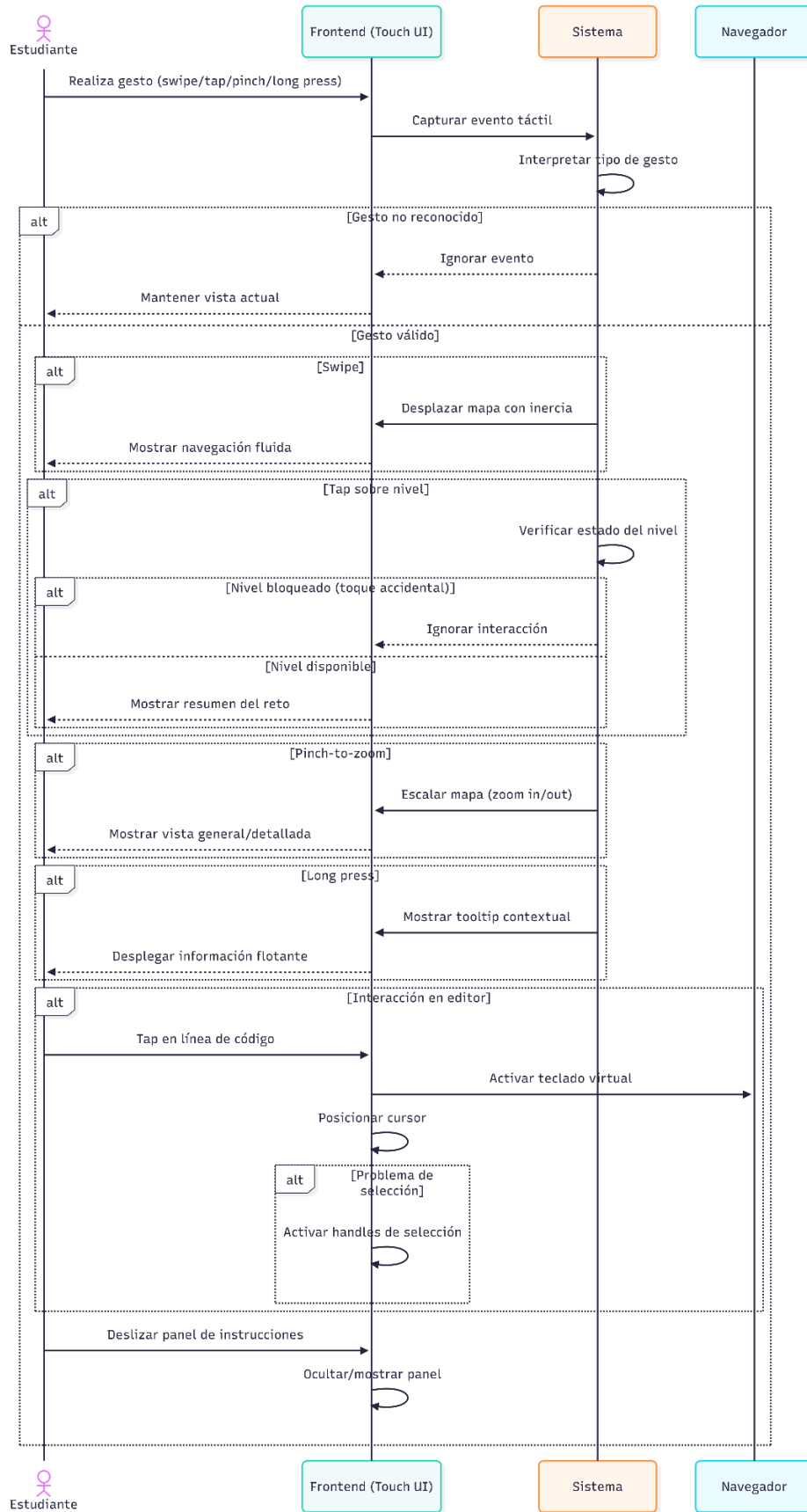
Módulo 6: Experiencia de Usuario (Adaptabilidad)

CU-18: Adaptar interfaz a dispositivos móviles (Responsividad).





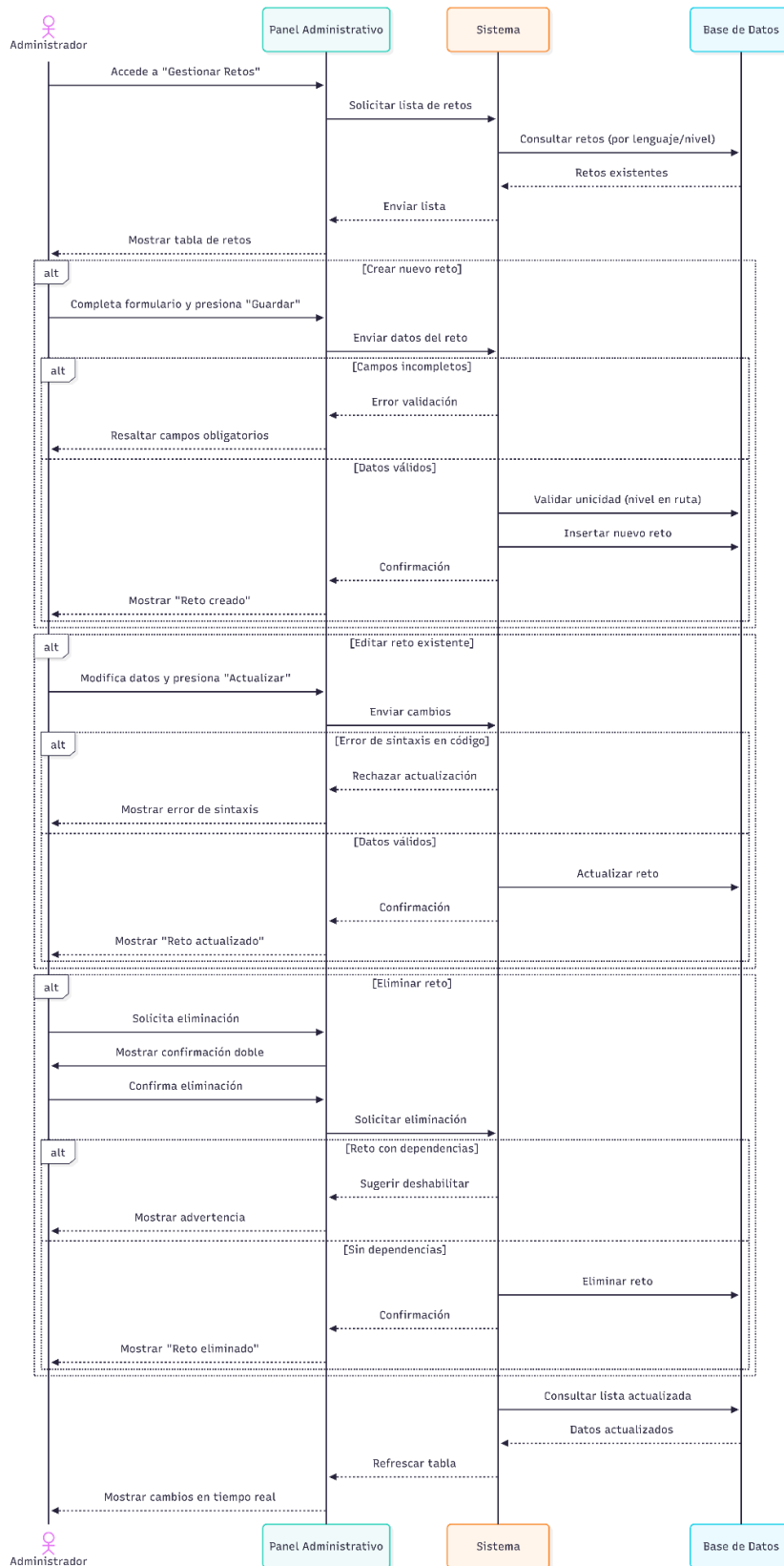
CU-19: Navegar mediante interfaz táctil (gestos en móviles).





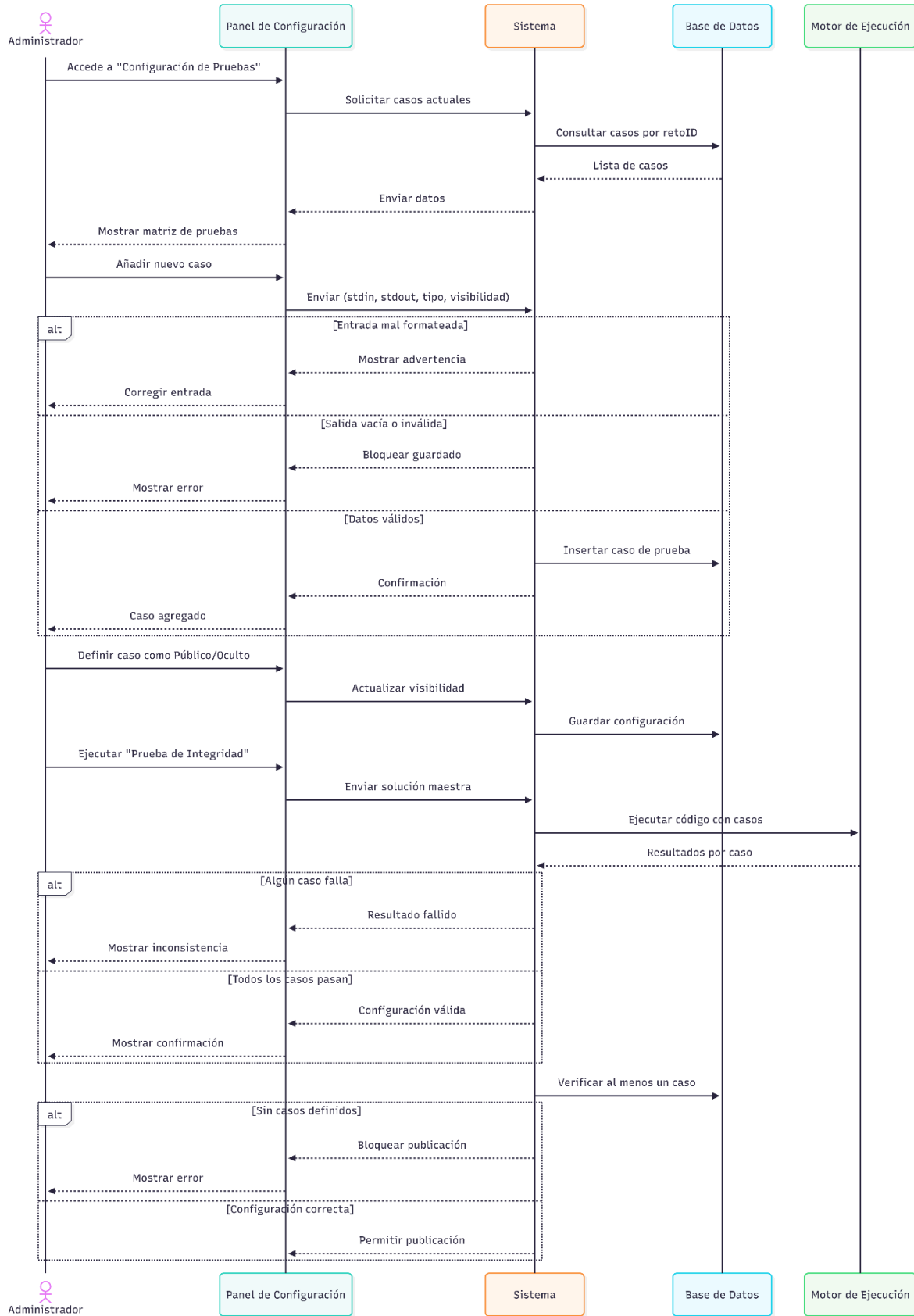
Módulo 7: Administración de Contenido (Rol Docente/Admin)

CU-20: Gestionar retos (CRUD: Crear, Leer, Actualizar, Eliminar ejercicios).



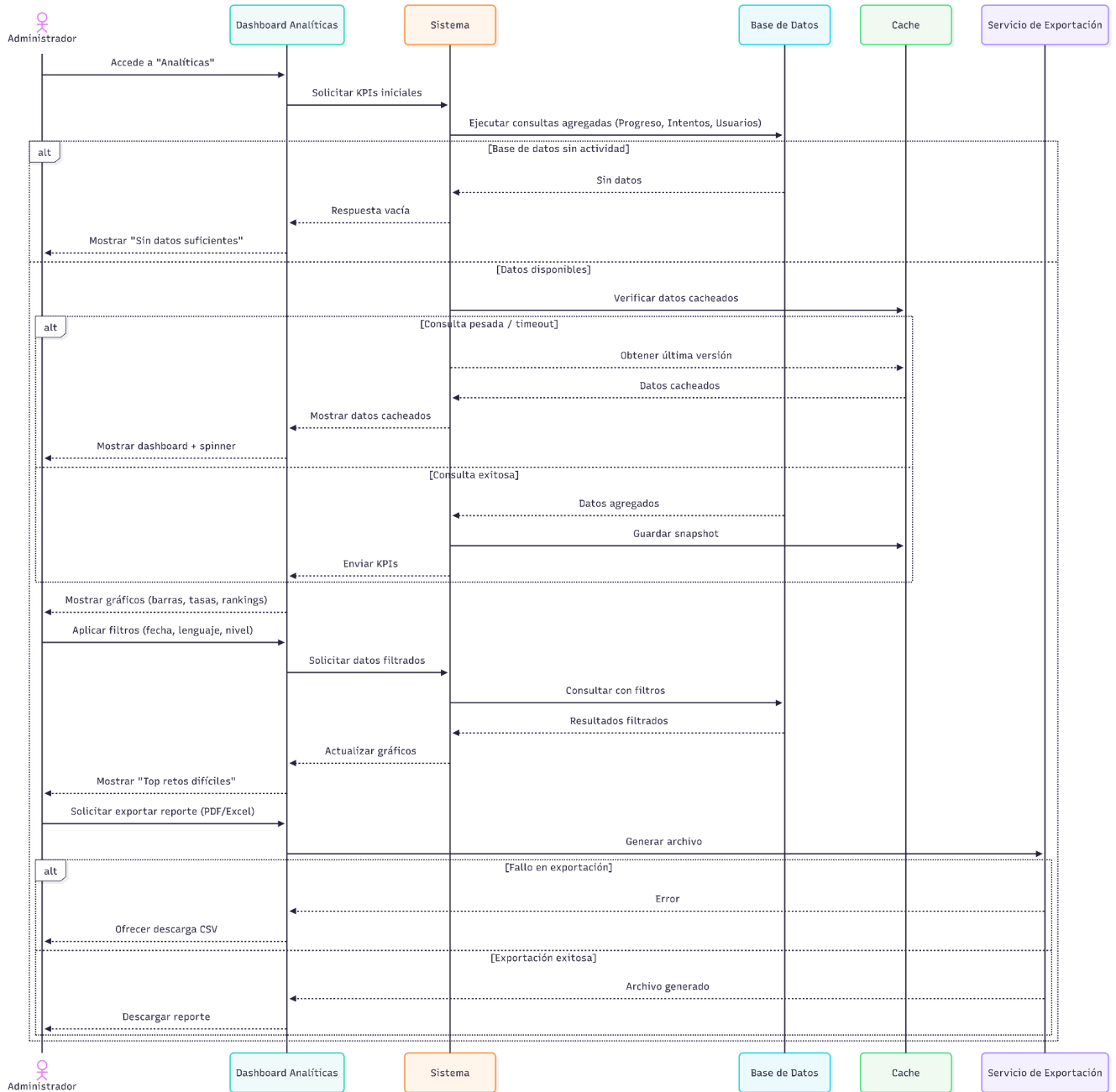


CU-21: Configurar casos de prueba y lenguajes soportados.

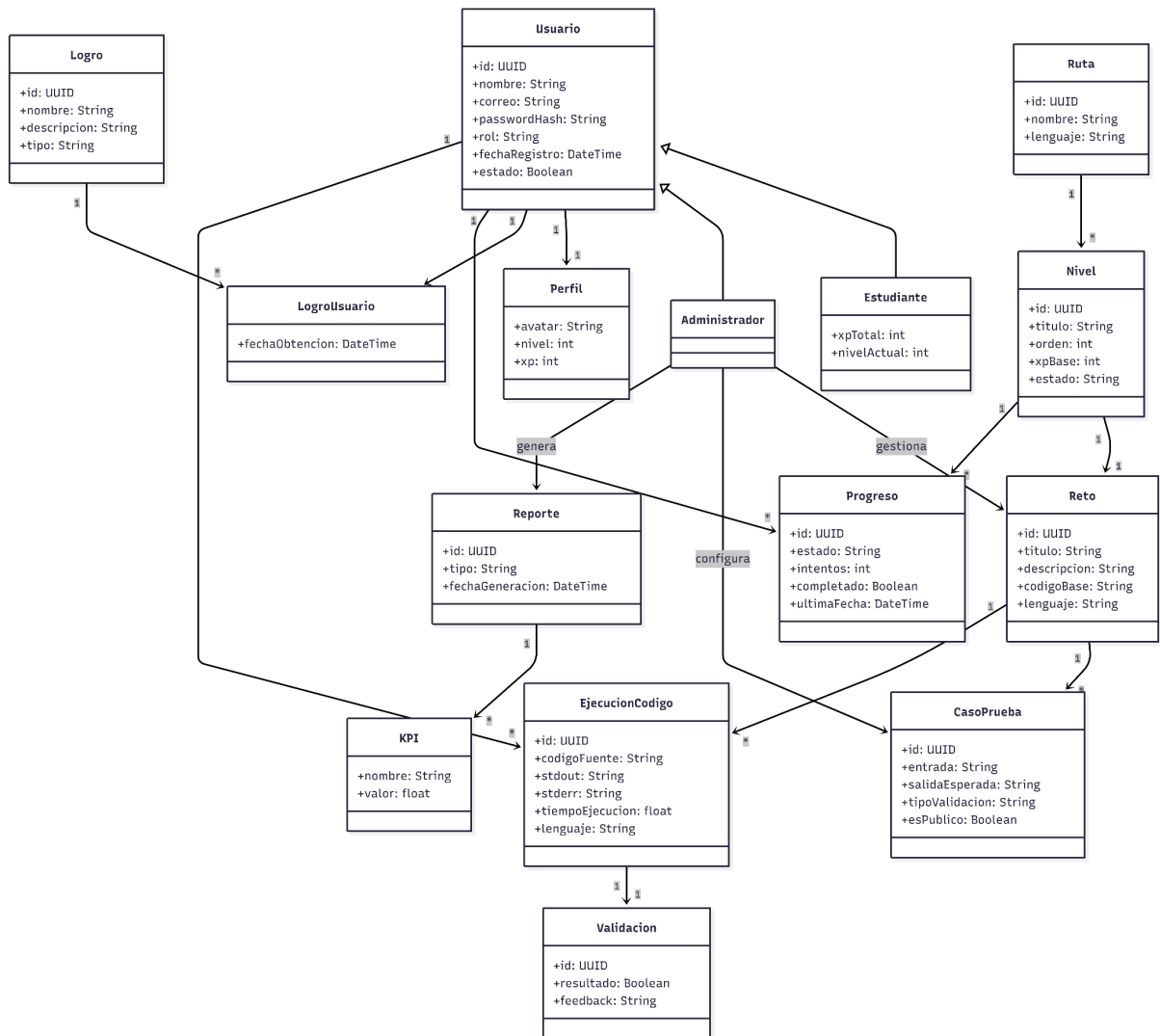




CU-22: Visualizar analíticas de desempeño estudiantil (reportes).



3.5. Diagrama de Clases



CONCLUSIONES

- **CodeBreaker** representa una solución innovadora para digitalizar y modernizar la enseñanza de la programación, atendiendo a las necesidades específicas de estudiantes, docentes y administradores académicos de la **UPT**.
- La plataforma centraliza y automatiza procesos clave como la **ejecución de scripts**, la **validación automática de ejercicios** y el **seguimiento del progreso académico**, lo que reduce la carga administrativa del docente y ofrece un feedback inmediato al estudiante, optimizando el ciclo de aprendizaje.
- El uso de tecnologías modernas y escalables (**Node.js, Express, React**) junto con la integración de la **API de Piston** garantiza un entorno de ejecución seguro (Sandbox), eficiente y multiplataforma, que permite a los usuarios practicar diversos lenguajes de programación sin necesidad de configurar entornos locales complejos.



- El cumplimiento estricto de la **Ley N° 29733 (Ley de Protección de Datos Personales en Perú)** y la implementación de estándares de seguridad en el manejo de código garantizan un entorno digital confiable, protegiendo la integridad de la información institucional y del alumnado.
- La interfaz web accesible y el editor de código integrado simplifican la experiencia de usuario, permitiendo que estudiantes con diferentes niveles de competencia técnica y limitaciones de hardware puedan desarrollar sus habilidades lógicas y técnicas de manera equitativa.
- En conjunto, **CodeBreaker** promueve la democratización de la educación tecnológica y la transformación digital dentro de la facultad, aportando una herramienta de alto valor académico que prepara a los futuros ingenieros para los retos del mercado de software global.

RECOMENDACIONES

REFERENCIAS

- Project Management Institute. (2017). *PMBOK® Sixth Edition*.
- Pressman, R. S. (2020). *Ingeniería de Software: Un enfoque práctico*. McGraw-Hill.
- Sommerville, I. (2019). *Software Engineering*. Pearson.
- SUNAT. (2023). *Normativa de comprobantes electrónicos en el Perú*.
- ISO/IEC 25010:2011 – *Software Product Quality Requirements and Evaluation*.
- Ley N.º 29733, Ley de Protección de Datos Personales, Perú.
- Documentación técnica oficial de MySQL, Node.js y Express .

BIBLIOGRAFÍA

-